A PROJECT REPORT

ON

"ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform"

A project report submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

Submitted By
Faheemuddin Sayyed (PRN: 23070122196)
Sanidhya Awasthi (PRN: 23070122192)
Raghav Sonchhatra (PRN: 23070122172)

UNDER THE GUIDANCE OF

Dr. Payal Kadam
Professor



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Pune – 412115, Maharashtra State, India
https://www.sitpune.edu.in/

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AY 2026-27

॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India

https://www.sitpune.edu.in/

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the Project work entitled "**ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform**" is carried out by **Faheemuddin Sayyed (23070122196), Sanidhya Awasthi (23070122192) and Raghav Sonchhatra (23070122172),** in partial fulfillment for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering**, Symbiosis International (Deemed University), Pune during the academic year 2026-2027.

Dr. Payal Kadam	Dr. Sonali Kothari
Project Guide	Head, Department of CSE

DECLARATION

We hereby declare that the project titled "**ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform**" submitted to Symbiosis Institute of Technology, Constituent of Symbiosis International (Deemed University) Pune for the award of the degree of Bachelor of Technology in Computer Science & Engineering is a result of original research carried out by us. We understand that the report may be made electronically available to the public. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of any degree or diploma.

Name(s) of Student(s)	:	Faheemuddin Sayyed, Sanidhya Awasthi, Raghav Sonchhatra
PRN	:	23070122196, 23070122192, 23070122172
Degree	:	Bachelor of Technology in CSE
Department	:	Computer Science & Engineering
Title of the project	:	ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

_____

(Signatures of all the Students of the project group)

Date:

ACKNOWLEDGEMENT

We want to express our sincere gratitude to everyone who supported us throughout this project. First and foremost, we thank our project guide, Dr. Payal Kadam, for valuable guidance, encouragement and feedback at every review. Dr. Kadam's questions about what the system could actually demonstrate shaped both the design of ForgeFlow and the way we report its results.

We would also like to thank the Head of the Department, Dr. Sonali Kothari, for providing the facilities and resources needed for this project, and for constant support and advice.

We thank the faculty members who reviewed our progress and our invention disclosure; their comments made our claims more precise. We also thank our families and friends for their care and support, and for their patience during the many evenings spent debugging deployments.

Lastly, we thank Symbiosis Institute of Technology, Pune for giving us the opportunity to work on this project and to enhance our skills and knowledge. We are proud to be a part of this institution.

ABSTRACT

AI coding assistants and prompt-to-app builders have made it easy to generate a web application, but verifying it is still hard. Recent studies show that AI-generated code has maintainability and security flaws, that repeatedly asking a model to fix its own code can make it less secure, that fewer than one in three model-generated infrastructure deployments succeed on the first attempt, and that unbounded repair loops keep spending budget long after they stop helping.

ForgeFlow is an AI-native, human-in-the-loop Progressive Web App that takes a web-app idea, given as screenshots or a structured specification, through requirements capture, UI design, full-stack code generation on a fixed React, Express and MongoDB stack, unit and end-to-end tests generated from the requirements, a self-healing repair loop, cloud deployment that adapts to the app's infrastructure needs, and validation against the live URL. The repair loop is diff-aware and bounded: it sees only the failing tests, the code they exercise and the changes since the last passing run, and it stops at an iteration cap, a budget cap, a regression or a lack of progress, handing the user a clear summary of where it is stuck. No stage is mandatory; users can enter, refine, skip or revisit any stage. Generated code runs only in isolated per-project Docker sandboxes, and an in-browser IDE keeps that code visible and editable.

The prototype comprises about 50,000 lines of production code built over 65 phases and more than 2,000 automated tests. It is delivered through a six-job CI pipeline and a health-checked continuous deployment to Oracle Cloud and Vercel, and it includes a benchmark harness that measures first-pass versus post-repair success, repair convergence and cost; through its own model interface it reaches 97.0% pass@1 on HumanEval (99.4% with bounded repair) and 92.7% on the BFCL function-calling benchmark. The project has produced an IEEE conference paper (under review), an invention disclosure submitted for filing and a submission to the IBM Bob 2.0 Hackathon.

Keywords: AI-native software engineering; large language model agents; automated program repair; self-healing software; human-in-the-loop systems; requirements-driven test generation; DevOps; progressive web application

TABLE OF CONTENTS

		Page
Certificate		i
Declaration		ii
Acknowledgement		iii
Abstract		iv
Table of Contents		v
List of Tables		vii
List of Figures		viii
List of Abbreviations		x
CHAPTER 1: INTRODUCTION		1
1.1	Introduction	1
1.2	Problem Statement	4
1.3	Scope of the Project	4
1.4	Research Hypothesis	6
1.5	Objectives	6
1.6	Mapping to Sustainable Development Goals	7
1.7	Project Team and Work Distribution	8
1.8	Organization of the Report	9
CHAPTER 2: LITERATURE REVIEW		10
2.1	Background	10
2.2	Summary of Literature Review and Research Gap	16
CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION		21
3.1	Software Tools, Platform and Frameworks Used	21
3.2	Hardware Requirements	24
3.3	Work Breakdown Structure	24
3.4	Functional Requirements	25
3.5	Non-Functional Requirements	28
3.6	Project Cost Estimation	29
3.7	Project Plan and Schedule	29
CHAPTER 4: METHODOLOGY		32
4.1	Overview of the Proposed Solution	32
4.2	System Architecture	33
4.3	Non-Linear Orchestration	36
4.4	Data Design	39
4.5	Class Design	41
4.6	Requirements and Design Stages	43

4.7 Build Stage 45
4.8 Requirements-Driven Test Generation and Execution 48
4.9 The Bounded, Diff-Aware Self-Healing Repair Loop 49
4.10 Deployment and Live Validation 53
4.11 Sandbox, Security and Threat Model 55
4.12 LLM Transport, Cost Discipline and Configuration 57
4.13 User Interface 58
4.14 Platform Delivery: CI/CD, Containerisation, Orchestration and Monitoring 63
4.15 End-to-End Hero Path 66
CHAPTER 5: RESULTS AND DISCUSSION 68
5.1 Testing Strategy 68
5.2 Test Cases and Results 69
5.3 Automated Test-Suite Results 71
5.4 Debugging: Defects Found by End-to-End Validation 73
5.5 Platform Delivery Verification 74
5.6 Performance and Bounds 75
5.7 Evaluation Harness and Benchmark Protocol 76
5.8 Benchmark Results 78
5.9 Project Outcomes 84
5.10 Summary of Results and Discussion 85
CHAPTER 6: CONCLUSION AND FUTURE SCOPE 87
6.1 Conclusion 87
6.2 Limitations 88
6.3 Future Scope 89
REFERENCES (APA FORMAT) 90
APPENDICES

A AIC Form 95
B Similarity Report 96
C AI Plagiarism Report 97
D Research Paper 98
E Project Competition: IBM Bob 2.0 Hackathon 99
F Other Project Outcomes 100

LIST OF TABLES

		Page
Table 1.1	Mapping of ForgeFlow to the Sustainable Development Goals	7
Table 1.2	Mapping of the project to the Programme Outcomes	8
Table 1.3	Group members and their contributions	9
Table 2.1	Summary of the literature review	16
Table 2.2	Capability comparison across classes of systems	18
Table 2.3	Identified gaps and the corresponding ForgeFlow design response	19
Table 3.1	Platform software stack and the specific use of each tool	21
Table 3.2	Fixed stack of every generated application	22
Table 3.3	External services and DevOps tooling	23
Table 3.4	Hardware requirements	24
Table 3.5	Functional requirements	26
Table 3.6	Non-functional requirements	28
Table 3.7	Project cost estimation	29
Table 3.8	Planned versus achieved milestones	31
Table 4.1	Data dictionary of the main control-plane collections	40
Table 4.2	Default bounds of the repair loop and the build (all admin-tunable)	53
Table 4.3	Main threats and mitigations from the threat model	56
Table 5.1	Levels of testing used in the project	68
Table 5.2	Representative test cases and results	69
Table 5.3	Automated test-suite results	71
Table 5.4	Defects found in live use, their root causes and fixes	73
Table 5.5	Verification of the delivery pipeline and infrastructure	75
Table 5.6	Worst-case bounds and measured performance	76
Table 5.7	Benchmark specifications	77
Table 5.8	Code-generation benchmark results, pass@1 and after bounded repair	79
Table 5.9	BFCL v3 non-live (Python) accuracy, scored by the official AST checker	81
Table 5.10	End-to-end results on the benchmark applications completed (one run each)	82
Table 5.11	Planned and achieved project outcomes	84
Table D.1	Details of the submitted research paper	98
Table E.1	IBM Bob 2.0 Hackathon participation details	99

LIST OF FIGURES

		Page
Figure 1.1	The six ForgeFlow stages from an idea to a live, validated URL	3
Figure 3.1	Work breakdown structure of ForgeFlow	25
Figure 3.2	Use case diagram of ForgeFlow	26
Figure 3.3	Gantt chart of the project, July to October 2026	30
Figure 4.1	System architecture: the presentation, control and execution planes and the external provider boundary	33
Figure 4.2	Level 0 data flow diagram (context diagram)	35
Figure 4.3	Level 1 data flow diagram	36
Figure 4.4	The non-linear stage model	37
Figure 4.5	State diagram of a pipeline stage (StageState.status)	38
Figure 4.6	Entity-relationship diagram of the control-plane database (forgeflow_meta)	40
Figure 4.7	Class diagram of the orchestration and repair core	42
Figure 4.8	Class diagram of the provider abstractions and deployment services	43
Figure 4.9	Activity diagram of the build stage	46
Figure 4.10	Requirement-to-repair traceability through criterion_id	48
Figure 4.11	The bounded self-healing repair loop: minimal context assembly, targeted patching, re-execution, termination guards in precedence order, and escalation as a terminal state	49
Figure 4.12	Infrastructure analysis, dual-mode credentials, ordered deployment and live validation with its repair path	54
Figure 4.13	Landing page of the ForgeFlow PWA	58
Figure 4.14	Projects dashboard with per-stage progress	59
Figure 4.15	Project workspace showing the stage navigator and captured requirements	60
Figure 4.16	Guided requirements form with the optional AI draft from a description	60
Figure 4.17	Design stage with text, screenshot and own-design intake	61
Figure 4.18	Deploy stage with dual-mode selection and the deployment topology	61
Figure 4.19	Provider credential vault in Settings	62
Figure 4.20	Admin dashboard: repair-loop bounds with their sources and allowed ranges	62
Figure 4.21	Production deployment of the platform: GitHub Actions, GHCR and Vercel, and the Oracle Cloud VM running the production Compose stack, the k3s demonstration and the monitoring stack	63

Figure 4.22	CI/CD pipeline: CI must be fully green before CD builds, pushes, deploys, verifies and ships the frontend; an unhealthy release is rolled back automatically	64
Figure 4.23	Rolling update with maxSurge 1 and maxUnavailable 0, and the broken-release path that ends in an automatic rollback	65
Figure 4.24	Monitoring data flow: targets scraped by Prometheus and visualised in Grafana	66
Figure 4.25	Sequence diagram of the hero path	67
Figure 5.1	Distribution of automated test definitions by module (2 October 2026)	72
Figure 5.2	Growth of the automated test suite (test definitions)	73
Figure 5.3	Industry benchmark results: code generation with and without bounded repair (top) and BFCL tool-calling accuracy by category (bottom)	80

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
APA	American Psychological Association
AST	Abstract Syntax Tree
BFCL	Berkeley Function-Calling Leaderboard
BYO	Bring Your Own (credentials)
CD	Continuous Deployment / Delivery
CI	Continuous Integration
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
DNS	Domain Name System
DORA	DevOps Research and Assessment
E2E	End-to-End
ER	Entity-Relationship
FE / BE	Frontend / Backend
FR	Functional Requirement
GHCR	GitHub Container Registry
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IaC	Infrastructure as Code
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
K8s	Kubernetes
LLM	Large Language Model
MBPP	Mostly Basic Programming Problems (benchmark)
MCP	Model Context Protocol
MSR	Mining Software Repositories
NFR	Non-Functional Requirement
OCI	Oracle Cloud Infrastructure

OCPU	Oracle CPU
ODM	Object Document Mapper
PO	Programme Outcome
pp	Percentage points
PRN	Permanent Registration Number
PWA	Progressive Web App
RAM	Random Access Memory
REST	Representational State Transfer
SDG	Sustainable Development Goal
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SPA	Single-Page Application
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
URI / URL	Uniform Resource Identifier / Locator
VM	Virtual Machine
WBS	Work Breakdown Structure

Chapter 1
Introduction

This chapter introduces ForgeFlow and the problem it addresses. It gives the motivation, problem statement and scope of the project, states its research hypotheses and four main objectives, maps it to the Sustainable Development Goals and Programme Outcomes, and describes the team's work distribution and the organisation of the report.

1.1 Introduction

Large language models (LLMs) have moved from autocompleting single lines of code to writing whole applications. A recent survey of LLMs in software engineering systematises 926 studies covering 112 tasks across all five phases of the software development lifecycle (Q. Zhang et al., 2026), and a survey of "vibe coding" describes a growing class of tools in which a natural-language request is turned directly into a running application (Ge et al., 2025). Commercial prompt-to-app builders now let a person with no programming background describe an idea and receive a deployed website within minutes.

Adoption has followed. In the 2025 Stack Overflow Developer Survey, 84% of more than 49,000 respondents said they use or plan to use AI tools in their development process, up from 76% a year earlier, and 51% of professional developers use them every day (Stack Overflow, 2025). The DORA 2025 report finds that 90% of about 5,000 technology professionals now use AI at work (DORA, 2025). Trust has not kept pace with this adoption. In the same Stack Overflow survey, 46% of developers distrust the accuracy of AI output while only 33% trust it and just 3% trust it highly; 66% name "almost right, but not quite" as their biggest frustration, and 45% say that debugging AI-generated code takes longer than writing it themselves. In a randomised controlled trial, experienced open-

source developers took 19% longer to finish tasks when AI was allowed, while believing they had been about 20% faster (Becker et al., 2025).

The cost of this gap now shows up after code is shipped. In an industry survey, 81% of enterprise technology leaders reported more production incidents caused by AI-generated code, and 70% said that maintaining test suites had become a bigger burden than writing the code (CloudBees, 2026). When frontier models were asked to generate cloud infrastructure for 153 real-world scenarios, only 20.8% to 30.2% of their outputs deployed successfully on the first attempt (T. Zhang et al., 2026). A benchmark of agent-written solutions to real feature requests found that 57% worked but only 11.8% were secure (Zhao et al., 2026), and asking a model repeatedly to make its code more secure can weaken security rather than improve it (Shukla et al., 2025).

Two observations follow from this evidence. First, generating an application is no longer the hard part; knowing whether the generated application actually works, before and after it is deployed, is. Second, the obvious remedy, letting the model keep fixing its own output until the tests pass, has its own failure mode. Empirical studies of LLM repair loops show that most of the achievable gain arrives in the first few iterations and that later iterations mostly consume tokens (Kiecker et al., 2026; Arimbur, 2026). An unbounded "keep fixing it" loop wastes budget and, as noted above, can make the code worse.

ForgeFlow is the system built in this project to address both observations. It is an AI-native, human-in-the-loop Progressive Web App (PWA) that takes a web-application idea, given as screenshots or a structured specification, through six stages: requirements, design, build, test with self-healing repair, deployment, and validation of the live site (Figure 1.1). Every generated application uses one fixed, well-understood stack: React with Vite and TypeScript for the frontend, Node.js with Express and TypeScript for the backend, and

MongoDB through Mongoose for data. Tests are generated from the user's own acceptance criteria, failures are repaired by a loop that is deliberately bounded and fed only the context it needs, and the same end-to-end tests are run again against the deployed URL.

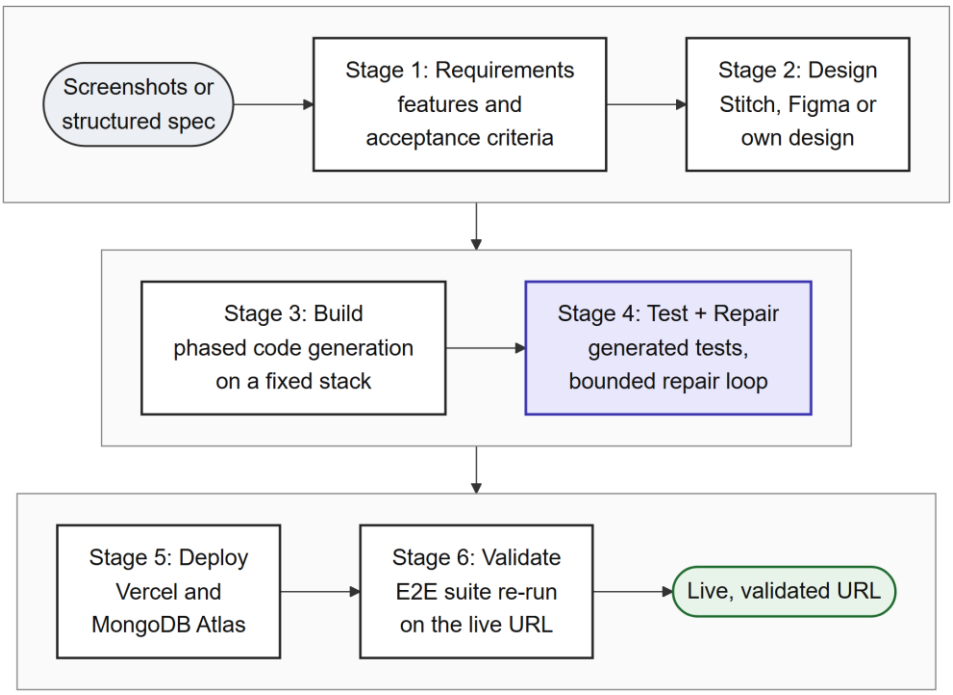


Figure 1.1: The six ForgeFlow stages from an idea to a live, validated URL. Every stage can be refined, skipped or revisited; only deploy after build and validate after deploy are enforced

The human stays in control throughout. No stage is mandatory: a user can start at any stage, refine any output through conversation, skip a stage (and undo the skip), or jump back to an earlier stage, in which case later stages are marked stale rather than deleted. Generated code runs only inside an isolated Docker sandbox created for each project, and an in-browser IDE with an editor, a terminal and a live preview keeps that code visible and editable instead of hiding it behind a chat window.

The project was carried out between July and October 2026 as 65 independently shippable phases grouped into 14 epics, each with a written specification and acceptance criteria. The platform itself is delivered through a six-job continuous integration pipeline and a health-checked continuous deployment to an Oracle Cloud virtual machine and Vercel.

1.2 Problem Statement

Current AI app-builders can generate and deploy web applications, but they cannot reliably verify them. Users get stuck in costly bug-fix loops with no end, nobody checks whether the deployed site actually works, and beginners have no safe way through cloud infrastructure. The problem addressed by this project is to design and build a human-in-the-loop platform that takes a web-app idea (from screenshots or a structured specification) through six stages, namely requirements, design, build, testing with bounded self-healing repair, deployment, and validation of the live site, while letting the user refine, revisit or skip any stage.

Four sub-problems make this statement concrete:

1. **Verification, not just generation.** Requirements must compile into executable tests, and test failures must drive a repair process, so that "done" means "the acceptance criteria pass" rather than "the model stopped writing".
2. **A repair process that terminates.** Repair must be bounded by an iteration cap, a budget cap and explicit detection of regressions and stalls, and it must hand the problem back to a human with a clear explanation when it cannot converge.
3. **Truth after deployment.** Passing tests in a development sandbox is not evidence that the deployed site works, so the same tests must be run again against the live URL, and live failures must re-enter the same repair process.
4. **Human control and beginner-safe infrastructure.** Every stage must be optional and revisitable, and the platform must work out what infrastructure an application needs (static frontend, backend, database) and deploy it without the user having to understand cloud configuration.

1.3 Scope of the Project

The scope was fixed early through a set of locked design decisions (recorded in the project's implementation plan) so that the work stayed tractable within one academic term and a development budget of ₹6,000.

1.3.1 In scope

- User authentication, a projects dashboard and a per-project workspace delivered as an installable PWA.
- Six optional pipeline stages: requirements, design, build, test with repair, deploy and live validation.
- A web IDE (Monaco editor, xterm.js terminal and live preview) running against per-project Docker sandboxes.
- Pluggable UI design from screenshots or text through Google Stitch (default) or Figma.
- Agentic generation of the fixed React, Express and MongoDB stack, with conversational change requests.
- Structured requirements compiled into unit and Playwright end-to-end tests, with bounded self-healing repair.
- Infrastructure analysis, dual-mode deployment (platform-owned or bring-your-own credentials) and a visual deployment topology.
- Post-deployment validation against the live URL, with a repair path for live failures.
- A per-project database with an in-app data browser, an evaluation harness, an admin configuration dashboard and cost tracking.

1.3.2 Out of scope

- Non-web targets such as mobile, native or desktop applications, and progressive web apps as generated output.
- More than one generated-application stack.
- Payments in generated applications, team collaboration or multiplayer editing, enterprise single sign-on and compliance, and a template marketplace.

Restricting generated applications to one stack is a deliberate trade-off. A single, opinionated target raises first-pass generation success, reduces the number of repair iterations and makes rigorous verification achievable within the project's time and budget.

1.4 Research Hypothesis

The project is built around three hypotheses. They are stated here as claims to be tested; Section 5.7 describes the evaluation protocol designed to test them, and Section 5.8 reports what has been measured so far.

1. **H1 (bounded repair).** A repair loop with an iteration cap, a stall guard, a regression guard and a budget guard reaches most of the pass-rate improvement of an unbounded loop at a substantially lower token cost, because most repair gains occur in the first few iterations.
2. **H2 (diff-aware context).** Giving the repair model only the failing tests, the source files those tests reach and the diff since the last passing run yields a higher repair success rate per token than giving it broader repository context.
3. **H3 (live validation).** Re-running the generated end-to-end suite against the deployed URL detects failures that sandbox testing alone does not, so post-deployment validation is necessary rather than redundant.

1.5 Objectives

The four main objectives below, set at the first review, guided the phase roadmap:

1. **Non-linear, human-in-the-loop generation.** To build a platform that turns screenshots, prompts or a structured specification into a working full-stack web application on the fixed React, Express and MongoDB stack, in which no stage is mandatory, every artifact is versioned, and untrusted generated code runs only in isolated per-project sandboxes.
2. **Verified, bounded self-healing repair (core contribution).** To compile structured acceptance criteria deterministically into unit and end-to-end tests traceable to each criterion, and to repair failing tests with a diff-aware loop bounded by an iteration cap, no-progress and regression detection and a budget cap, which escalates to the user when it cannot converge.
3. **Infrastructure-aware deployment and live validation.** To classify each application's infrastructure needs, deploy it with platform-owned or the user's own

credentials behind a live component topology, and validate the live URL after every deployment, repairing and redeploying on failure.

4. **Quantitative evaluation.** To measure first-pass versus post-repair success, repair convergence, cost and latency on reproducible benchmarks.

1.6 Mapping to Sustainable Development Goals

This section relates the project to the United Nations Sustainable Development Goals (Section 1.6.1) and to the Programme Outcomes of the National Board of Accreditation (Section 1.6.2).

1.6.1 Sustainable Development Goals

Table 1.1 maps the project to the United Nations Sustainable Development Goals (SDGs). SDG 9 is the primary goal; SDG 8 is a strong secondary goal; SDG 4 and SDG 12 are supporting goals.

Table 1.1: Mapping of ForgeFlow to the Sustainable Development Goals

Sustainable Development Goal	Strength	Justification
SDG 9: Industry, Innovation and Infrastructure	Primary	Automates the software development lifecycle with AI while keeping it verifiable, and gives beginners a safe, guided path to cloud infrastructure through infrastructure analysis and managed deployment.
SDG 8: Decent Work and Economic Growth	Secondary	Lets startups, small businesses and non-experts ship verified web applications with lower cost and effort, and keeps the human in charge of the work rather than replacing them.
SDG 4: Quality Education	Supporting	Generated code stays visible and editable in an in-browser IDE, so users can see how a full-stack application is built, tested and deployed rather than receiving a black box.
SDG 12: Responsible Consumption and Production	Supporting	Bounded repair, per-project and global budget caps, and role-based model routing reduce wasted compute and avoid runaway token spending.

1.6.2 Programme Outcomes

Table 1.2 maps the project to the twelve Programme Outcomes (POs) of the National Board of Accreditation. PO7 (Environment and Sustainability) is not claimed because the link is weak.

Table 1.2: Mapping of the project to the Programme Outcomes

Programme Outcome	Justification
PO1 Engineering Knowledge	Applies software engineering, databases, distributed systems and AI/LLMs.
PO2 Problem Analysis	The literature survey found that AI app-builders do not verify what they build.
PO3 Design/Development of Solutions	Three-plane architecture: FastAPI control plane, Docker sandboxes and a React PWA.
PO4 Investigation of Complex Problems	Benchmark harness measuring pass rates, repair convergence and cost.
PO5 Modern Tool Usage	LLM APIs, Docker, Playwright, Vercel, MongoDB Atlas, GitHub Actions, Ansible, Kubernetes, Prometheus.
PO6 The Engineer and Society	Makes building software easier for non-experts while the human stays in control.
PO8 Ethics	Sandboxed untrusted code, encrypted secrets, a written threat model and no runaway spending.
PO9 Individual and Team Work	Three-member team with branch, review and merge workflow.
PO10 Communication	IEEE conference paper, three review presentations and this report.
PO11 Project Management and Finance	Phased roadmap, ₹6,000 budget and per-project token cost tracking.
PO12 Life-long Learning	Built on 2025–26 research and new tooling, revisited at every review.

1.7 Project Team and Work Distribution

The project was carried out by a team of three under the guidance of Dr. Payal Kadam. Every change was developed on its own branch, reviewed by another member and then merged. Table 1.3 lists each member's primary contributions, taken from the repository history (11 July to 2 October 2026) and the review presentations; all three members contributed to testing, documentation and the reviews.

Table 1.3: Group members and their contributions

Name	PRN	Primary contributions
Faheemuddin Sayyed	23070122196	Repository tooling, data layer, authentication, PWA shell and realtime hub (01–05); sandbox manager, workspace files, terminal, Monaco IDE, live preview and DesignProvider interface (11–16); test generation and test runner (27–28); DB provisioning, deploy orchestration, topology, live validation, data browser, evaluation harness and observability (36–46); phased build, provider compatibility, Vercel backend, preview lifecycle, scope-aware build and layout (56–61); deploy truthfulness, repair reachability and truthful phase completion (62–64); platform CI/CD, Ansible, Kubernetes and monitoring.
Sanidhya Awasthi	23070122192	Stage state machine, conversation and artifact model, conductor, workspace UI and sandbox image (06–10); Stitch and Figma design providers, design stage, LLM client and agent tools (17–21); requirements schema and UI (25–26); credential vault and deploy adapters (34–35); security hardening, resilience, PWA, demo and layered configuration (47–51); design clarifying questions; offline test database (65); CI fixes and the DevOps reports.
Raghav Sonchhatra	23070122172	Fixed-stack app skeleton, codegen agent loop and build stage (phases 22–24); the self-healing repair core: failure analyzer, diff-aware repair agent, bounded loop controller and repair UI (29–32); infrastructure analyzer (33); admin dashboard UI (52); skeleton clean-up and build verification with self-heal (54–55); patent-disclosure revision and the paper's figures.

1.8 Organization of the Report

The rest of this report is organised as follows. Chapter 2 reviews the literature on LLM-based code generation, program repair, test generation, human-in-the-loop control, design-to-code and deployment, and identifies the research gap. Chapter 3 specifies the software requirements: tools and technologies, hardware, work breakdown, functional and non-functional requirements, cost and schedule. Chapter 4 describes the methodology, covering the architecture, design diagrams and the implementation of each subsystem, with the bounded repair loop treated in most detail. Chapter 5 presents the testing strategy, test cases and results, the defects found during end-to-end validation, the platform's delivery pipeline and the project outcomes. Chapter 6 concludes and outlines future work. References and appendices follow.

Chapter 2
Literature Review

This chapter places ForgeFlow in the context of existing research and practice. Section 2.1 reviews the relevant literature thread by thread, from the use of LLMs across the software lifecycle to the delivery of agent-written code. Section 2.2 then brings these findings together in a summary of the reviewed sources, a comparison of existing classes of systems with ForgeFlow, and a precise statement of the research gap that the project addresses.

2.1 Background

The literature relevant to ForgeFlow spans several research threads that have, until recently, been studied separately: LLMs across the software lifecycle, agentic code generation, automated program repair, test generation and traceability, human control of AI agents, multimodal design-to-code, and the delivery of agent-written code through CI/CD and deployment. This chapter reviews each thread, concentrating on 2024–26 work, and then identifies the gap that ForgeFlow addresses. Where a cited source is a preprint or an industry survey rather than a peer-reviewed publication, the text says so.

2.1.1 LLMs across the software lifecycle and the adoption–trust gap

Q. Zhang et al. (2026) survey LLMs for software engineering and systematise 926 studies covering 112 tasks across the requirements, design, development, testing and maintenance phases, showing that LLMs are now used throughout the lifecycle rather than only for code completion. Ge et al. (2025), in a preprint survey of vibe coding, describe the user-centric category in which a natural-language request becomes a deployable application, and they distinguish planning-driven and test-driven forms of the practice. These two surveys

establish that generation is mature; the question this project takes up is what happens after generation.

Industry surveys show that adoption is high and rising while trust is low. The Stack Overflow 2025 Developer Survey reports that 84% of developers use or plan to use AI tools, up from 76% a year earlier, yet 46% distrust the accuracy of AI output against 33% who trust it (Stack Overflow, 2025). The DORA 2025 report finds that AI adoption now improves delivery throughput but is still associated with lower delivery stability, and that AI amplifies existing practice: teams with fast feedback loops and strong safety nets gain, while weak pipelines ship poor work faster (DORA, 2025). Becker et al. (2025), in a randomised controlled trial with experienced open-source developers, found that tasks took 19% longer when AI tools were allowed, even though the developers believed they had been faster. The CloudBees (2026) industry survey reports that 81% of enterprise technology leaders see more production incidents caused by AI-generated code.

2.1.2 Agentic code generation and issue resolution

Benchmarks such as SWE-bench, which evaluates whether models can resolve real GitHub issues from popular Python repositories, pushed research from function-level generation to repository-level tasks (Jimenez et al., 2024). SWE-agent showed that the interface between an agent and the computer, meaning the commands, file views and test runners the agent can use, strongly affects how well it performs (J. Yang et al., 2024). Multi-agent frameworks followed: MAGIS divides GitHub issue resolution among collaborating agents (Tao et al., 2024), and AgentCoder separates programmer, test-designer and test-executor agents so that generated code is checked against generated tests (D. Huang et al., 2024). Jiang et al. (2025) survey more than 150 primary studies on agentic issue resolution and classify them into benchmarks, techniques and empirical studies.

An influential counter-result is Agentless (Xia et al., 2025). It showed that a simple, fixed three-phase pipeline of localisation, repair and patch validation can match or outperform complex autonomous agents on issue resolution at a much lower cost. ForgeFlow follows this lesson: its pipeline is fixed and deterministic, the project skeleton is copied rather than generated, and the model is called only where real synthesis is needed.

2.1.3 Automated program repair and the economics of iteration

LLM-based program repair is an active field. RepairAgent is an autonomous agent that plans and calls repair tools and fixes real-world defects at competitive rates (Bouzenia et al., 2025). K. Huang et al. (2025), in IEEE Transactions on Software Engineering, show through extensive fine-tuning experiments that repair quality depends heavily on how the repair task is framed and on the context supplied to the model. Olausson et al. (2024) asked whether self-repair is a silver bullet and found that, once the cost of repair is accounted for, the gains are often modest and vary widely, because self-repair is limited by the quality of the model's own feedback. Self-debugging improves results, but only to the extent that execution feedback is informative (X. Chen et al., 2024).

Work from 2026 adds the economic question of how many repair attempts are worth making. Kiecker et al. (2026), in a preprint evaluating LLM-based repair loops, report that the first three to four iterations account for most of the achievable gain while later iterations add little. Arimbur (2026), studying iterative self-repair across seven models and several benchmarks, finds most gains in the first two rounds. Shukla et al. (2025) provide the safety argument: when a model is repeatedly asked to make generated code more secure, security can degrade with further iterations. Taken together, these results argue that a repair loop should be bounded by design, not merely measured after the fact.

2.1.4 Repair context and fault localisation

Several studies show that what the repair model is shown matters more than how much it is shown. Sepidband et al. (2026), in a preprint covering 500 SWE-bench Verified tasks and 61 context configurations, find a 15 to 17 times gain from file-level localisation over a no-file baseline, with the best results at roughly six to ten relevant files, and they observe that expanding context to more lines often adds noise. RepoGraph represents a repository as a line-level code graph and retrieves k-hop ego-graphs as context, which improves software-engineering agents (Ouyang et al., 2025). LocAgent uses dependency graphs and multi-hop search to reach 92.7% file-level localisation accuracy (Z. Chen et al., 2025). These results support ForgeFlow's import-graph walk, which follows the failing test's imports to the file that holds the bug.

2.1.5 Test generation and requirements traceability

LLM-based unit-test generation has been studied at scale. Schäfer et al. (2024) gave an early empirical evaluation of LLM-based unit-test generation for JavaScript. Ouédraogo et al. (2026) compare prompt-engineering strategies for class-level unit tests over a very large number of generated tests, and Shang et al. (2025) fine-tune 37 models across three unit-testing scenarios and five benchmarks. Closer to ForgeFlow's goal, Alian et al. (2025) generate end-to-end tests from application features, and Gu et al. (2025) use hybrid program analysis to guide iterative test generation and improve the validity of generated suites.

A key negative result comes from Haroon et al. (2026), a preprint that evaluates eight LLMs on 22,374 program variants. When the program under test changes, generated tests lose validity, and more than 99% of the new failing tests still pass on the original program: the model tests the code it remembers rather than the code it was given. The authors

recommend computing a code diff and highlighting the changed regions to direct the model's attention. This is exactly what ForgeFlow's repair context does. On the requirements side, TraceLLM shows that prompt-engineered LLMs can recover links between requirements and artifacts after the fact (Alturayeif et al., 2026); ForgeFlow instead builds the link in from the start through a criterion identifier that travels from each acceptance criterion to its tests, results and repair context.

2.1.6 Human-in-the-loop control and ambiguity

Industrial practice favours keeping engineers in control. HULA, deployed in Atlassian Jira, lets engineers approve both the coding plan and the resulting code (Takerngsaksiri et al., 2025). Its follow-up names the high cost of correctness testing and the variability of LLM-based evaluation as the main open challenges (Pasuksmit et al., 2025). Observational work finds that professional developers do not simply delegate to agents but continually restrain, monitor and redirect them (R. Huang et al., 2025), and a large-scale preprint analysis of 20,574 real coding-agent sessions finds widespread misalignment between what developers intend and what agents do, with 91.49% of visible misalignment resolutions requiring explicit user correction (Tang et al., 2026).

Ambiguity in requirements is a related problem. ClarifyCodeBench shows that strong code generators are not necessarily good at asking clarifying questions, and clarification quality degrades as ambiguity increases (Fang et al., 2026). The agentic software-engineering roadmap of Hassan et al. (2025) calls for agents that initiate callbacks to humans when they meet ambiguity. ForgeFlow applies this by routing a design provider's question to the user instead of silently falling back to another provider.

2.1.7 Multimodal design-to-code

Design2Code benchmarks multimodal models on converting 484 real web pages into code and shows that models still lag in recalling visual elements and reproducing layout (Si et al., 2025). For this reason ForgeFlow treats design as a pluggable, human-refinable stage with versioned artifacts and an explicit skip path, rather than a one-shot conversion whose output is passed downstream unchecked.

2.1.8 Deployment, CI/CD and the delivery of agent-written code

Deployment is a separate failure mode from code quality. T. Zhang et al. (2026), in work accepted at FSE 2026, evaluate LLM-generated infrastructure-as-code across 153 real-world scenarios and 58 cloud services and report first-attempt deployment success of only 20.8% to 30.2%. Ghammam and Almukhtar (2026) mine AI-generated build code and find that agents systematically introduce maintainability and security smells.

Mining-software-repositories research from 2026 shows that continuous integration is now where an agent's work is judged. Li et al. (2025) built the AIDev dataset of more than 456,000 agent-authored pull requests from five agents across 61,000 repositories and found that agents are faster than humans but have their pull requests accepted less often. Ehsani et al. (2026) analyse 33,000 agentic pull requests and find that rejected ones are larger, touch more files and often fail the project's CI/CD checks. Ghaleb (2026) reports that changes to CI/CD configuration make up only 3.25% of agent changes and are merged less often than other changes. Fixing what CI rejects remains hard: on CI-Repair-Bench, which validates every candidate fix by re-running the full CI workflow on 567 real failures from 103 repositories, the best model fixes only 18.9%, and environment, dependency and configuration failures are the hardest classes (Muna et al., 2026).

2.1.9 Sandboxing, cost and model routing

Running untrusted, model-generated code needs strong isolation. SandboxEval documents how wide the attack surface of naively configured execution environments is (Rabin et al., 2025), which is why ForgeFlow treats the sandbox boundary, rather than the model's behaviour, as the security control. On cost, RouteLLM learns to route each request between a stronger and a weaker model using preference data and reduces cost by more than half without losing quality (Ong et al., 2025). ForgeFlow applies a simpler, role-based version of the same idea, sending classification and routing work to a cheap model and only real code work to the code-generation model, which directly addresses the cost-of-correctness-testing challenge named by Pasuksmit et al. (2025).

2.2 Summary of Literature Review and Research Gap

This section draws the reviewed work together. It first summarises the most relevant sources and the way each one shaped ForgeFlow, and then compares the existing classes of systems to state the research gap precisely.

2.2.1 Summary of the reviewed literature

Table 2.1 summarises the most relevant sources, their main findings and how each one shaped ForgeFlow.

Table 2.1: Summary of the literature review

Source	Key finding	Relevance to ForgeFlow
Q. Zhang et al. (2026)	Survey of 926 studies on 112 SE tasks across five lifecycle phases.	Generation across the lifecycle is mature; verification is the open problem.
Ge et al. (2025)	Survey formalising vibe coding, including test-driven variants.	Defines the prompt-to-app category ForgeFlow belongs to.
Stack Overflow (2025)	84% adoption, 46% distrust vs 33% trust of AI output.	Motivates verification and visibility of generated code.
DORA (2025)	AI raises throughput but correlates with lower delivery stability.	Motivates gated delivery and live validation.
Becker et al. (2025)	RCT: experienced developers 19% slower with AI while feeling faster.	Perceived and real productivity differ; outcomes must be measured.

Source	Key finding	Relevance to ForgeFlow
CloudBees (2026)	81% of leaders report more AI-caused production incidents.	Failures reach production; validate after deployment.
Jimenez et al. (2024)	SWE-bench: real GitHub issues as a benchmark.	Repository-level, test-validated evaluation.
J. Yang et al. (2024)	Agent-computer interface strongly affects agent success.	Strict, sandbox-bound tool surface for agents.
Xia et al. (2025)	Simple fixed pipeline matches complex agents at lower cost.	Deterministic pipeline; skeleton copied, not generated.
Bouzenia et al. (2025)	Autonomous repair agent with tool planning.	Contrast: ForgeFlow bounds the agent inside a controller.
K. Huang et al. (2025)	Repair quality depends on task framing and context.	Minimal, structured repair context.
Olausson et al. (2024)	Self-repair gains are modest once cost is counted.	Repair must be costed and bounded.
Kiecker et al. (2026)	Most repair gain in the first three to four iterations.	Default iteration cap of 5 and stall guard of 2.
Arimbur (2026)	Most gains in the first two rounds across seven models.	Supports early stopping.
Shukla et al. (2025)	Iterative “make it secure” prompting can degrade security.	Unbounded loops are unsafe, not just wasteful.
Sepidband et al. (2026)	File-level localisation gives 15–17x gain; 6–10 files is best.	Per-file and whole-context character caps.
Ouyang et al. (2025)	Repository code graph improves SE agents.	Graph-shaped repair context.
Z. Chen et al. (2025)	Graph-guided localisation reaches 92.7% file accuracy.	Import-graph walk from the failing test.
Haroon et al. (2026)	>99% of failing generated tests pass on the old program; highlight diffs.	Diff since last passing run in repair context.
Alian et al. (2025)	Feature-driven end-to-end test generation.	Acceptance criteria compiled to Playwright tests.
Alturayeif et al. (2026)	LLMs recover requirement-artifact links.	ForgeFlow builds traceability in via criterion_id.
Takerngsaksiri et al. (2025)	HULA keeps engineers in control in Jira.	Every stage is refinable, skippable and revisitable.
Pasuksmit et al. (2025)	Open challenge: cost of correctness testing.	Role-based model routing and budget caps.
Tang et al. (2026)	Agent-developer misalignment in 20,574 sessions.	Human approval and reversible skips.
Fang et al. (2026)	Code generators are poor clarifiers.	Provider questions routed to the user.
Si et al. (2025)	Design-to-code still lags on layout fidelity.	Design is a pluggable, human-refinable stage.
T. Zhang et al. (2026)	20.8–30.2% first-attempt IaC deployment success.	Infrastructure analyzer and truthful deployment.
Muna et al. (2026)	Best LLM fixes 18.9% of real CI failures.	Environment failures never reach the repair agent.
Rabin et al. (2025)	Naive execution sandboxes are easy to escape.	Hardened per-project Docker sandboxes.

Source	Key finding	Relevance to ForgeFlow
Ong et al. (2025)	Learned routing halves cost without quality loss.	Cheap model for routing, capable model for code.

Read together, the rows of Table 2.1 point in one direction. The adoption and incident studies show that AI-generated code is widely used but not trusted; the repair studies show that most of the benefit of iterating arrives in the first few attempts and that further attempts can do harm; and the localisation and test-generation studies show that a small, well-chosen context anchored on what has changed works better than a large one. The right-hand column records how each finding became a concrete design decision in ForgeFlow rather than a general aspiration.

2.2.2 Research gap

The reviewed work falls into three groups. Research agents and benchmarks (for example SWE-agent, Agentless and RepairAgent) generate and repair code but stop before deployment. LLM test-generation and repair tools improve individual steps but are not part of a delivery system. Commercial prompt-to-app builders generate and deploy applications but do not verify them against the user's requirements, do not validate the deployed site, and offer no principled stopping rule when fixes fail. Table 2.2 compares these groups with ForgeFlow.

Table 2.2: Capability comparison across classes of systems

Capability	Research agents	LLM test / repair tools	Commercial builders	ForgeFlow
Multimodal design intake (screenshot to UI)	No	No	Yes	Yes (pluggable)
Full-stack application generation	Partial	No	Yes	Yes (fixed stack)
Requirements to executable, traceable tests	No	Partial	No	Yes
Bounded repair: iteration cap and stall detection	No	No	No	Yes
Diff-aware minimal repair context	No	Partial	No	Yes

Capability	Research agents	LLM test / repair tools	Commercial builders	ForgeFlow
Human escalation on non-convergence	No	No	No	Yes
Automated cloud deployment	No	No	Yes (locked-in)	Yes (dual-mode)
Post-deployment live validation	No	No	No	Yes
Non-linear, skip-any-stage control	No	No	Partial	Yes
Per-run cost accounting and budget caps	No	No	No	Yes

The gap can therefore be stated precisely: **no existing system combines requirement-derived verification, a repair loop that is bounded by construction and escalates to a human, deployment, and validation of the live artifact, within a workflow where the human can enter, skip or revisit any stage.** Research systems verify but do not deploy; commercial builders deploy but do not verify; neither validates the live site, and none bounds the repair loop.

Table 2.3 lists the specific gaps identified in the literature and the ForgeFlow mechanism designed in response to each.

Table 2.3: Identified gaps and the corresponding ForgeFlow design response

Gap identified	Evidence	ForgeFlow response
AI-generated code has systematic quality and security flaws	Ghammam & Almukhtar (2026); Zhao et al. (2026)	Verification over generation: requirements compile into executable tests
Repair gains stop after a few iterations; iterating can degrade security	Kiecker et al. (2026); Arimbur (2026); Shukla et al. (2025)	Iteration cap 5, stall guard 2, regression guard 2, no-op guard 2, budget guard
The right file matters more than more context	Sepidband et al. (2026); Ouyang et al. (2025); Z. Chen et al. (2025)	Import-graph repair context with character caps and a bounded file request
Generated tests lose validity when code changes	Haroon et al. (2026)	Diff since the last passing run included in every repair context
Deployment fails separately from code quality	T. Zhang et al. (2026)	Infrastructure analyzer, mirrored environment variables, real provider logs, database pre-flight check
Environment and configuration failures are the hardest CI class	Muna et al. (2026)	Environment failures classified first and escalated with zero repair iterations

Gap identified	Evidence	ForgeFlow response
Users must correct and override agents	Tang et al. (2026); R. Huang et al. (2025)	Reversible skips, human approval of a build, conductor-owned transitions
Models guess when a brief is ambiguous	Fang et al. (2026); Hassan et al. (2025)	Design-provider questions parked and routed to the user
Cost of correctness testing is an open challenge	Pasuksmit et al. (2025); Ong et al. (2025)	Role-based model routing, per-run cost accounting, budget caps

Two features of Table 2.3 are worth noting. First, every gap is answered by a mechanism that can be checked, such as a numeric guard, a character cap or a classification step, rather than by a hope that the model will behave better. Second, several responses depend on one another: the diff in the repair context exists only because the workspace is git-backed, and environment failures can be stopped at zero iterations only because every failure is classified before repair begins. Chapter 4 describes how each mechanism is implemented, and Chapter 5 reports how it was tested.

Chapter 3
Software Requirements Specification

This chapter specifies what ForgeFlow must do and the conditions under which it must do it. It lists the software and hardware used, the work breakdown, the functional and non-functional requirements, the cost estimate and the project schedule.

3.1 Software Tools, Platform and Frameworks Used

ForgeFlow involves two distinct software stacks: the stack of the platform itself, and the fixed stack of every application it generates. Keeping them separate is a design decision. The platform is written in Python because Python suits the agent loop, Docker orchestration and asynchronous I/O, while generated applications are always Node.js and React. Table 3.1 lists the platform's tools and their specific use.

Table 3.1: Platform software stack and the specific use of each tool

Tool or framework	Version	Specific use in ForgeFlow
Python	3.11	Language of the control plane (agents, orchestration, sandbox management).
FastAPI and Uvicorn	0.115+	REST API and WebSocket server; asynchronous request handling for long agent, Docker and provider calls.
Pydantic v2 and pydantic-settings	2.7+	Typed schemas at every boundary; rejects malformed LLM output; layered configuration.
Motor and Beanie	3.5+ / 1.26+	Asynchronous MongoDB driver and typed ODM for the control-plane database forgeflow_meta.
docker (Python SDK)	7.1+	Creates, limits, executes in and reaps per-project sandbox containers.
anthropic and openai SDKs	0.39+ / 1.40+	Two LLM transports behind one seam: Anthropic Messages and any OpenAI-compatible Chat Completions endpoint.
httpx	0.27+	HTTP calls to design and deployment providers (Stitch, Figma, Vercel, Render).
cryptography (Fernet)	43+	Encrypts provider tokens and secret settings at rest.
python-jose and passlib (bcrypt)	3.3+ / 1.7+	JWT session tokens and bcrypt password hashing.
prometheus-client	0.20+	Exposes /metrics with request counts, latency histograms and build information.
pytest and pytest-asyncio	8.2+	Backend unit and integration tests (1,592 test functions).
ruff, black, mypy	current	Linting, formatting and static type checking of the control plane.

Tool or framework	Version	Specific use in ForgeFlow
React and React Router	18.3 / 6.28	Platform PWA: dashboard, workspace, stage panels, admin dashboard.
Vite and TypeScript (strict)	5.4 / 5.5	Frontend build tooling and type safety.
Tailwind CSS	3.4	Semantic design tokens with light and dark themes.
Zustand and TanStack Query	4.5 / 5.59	Client UI state and server-state caching with automatic refetch on stage transitions.
Monaco Editor and xterm.js	0.52 / 5.5	In-browser IDE editor and terminal bridged to the sandbox shell.
@xyflow/react (React Flow)	12	Interactive deployment topology with per-node status, logs and redeploy controls.
vite-plugin-pwa (Workbox)	0.20	Installable PWA with an offline application shell.
framer-motion, react-resizable-panels, lucide-react	current	Micro-interactions, resizable and collapsible workspace panels, icons.
Vitest and Testing Library	2.1 / 16	Frontend unit tests (449 test cases).
ESLint and Prettier	8.57 / 3.3	Frontend linting and formatting.

Two entries in Table 3.1 shape the rest of the system. The docker SDK lets the control plane create, limit and remove sandbox containers, so generated code always runs inside a container and never in the control plane itself. The anthropic and openai SDKs sit behind a single transport seam, so the language-model provider can be changed through configuration without touching agent code (Section 4.12).

Every application that ForgeFlow generates uses the fixed stack in Table 3.2. It is shipped as an opinionated skeleton in templates/app-skeleton that is copied, never generated, into each project's workspace.

Table 3.2: Fixed stack of every generated application

Tool	Version	Specific use in generated applications
React, React Router, Vite, TypeScript	18.3 / 6.26 / 5.4 / 5.6	Single-page frontend with a typed API client.
Tailwind CSS	3.4	Styling of generated pages from the design artifact.
Node.js, Express, TypeScript	20 / 4.21 / 5.6	REST backend with health route, error middleware and configuration.
MongoDB and Mongoose	8.7	Persistence through generated Mongoose schemas; connection cached for serverless functions.
Zod	3.23	Request validation on the backend.
Vitest and Testing Library	2.1 / 16	Frontend unit tests generated from acceptance criteria.
Jest, supertest, ts-jest	29 / 7 / 29	Backend unit tests that exercise the Express app in-process.

Tool	Version	Specific use in generated applications
mongodb-memory-server-core	11.2	Real in-memory MongoDB for tests inside the offline sandbox.
Playwright	1.48.2	End-to-end tests run against the sandbox preview and again against the live URL.

Table 3.3 lists the external services and the tools used to deliver, run and monitor the platform itself.

Table 3.3: External services and DevOps tooling

Service or tool	Specific use
MongoDB Atlas	Shared cluster that hosts one isolated database per deployed generated application.
Vercel	Hosts the platform PWA and the generated applications (static frontend plus an Express function backend).
Render (optional)	Alternative backend host for a persistent process, behind DEPLOY_BE_PROVIDER=render.
Google Stitch and Figma (MCP)	Design providers: image-to-UI and text-to-UI, with Figma as the alternative.
Docker Engine, Colima, Docker Compose	Local development runtime and the production stack (api, mongo, appdb, caddy).
Caddy	Reverse proxy that routes each project's preview subdomain to its sandbox container.
nginx and Let's Encrypt	TLS termination and WebSocket upgrade on the production virtual machine.
GitHub Actions and GHCR	Six-job CI on every push; CD that builds a multi-architecture image and deploys it with rollback.
Ansible	Idempotent provisioning of the Oracle Cloud VM with seven roles.
k3s (Kubernetes)	Orchestration demonstration: Deployment, StatefulSet, rolling update and rollback.
Prometheus, Grafana, exporters	Metrics scraping, a 16-panel dashboard and ten alert rules.
gitleaks, pip-audit, pnpm audit	Secret scanning over full history and dependency vulnerability audits in CI.

The hosted services that ForgeFlow relies on by default, namely MongoDB Atlas, Vercel, Google Stitch and GitHub, were all used within their free tiers (Table 3.7), which is why the cost estimate consists almost entirely of language-model usage. The CI/CD, configuration-management, orchestration, monitoring and audit tools in the lower part of Table 3.3 serve the platform's own delivery pipeline, described in Section 4.14, rather than the generated applications.

3.2 Hardware Requirements

Table 3.4 lists the hardware used for development and production. Because generated code runs in containers with fixed limits, the server's capacity determines how many projects can build or test at the same time.

Table 3.4: Hardware requirements

Component	Specification
Development workstation	Windows 11 or macOS laptop, 4 or more CPU cores, 16 GB RAM recommended, 30 GB free disk; Docker Desktop or Colima, Node.js 20, Python 3.11.
Production server	Oracle Cloud Infrastructure VM.Standard.A1.Flex (Ampere Altra, arm64): 3 OCPU, 16 GB RAM, about 50 GB boot volume, Ubuntu 24.04 LTS, 2 GB swap.
Control-plane container	Limited to 2 CPU and 4 GB memory; one Uvicorn worker by design.
Per-project sandbox	1 CPU, 1 GB memory, at most 256 processes; reaped after 15 minutes idle.
Client	Any modern browser (Chrome, Edge or Firefox); ForgeFlow can be installed as a PWA.

3.3 Work Breakdown Structure

The work was decomposed into 65 phases grouped into 14 epics (numbered 0 to 13) and five milestones. Each phase has its own specification in the repository's plans folder with a goal, dependencies, file-level tasks, an out-of-scope list and testable acceptance criteria, and a phase was marked done only when all of its acceptance criteria passed. Epics 12 and 13 and the later phases were added in response to issues found during live use. Figure 3.1 shows the breakdown.

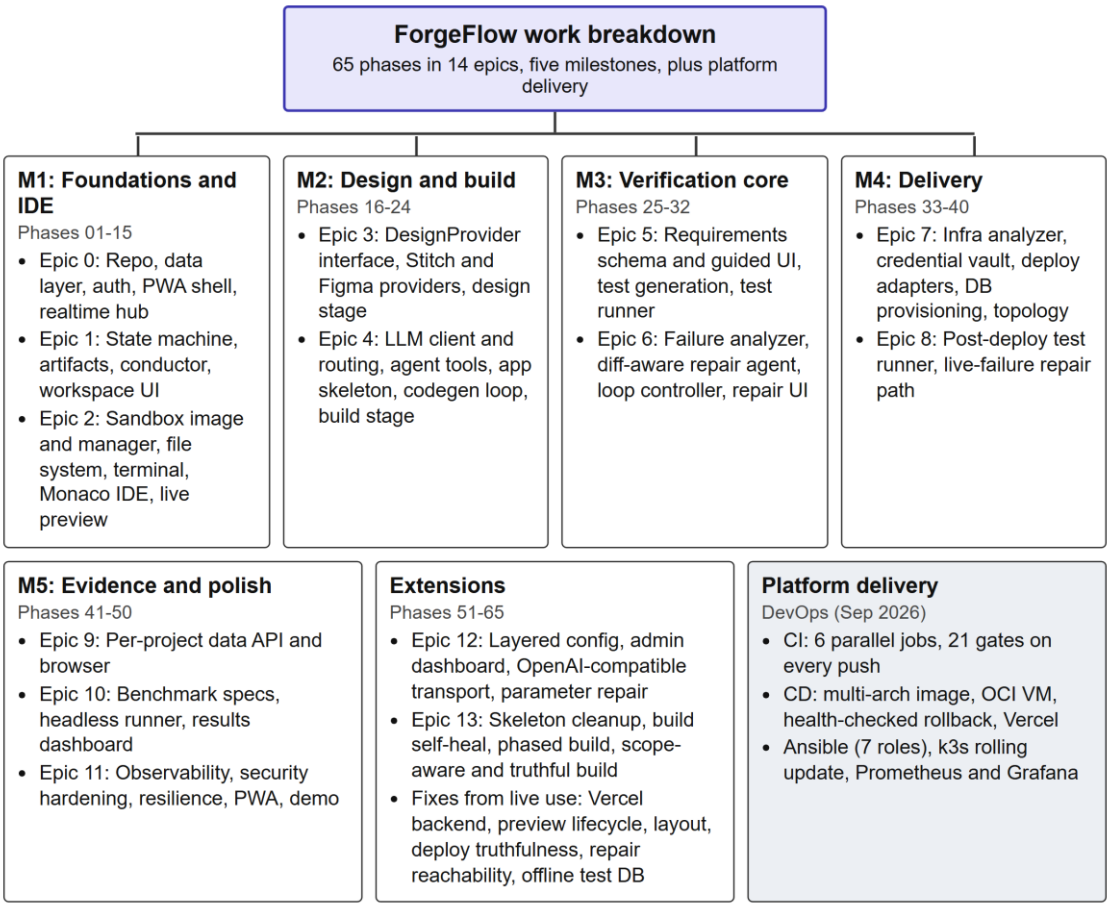


Figure 3.1: Work breakdown structure of ForgeFlow

The milestones were defined so that each one leaves a runnable system. M1 delivered an authenticated workspace with a working web IDE, terminal and preview over sandboxes; M2 turned screenshots or text into a design and a generated, running application; M3 delivered requirements-to-tests and the bounded repair loop; M4 delivered deployment, the topology view and live validation; and M5 added the data browser, the evaluation harness, hardening, PWA polish and the demonstration scenario.

3.4 Functional Requirements

Figure 3.2 shows the actors and the main use cases. The user drives the pipeline; the administrator configures the platform and monitors cost; external providers supply language models, designs and hosting.

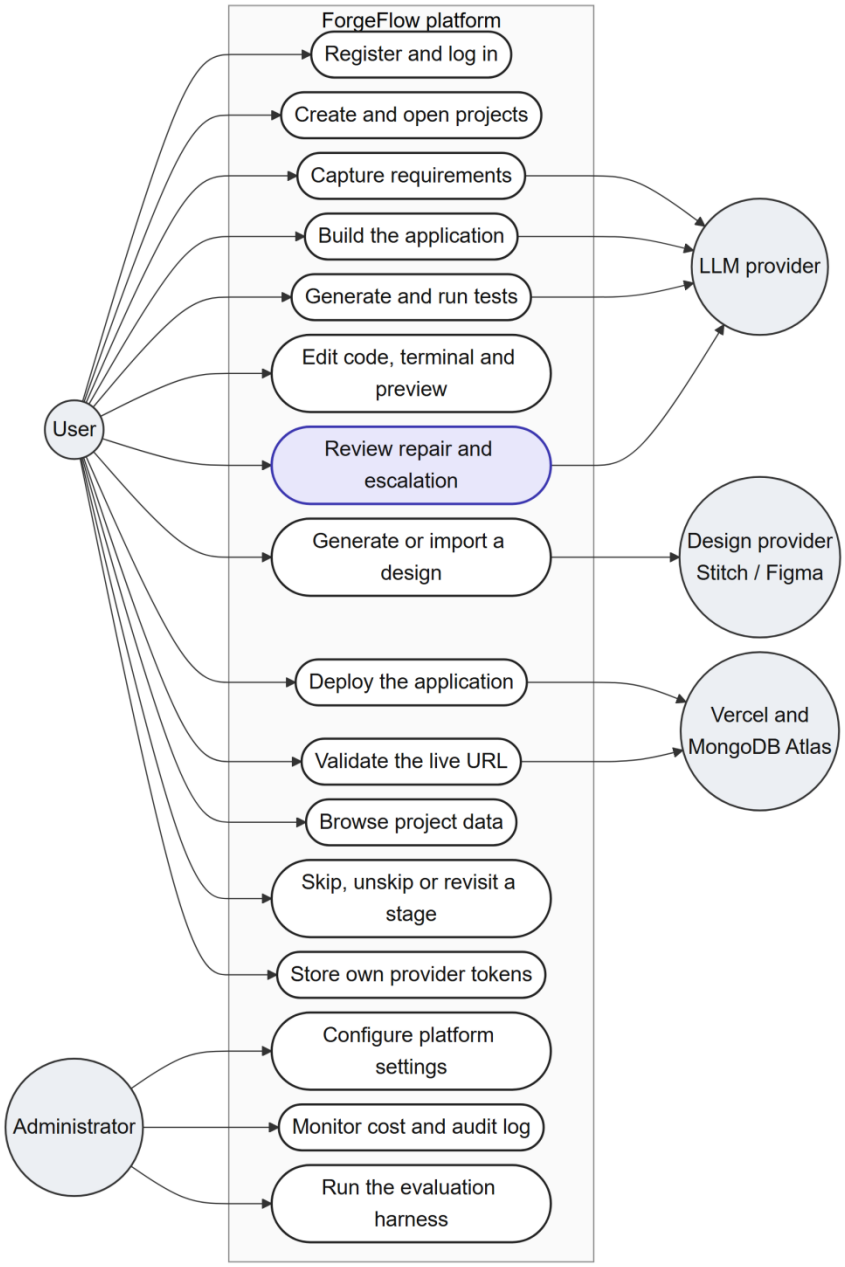


Figure 3.2: Use case diagram of ForgeFlow

Table 3.5 lists the functional requirements and the subsystem that realises each one.

Table 3.5: Functional requirements

ID	Requirement	Realised by
FR-1	Register, log in and manage projects; each project has its own workspace, sandbox and database.	Auth, project service
FR-2	Accept an app idea as screenshots or a structured specification.	Requirements and design stages
FR-3	Capture requirements as features with inputs, expected behaviours and acceptance criteria, through a guided form or an AI draft that is never auto-saved.	RequirementSpec, requirements UI

ID	Requirement	Realised by
FR-4	Generate a UI design from text or screenshots, refine it conversationally, import an own design or skip the stage.	DesignProvider (Stitch, Figma)
FR-5	Generate a working full-stack application on the fixed stack and stream the code live into the IDE.	Codegen agent, build stage
FR-6	Accept conversational change requests and route small changes to an incremental pass.	Scope classifier, build stage
FR-7	Compile acceptance criteria into unit and end-to-end tests, each tagged with its criterion_id.	Test-generation agent
FR-8	Run tests in the sandbox and report structured per-test results.	Test runner and parsers
FR-9	Repair failures with a bounded, diff-aware loop and escalate to the user when it cannot converge.	Repair loop controller
FR-10	Analyse the application's infrastructure needs and deploy the frontend, backend and database in order.	Infra analyzer, deploy orchestrator
FR-11	Support platform-owned (seamless) and bring-your-own credentials, stored encrypted.	Secret vault
FR-12	Show a live deployment topology with status, provider logs, redeploy and teardown.	Topology UI
FR-13	Re-run the end-to-end suite against the live URL and send live failures to repair and redeploy.	Live validation controller
FR-14	Let the user enter, refine, skip, unskip or revisit any stage, and approve a build.	Conductor, state machine
FR-15	Provide an in-browser editor, terminal, live preview and a per-project data browser.	Web IDE, data browser
FR-16	Let an administrator configure every platform setting, monitor cost and review an audit log.	Admin dashboard
FR-17	Run benchmark specifications headlessly and report pass rates, iterations, cost and time.	Evaluation harness

The seventeen requirements in Table 3.5 follow the product order of the pipeline: project management and intake (FR-1 to FR-3); design, build and change requests (FR-4 to FR-6); test generation, execution and repair (FR-7 to FR-9); and deployment and live validation (FR-10 to FR-13). The remaining four cover stage control, the in-browser IDE, administration and evaluation, which cut across all stages (FR-14 to FR-17). Every requirement is realised by a named subsystem, and Chapter 4 describes how each subsystem works.

3.5 Non-Functional Requirements

Non-functional requirements define the qualities that the platform must keep while it delivers the functions above. Table 3.6 lists eleven of them. Several are not ordinary quality targets but invariants of the design, which every subsystem must respect.

Table 3.6: Non-functional requirements

ID	Quality attribute	Requirement
NFR-1	Non-linearity	No stage is mandatory; the only hard prerequisites are deploy after build and validate after deploy. Refining a stage marks later stages stale but never deletes them.
NFR-2	Isolation and security	Untrusted generated code runs only in sandboxes: non-root, all capabilities dropped, no new privileges, read-only root, no host network, CPU, memory and process limits.
NFR-3	Data isolation and secrets	One database per project; provider tokens encrypted at rest, decrypted only at call time, never logged or returned to the client.
NFR-4	Boundedness	Every loop has a hard bound: repair iterations, stalls, regressions, empty attempts, tool turns, build phases and wall-clock time.
NFR-5	Cost discipline	Models are chosen by role; token usage and rupee cost are recorded per run; per-project and global budget caps halt work with a warning at 80%.
NFR-6	Reliability	Provider failures degrade gracefully; long tasks are event-driven and resumable from a sequence number after a disconnect.
NFR-7	Auditability	Artifacts are versioned, never overwritten; every repair attempt and configuration change is recorded.
NFR-8	Usability	Installable PWA with light and dark themes, resizable panels, a focus mode and generated code that stays visible and editable.
NFR-9	Maintainability	Full type hints and strict TypeScript; ruff, black, mypy, ESLint and Prettier clean; one specification per phase.
NFR-10	Portability	No code above the LLM transport assumes a vendor's request shape, model names or error format.
NFR-11	Observability	Structured JSON logs, per-run traces, a cost dashboard and Prometheus metrics with bounded label cardinality.

Three of these requirements distinguish ForgeFlow from a conventional web application. Non-linearity (NFR-1) means that every stage must tolerate missing or stale upstream work. Boundedness (NFR-4) means that no loop in the system, whether a repair loop, a build or an agent's tool loop, may run without a hard limit. Isolation and secrecy (NFR-2 and NFR-3) mean that untrusted code never runs in the control plane and that provider

tokens are never stored in plain text, logged or returned to the client. Each of these is checked by dedicated automated tests (Section 5.2).

3.6 Project Cost Estimation

The project was planned against a development and demonstration budget of ₹6,000, almost all of it for language-model API usage. Hosting and tooling were kept within free tiers. Table 3.7 gives the estimate. Actual model spending is recorded per project and per run by the platform's own cost tracking, and the budget caps in the admin dashboard prevent it from exceeding the allocation.

Table 3.7: Project cost estimation

Item	Basis	Estimated cost (₹)
LLM API usage (development, demonstration, evaluation)	Planned allocation, enforced by budget caps	6,000
Production server	Oracle Cloud Always Free Ampere A1 allowance	0
Frontend and generated-app hosting	Vercel Hobby tier	0
Generated-app databases	MongoDB Atlas M0 shared cluster	0
Design generation	Google Stitch free quota (about 350 generations per month)	0
Source control, CI/CD and container registry	GitHub, GitHub Actions and GHCR free tiers	0
Domain name and TLS certificates	Dynu dynamic DNS and Let's Encrypt	0
Development hardware	Team members' existing laptops	0
Software tools and libraries	Open-source	0
Total planned cash outlay		6,000

3.7 Project Plan and Schedule

Development ran from 11 July 2026, when the repository and the implementation plan were created, to the final review at the end of October 2026. The first prototype covering all five milestones was built quickly, between 12 and 22 July, because the phase

specifications had been written in advance. The following months were spent hardening the system against problems found in live use, writing the paper and the invention disclosure, automating the platform's own delivery, and preparing the evaluation. Figure 3.3 shows the schedule. Dates for development work come from the repository history; review dates come from the review presentations.

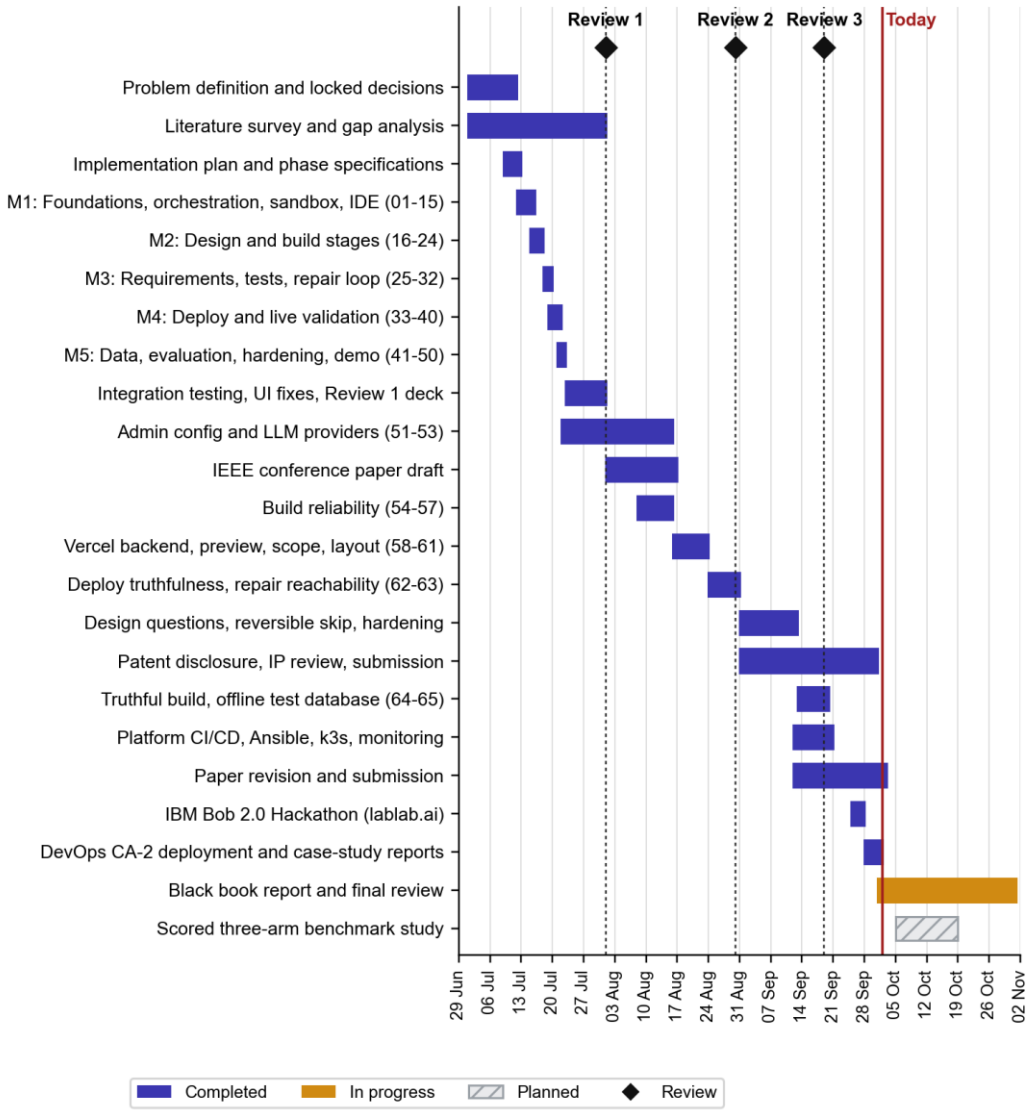


Figure 3.3: Gantt chart of the project, July to October 2026

Table 3.8 compares the milestones planned at the start of the project with their status on 2 October 2026.

Table 3.8: Planned versus achieved milestones

Milestone	Planned	Status (2 October 2026)
M1 to M5: first end-to-end prototype (phases 01–50)	July 2026	Achieved 22 July
Review 1: working prototype and literature survey	1 August	Achieved
Admin configuration and multi-provider LLM support (51–53)	August	Achieved
IEEE conference paper draft	Mid-August	Draft 16 August; submitted, under review
Build reliability and live-use fixes (54–63)	August	Achieved by 30 August
Review 2: research gaps, design mapping, tools	30 August	Achieved
Truthful build and offline test database (64–65)	September	Achieved 19 September
Review 3: end-to-end verification on real deployments	19 September	Achieved
Platform CI/CD, configuration management, orchestration, monitoring	September	Achieved 20 September
Invention disclosure	September	Submitted for filing
IBM Bob 2.0 Hackathon submission	25–27 September	Submitted
Scored three-arm benchmark study	Weeks 8–10	Industry benchmarks done; application corpus partial (Section 5.8)
Final report and review	October	In progress

All of the development milestones and reviews, and the paper, invention-disclosure and hackathon submissions, were completed within the period for which they were planned; the final report and review were in progress at the time of writing. The only item that fell short of its plan is the scored three-arm benchmark study: the industry benchmarks were run in full but the application corpus only in part, so the study is reported as partial in Section 5.8, and completing it is the first item of future work in Section 6.3.

Chapter 4
Methodology

This chapter describes how ForgeFlow was designed and built. It begins with the principles behind the design and the overall architecture, presents the design diagrams (data flow, state, entity-relationship, class and activity diagrams), and then describes the implementation of each subsystem, with the bounded repair loop treated in most detail because it is the project's core contribution. The chapter ends with the user interface and with the pipeline that delivers the platform itself.

4.1 Overview of the Proposed Solution

ForgeFlow carries a web-application idea from intake to a live, validated URL. The user supplies screenshots or a written specification; ForgeFlow helps capture structured requirements, produces a UI design, generates the application, generates tests from the requirements, repairs failures within strict bounds, deploys the application and checks that the deployed site behaves as the requirements say. Six design principles, fixed at the start of the project, govern every subsystem:

1. **P1: Verification over generation.** The defensible contribution is knowing when the system is finished, not how quickly it emits code.
2. **P2: One fixed generated-app stack.** Every generated application is React with Vite and TypeScript, Express with TypeScript, and MongoDB through Mongoose, tested with Vitest, Jest with supertest, and Playwright. A hand-written skeleton is copied rather than generated, which removes scaffolding tokens entirely and follows the lesson of Agentless (Xia et al., 2025).
3. **P3: Cost discipline as an architectural concern.** Models are selected by role: a cheap model for classification, summaries and routing, and the code-generation model only for real code, test and repair work. Every call is costed and checked against budget caps.

4. **P4: Never overwrite; always version.** Designs, code changes, requirements, test suites, test runs, repair attempts, deployments and infrastructure plans are stored as new versions, which makes revisiting a stage safe.
5. **P5: Untrusted code runs only in sandboxes.** The control plane never executes generated code. The sandbox boundary, not the model's good behaviour, is the security control (Rabin et al., 2025).
6. **P6: Non-linearity is an invariant.** No stage is mandatory, and every downstream stage tolerates missing or stale upstream artifacts.

4.2 System Architecture

ForgeFlow is organised into three planes and an external provider boundary (Figure 4.1).

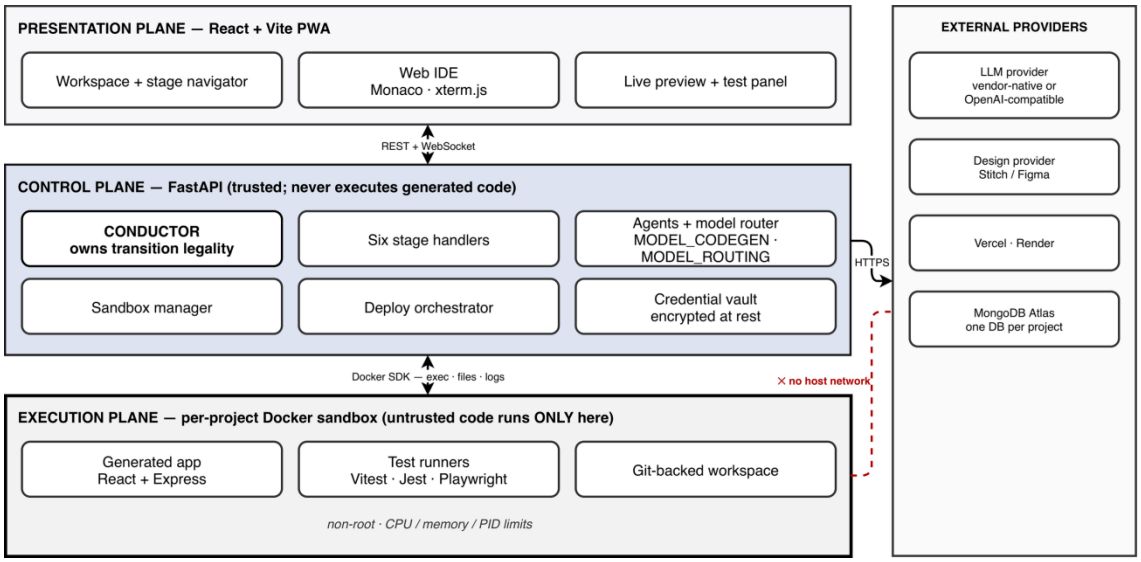


Figure 4.1: System architecture: the presentation, control and execution planes and the external provider boundary. Untrusted generated code executes only inside the per-project sandbox, which has no host network

- **Control plane.** A FastAPI service in Python 3.11 with Pydantic v2 schemas at every boundary and Motor and Beanie over MongoDB. It contains authentication, the project lifecycle and stage state, the conductor, the stage handlers, the agent layer and model router, the sandbox manager, the deployment orchestrator, the encrypted credential vault, the evaluation harness, the layered configuration service and a WebSocket hub. Control-plane metadata lives in the forgeflow_meta database.

- **Execution plane.** One Docker container per active project, holding a git-backed workspace, the generated application, its development servers (proxied as a live preview) and the test runners. Git is not incidental: the diff since the last passing run, which the repair loop depends on, is computed from it.
- **Presentation plane.** An installable React 18 PWA: dashboard, workspace with a clickable stage navigator, Monaco IDE that renders code as the agent writes it, xterm.js terminal, live preview, test and repair panel, React Flow deployment topology, project data browser and admin dashboard.
- **External providers.** LLM providers (Anthropic or any OpenAI-compatible endpoint), design providers (Stitch or Figma), hosting (Vercel, with Render as an option) and MongoDB Atlas.

All long-running work is event-driven. Each realtime event carries the fields event, project_id, stage, payload and seq. The browser subscribes per project over one WebSocket and, after a disconnect, resumes from the last sequence number it saw, so a page reload does not lose an in-flight build or repair loop. Build runs are detached from the request that started them, so they survive a closed tab or a proxy timeout.

4.2.1 Data flow diagrams

Figure 4.2 is the context-level data flow diagram. ForgeFlow exchanges requirements, guidance and approvals with the user, settings and cost information with the administrator, minimal-context prompts and generated code with the LLM provider, briefs and screens with the design provider, and build output, URLs and logs with the hosting provider; it also provisions a per-project database on MongoDB Atlas and receives its connection URI and status.

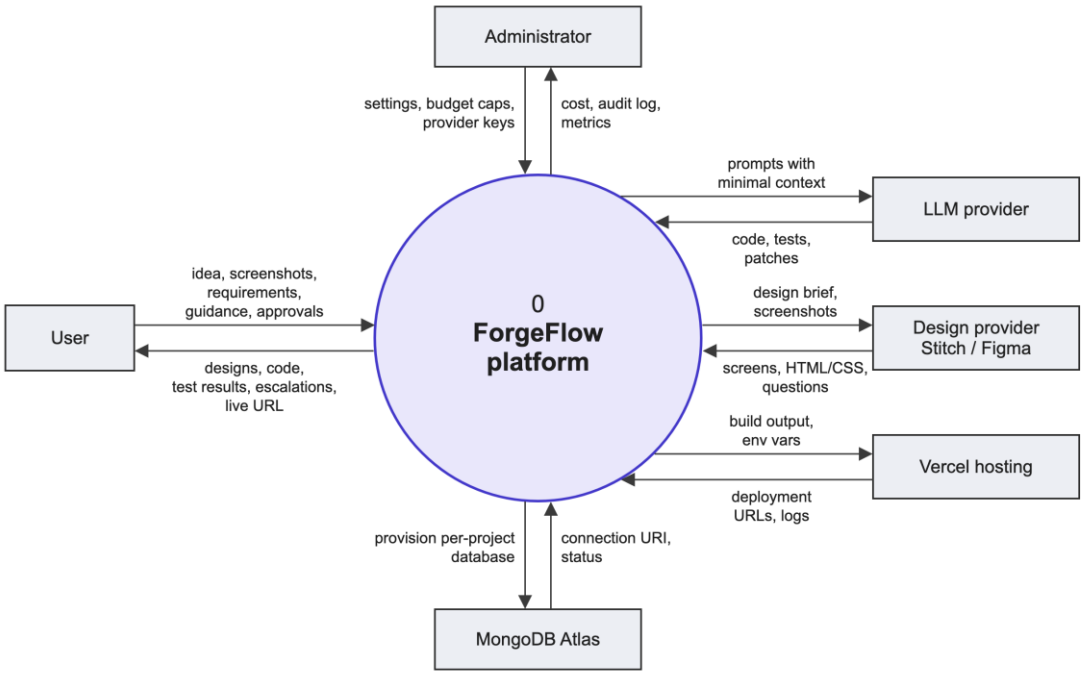


Figure 4.2: Level 0 data flow diagram (context diagram)

Figure 4.3 decomposes the platform into eight processes and four data stores. The control-plane database (D1) holds projects, stage states, versioned artifacts and runs; the sandbox workspace (D2) holds the git-backed source; the credential vault (D3) holds encrypted tokens; and each deployed application gets its own database (D4). The repair process (6.0) reads failing tests from D1 and the diff since the last pass from D2, and writes patches back to D2. Live failures from validation (8.0) flow back into the same repair process.

The eight processes follow the pipeline. Managing projects and stages (1.0) turns the user's intents into stage states, and capturing requirements (2.0) stores the user's features and acceptance criteria in D1 as a RequirementSpec. Generating the design (3.0) reads the specification and writes a design artifact back to D1. Building the application (4.0) reads the specification and the design and writes source files into D2, and generating and running tests (5.0) reads the acceptance criteria, writes test files into D2 and records each TestRun in D1. Deployment (7.0) takes the build output from D2 and the provider tokens from D3 only at call time, provisions the application's database in D4 and records the deployment

in D1, after which validation (8.0) runs the tests against the live URL and returns the validated URL to the user.

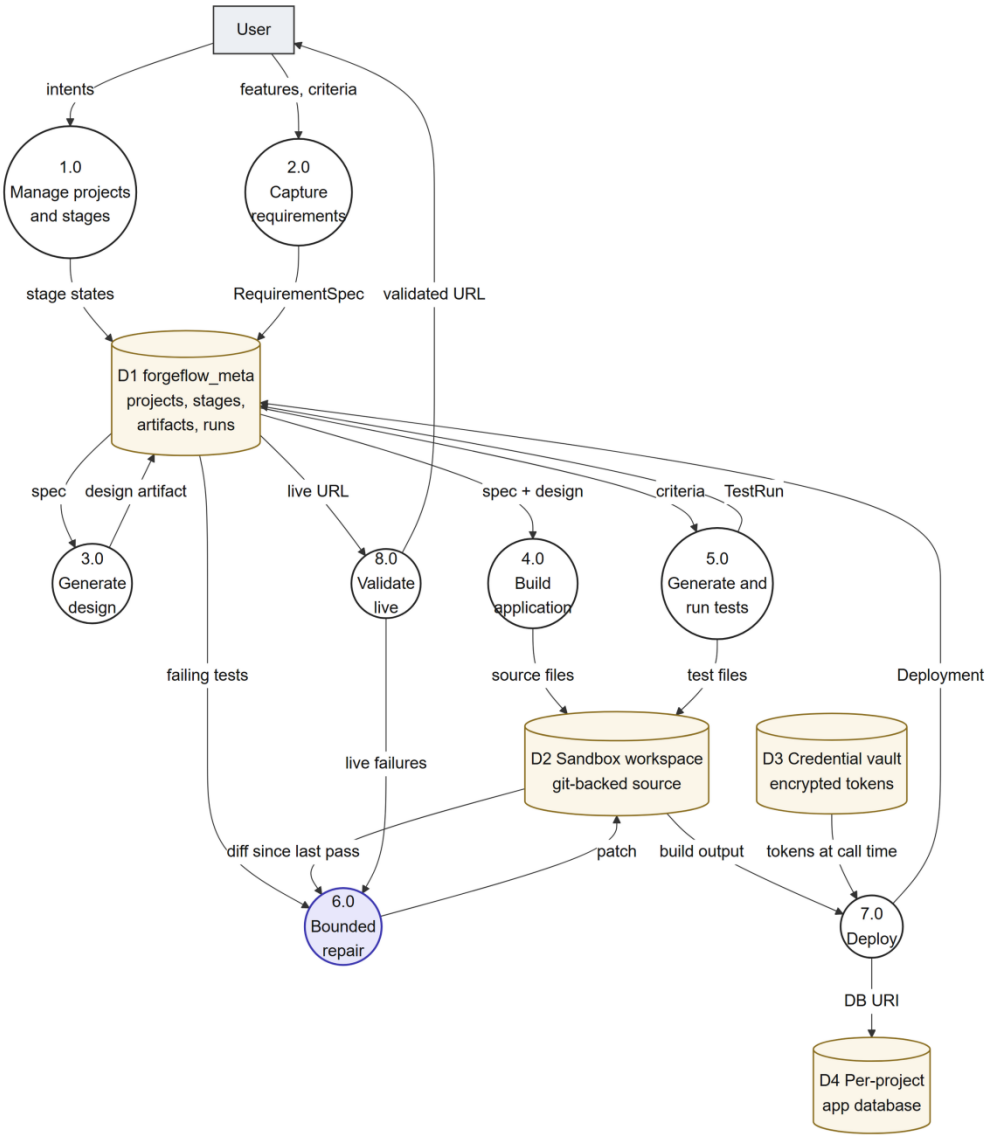


Figure 4.3: Level 1 data flow diagram

4.3 Non-Linear Orchestration

The product order of the stages is requirements, design, build, test, deploy and validate. This order drives the navigator, the default entry stage of a new project and the staleness cascade, but it gates nothing (Figure 4.4). Requirements come before design because a UI should not be designed before it is known what the application has to do; when a

requirement specification exists it is added to the design provider's prompt, but design also works without one.

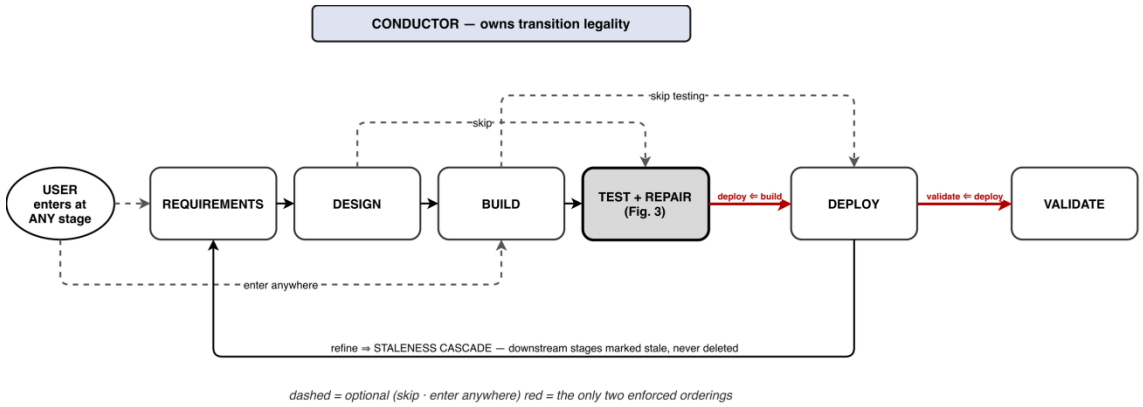


Figure 4.4: The non-linear stage model. Dashed edges are optional (enter anywhere, skip); the two red edges are the only enforced orderings; refining a stage marks later stages stale without deleting them

4.3.1 Stage state machine

Each project keeps an independent StageState for every stage, with one of six statuses: empty, in_progress, awaiting_user, complete, skipped and stale. Figure 4.5 shows the state diagram. The rules are implemented in a pure module with no input or output, which makes every rule exhaustively testable:

- **enter** moves an empty stage to in_progress and is legal regardless of other stages, except for the two hard prerequisites.
- **skip** moves any stage to skipped and records the status it held as a restore point.
- **unskip** restores the recorded status (in_progress is restored as awaiting_user, because whatever was running is gone). This makes skips reversible without a paid rebuild.
- **complete** marks a stage complete; a human can approve a build directly, bypassing the agent's verification gate when the user judges the build good.
- **refine** reopens a complete, skipped or stale stage and marks every later non-empty stage stale; artifacts are never deleted.
- **Hard prerequisites**: entering or completing deploy requires build to be complete, and entering or completing validate requires deploy to be complete. These are the only mandatory orderings; skip, refine and unskip are never blocked.

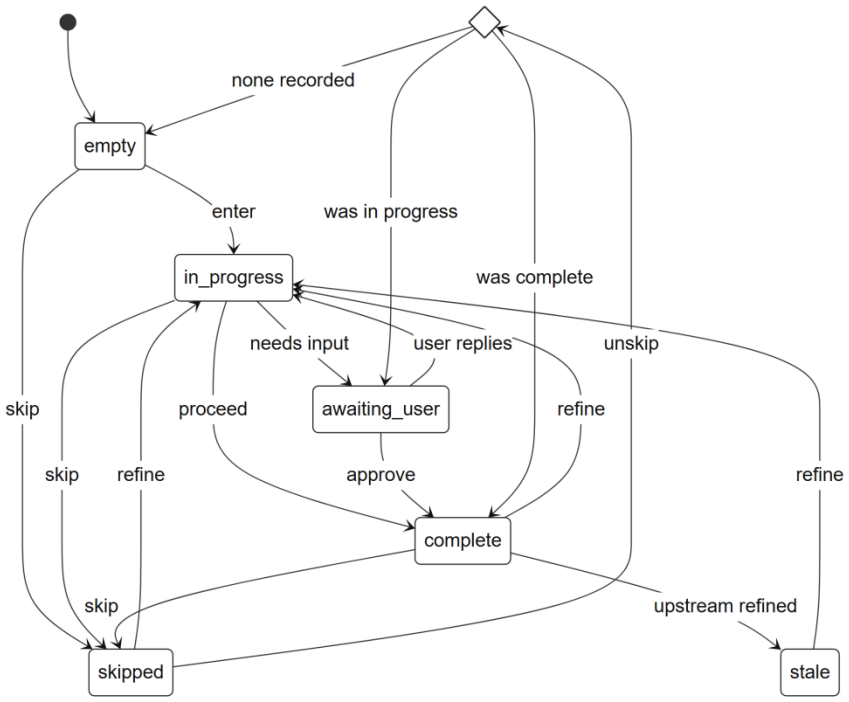


Figure 4.5: State diagram of a pipeline stage (StageState.status)

```
def apply_transition(states, stage, action, *, restore_point=None):
    result = can_transition(states, stage, action)
    if not result.allowed:
        raise ValueError(result.reason)
    from_status = states.get(stage, StageStatus.empty)
    new_states, stale, next_restore = dict(states), (), None
    if action is Action.enter:
        to_status = from_status if from_status is not StageStatus.empty else StageStatus.in_progress
    elif action is Action.skip:
        to_status = StageStatus.skipped
        next_restore = restore_point if from_status is StageStatus.skipped else from_status
    elif action is Action.complete:
        to_status = StageStatus.complete
    elif action is Action.unskip:
        to_status = restore_target(restore_point)
    else:  # refine: reopen this stage, mark every non-empty later stage stale
        to_status = StageStatus.in_progress
        stale = tuple(s for s in downstream_of(stage) if states.get(s) is not StageStatus.empty)
        for s in stale:
            new_states[s] = StageStatus.stale
    new_states[stage] = to_status
    return ApplyResult(new_states, stage, from_status, to_status, stale, next_restore)
```

Listing 4.1: Core of the pure stage state machine (backend/app/projects/state_machine.py, abridged)

4.3.2 The conductor

The conductor is the single owner of transition legality; the user interface only expresses intent. An intent carries a message, a target stage and an action (refine, proceed, skip or unskip). The conductor handles each intent in seven steps:

1. Resolve project ownership and take a snapshot of the stage statuses.
2. Map the user action to a state-machine action (proceed becomes complete; refine becomes refine for a finished stage and enter otherwise) and reject an illegal transition before any side effect.
3. Persist the user's message.
4. Open a costed Run for the work.
5. Dispatch to the stage handler and persist its messages and versioned artifacts.
6. Apply the state transition, which also emits a stage.transition event.
7. Close the Run with its cost and outcome.

Centralising legality in one component keeps non-linearity a property of the system rather than of one screen. An unskip is answered by step 6 alone, because restoring a status requires no handler and produces no artifact.

4.4 Data Design

The control plane stores its state in MongoDB. A document database fits because artifacts are heterogeneous and change shape by design (designs, diffs, test runs, repair attempts, deployments), and because one database per project gives isolation for generated-application data without extra machinery. Figure 4.6 shows the entities and relationships of the control-plane database, and Table 4.1 lists the main attributes.

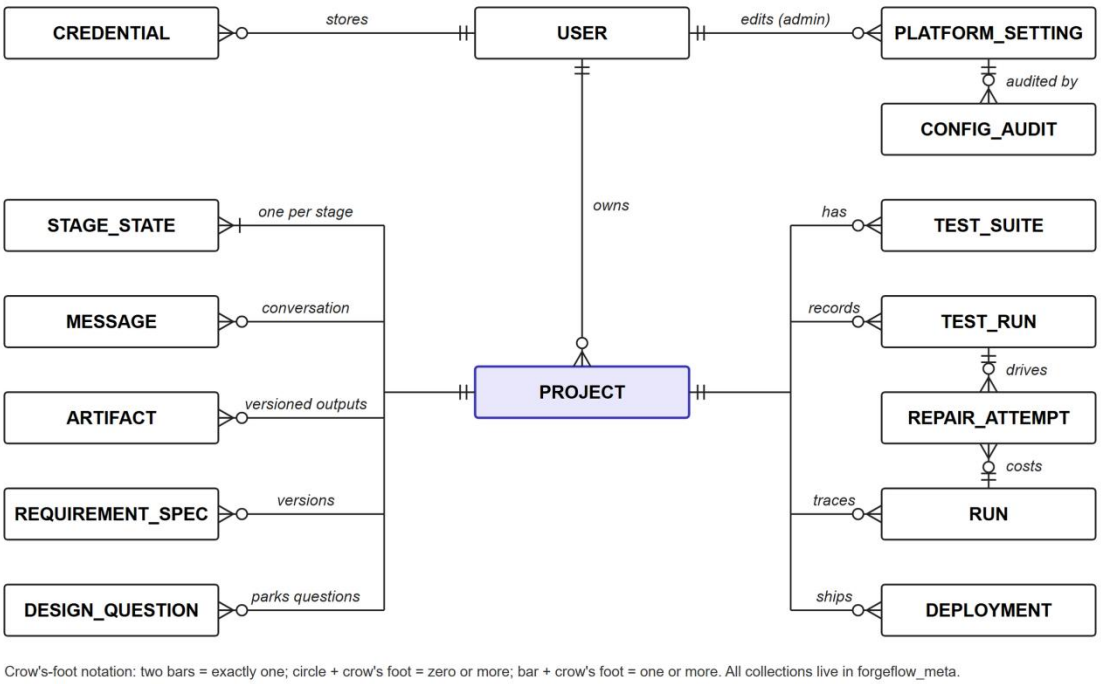


Figure 4.6: Entity-relationship diagram of the control-plane database (forgeflow_meta)

Table 4.1: Data dictionary of the main control-plane collections

Collection	Key attributes	Purpose
users	email, hashed_password (bcrypt), role (user or admin), settings	Accounts and roles
credentials	user_id, kind (vercel, render, mongo_uri, stitch, figma), encrypted_secret, scope (platform or byo)	Encrypted provider tokens
projects	user_id, name, current_stage, stack (fixed-mern-ts), sandbox_id, design_provider, app_db_name	A user's application in progress
stage_states	project_id, stage, status, previous_status (restore point), artifacts	Independent state per stage
messages	project_id, stage, role, content, artifacts, token_usage	Per-stage conversation
artifacts	project_id, stage, type, version, ref (blob pointer), meta	Versioned outputs of every stage
requirement_specs	project_id, app_name, features [name, description, inputs, expected_behaviors, acceptance_criteria (id, text, kind)], version, inferred	Structured requirements
test_suites	project_id, kind (unit or e2e), files, generated_from (criterion ids), version	Generated tests
test_runs	project_id, suite_refs, results, failures, stdout_ref, repair_context_ref, env (sandbox or live)	Normalised test results
repair_attempts	project_id, run_id, iteration, target_files, diff_ref, resulting_run_id, outcome, reverted	One patch iteration
runs	project_id, kind, cost (tokens, inr, reasoning_tokens), progress, tool_calls, outcome, started_at, finished_at	Costed unit of agent work
deployments	project_id, mode (seamless or byo), fe/be/db targets, urls, refs (provider ids), status, topology_snapshot, logs_ref	Deployment records
design_questions	project_id, provider, question, suggestions, status, answer	Questions parked for the user

Collection	Key attributes	Purpose
platform_settings, config_audit	key, value (encrypted if secret), category; before and after values per change	Admin configuration and its audit trail

The data model in Table 4.1 supports the design principles of Section 4.1 directly. Artifacts, requirement specifications and test suites carry a version number, so a stage's output is never overwritten (P4). Each stage has its own record in stage_states, including the restore point that makes a skip reversible, so stages change state independently of one another (P6). The credentials collection stores only encrypted secrets, and the runs, test_runs and repair_attempts collections record the cost, the results and every repair iteration from which the convergence metrics in Section 4.9.4 are computed.

4.5 Class Design

The backend is organised around a small number of interfaces with interchangeable implementations. Figure 4.7 shows the orchestration and repair classes. The Conductor checks legality through the state machine and dispatches intents to stage handlers. The build and validation controllers both reuse a single RepairLoopController rather than implementing their own loops, so every repair in the system, whether triggered by a failing test, a failed build check or a live failure, obeys the same bounds.

In Figure 4.7 the Conductor depends on two collaborators. It asks the StateMachine module, through can_transition and apply_transition, whether an intent is legal, and dispatches each legal intent to a StageHandler, an interface implemented by the build, design, deploy and validate handlers. The build handler delegates verification to a BuildVerificationController, and the validate handler delegates live checks to a LiveValidationController that forwards live failures. Both feed the RepairLoopController, which is composed of four parts: a RepairContextAnalyzer that assembles the minimal context, a RepairAgent that proposes a patch, ConvergenceMetrics that track the failing

count, regressions, tokens and rupee cost per iteration, and an Escalation record holding the reason, the summary and the diffs already tried.

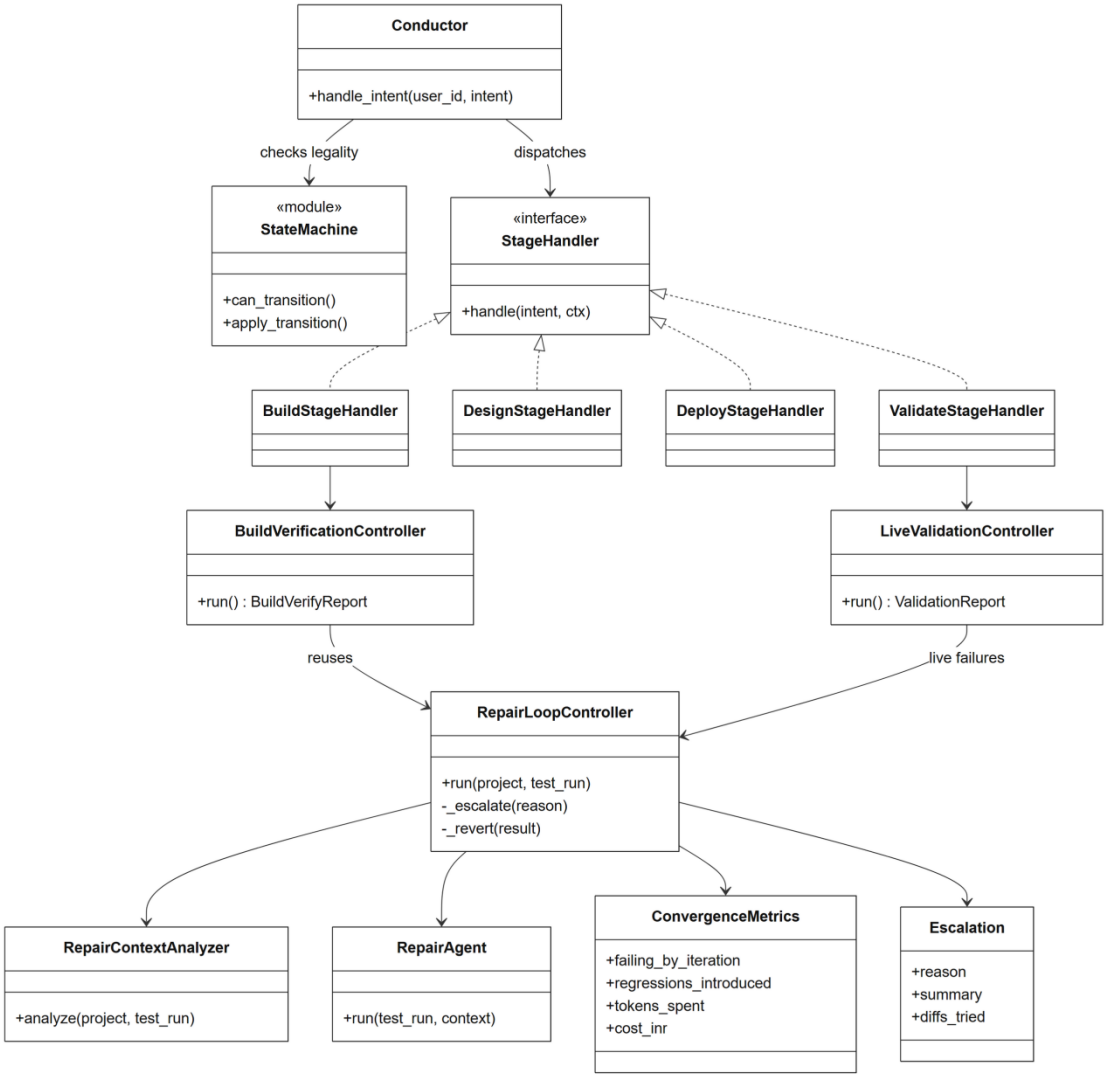


Figure 4.7: Class diagram of the orchestration and repair core

Figure 4.8 shows the provider abstractions. DesignProvider has Stitch, Figma and fake implementations, so switching providers needs no change to the design stage. DeployProvider has Vercel and Render implementations that share a base class. The Transport protocol hides whether a model call goes to the Anthropic SDK or to an OpenAI-compatible endpoint, so no agent, tool or cost code depends on a vendor.

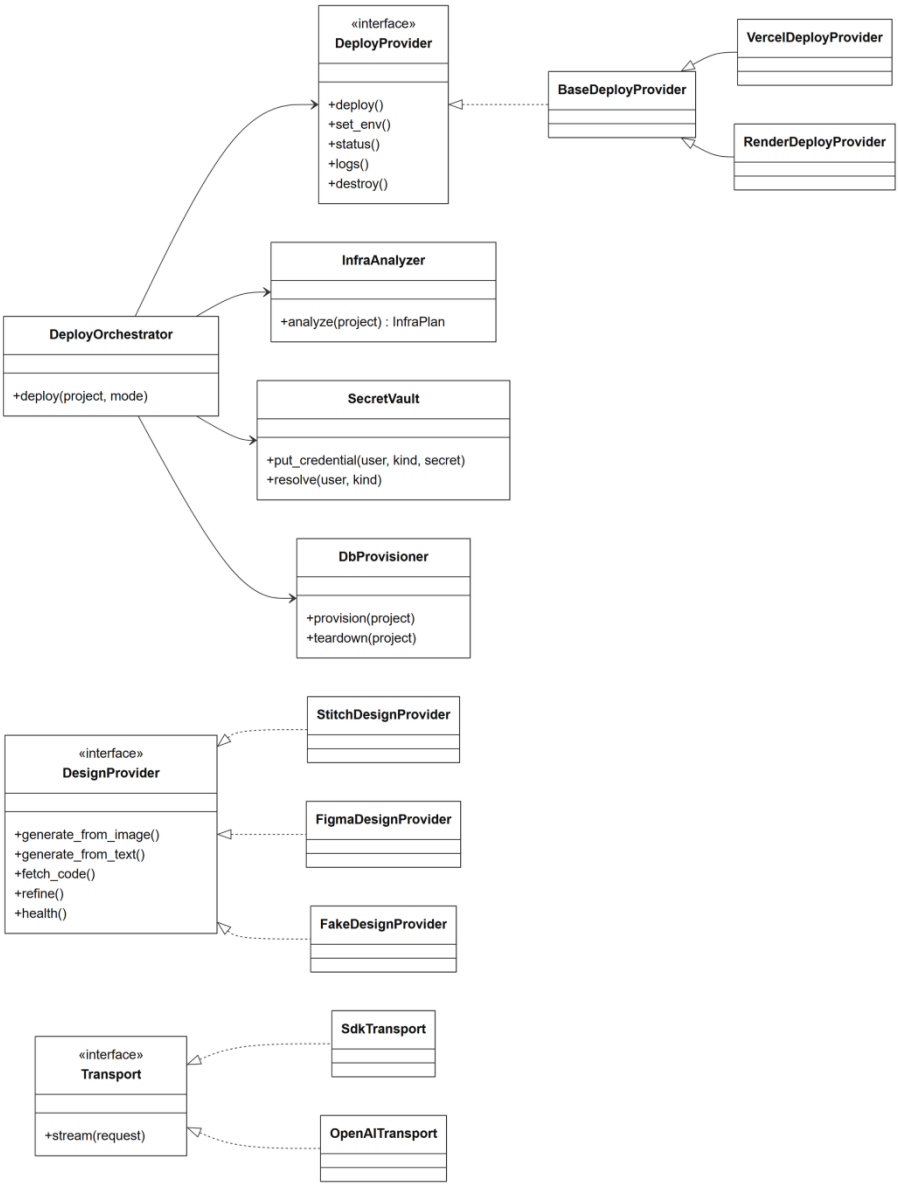


Figure 4.8: Class diagram of the provider abstractions and deployment services

4.6 Requirements and Design Stages

The first two stages of the pipeline prepare the inputs to the build: a structured specification of what the application must do, and a UI design of how it should look. Both stages are optional; when either is skipped, the build falls back to a deterministic plan instead of refusing to run (Section 4.7).

4.6.1 Structured requirements capture

Requirements are captured as a RequirementSpec: a list of features, each with a name, a description, inputs, expected behaviours and acceptance criteria. Every acceptance criterion has an identifier and a kind (unit, end-to-end or either) that tells the test generator how to verify it. Capture is deliberately structured rather than free prose, because structured input makes test generation deterministic and evaluable. There are two entry points: a guided form that adds features one at a time, and an AI draft that turns a free-text description of the whole application into a proposed feature list using the cheap routing model. The draft is never saved automatically; it is shown for editing, in keeping with the human-control principle. Listing 4.2 shows part of the specification used for the demonstration application.

```
requirements:
  features:
    - name: Clear completed
      description: A single action that removes every completed todo at once, leaving the rest.
      expected_behaviors:
        - Clicking "Clear completed" removes all done todos in one step.
        - Incomplete todos are never touched by "Clear completed".
      acceptance_criteria:
        - text: Clear completed removes every completed todo from the list and from storage.
          kind: e2e
        - text: Clear completed leaves all incomplete todos untouched, even when some are complete.
          kind: unit
```

Listing 4.2: Part of the hero demonstration specification (eval/specs/hero-todo.yaml)

Because no stage is mandatory, a user may skip requirements entirely. In that case the test generator warns and offers either to capture requirements or to infer a minimal set from the code; it never refuses to run.

4.6.2 Pluggable design stage

The design stage is built on a DesignProvider interface with five operations: generate from images, generate from text, fetch the generated HTML and CSS, refine an existing design, and report health and capabilities. Google Stitch is the default provider and Figma is the

alternative; a fake provider supports testing. Switching providers requires no change to the stage handler. Credentials are held centrally, short-lived access tokens are refreshed automatically, and monthly quota use is tracked per provider. Each ForgeFlow project generates into its own Stitch project, named after the application, so designs never leak between projects.

Two behaviours deserve mention. First, provider failures such as quota exhaustion fail soft: a resilient wrapper falls back to the next provider with a clear message instead of ending the pipeline. Second, a provider's question is treated as a question, not a failure. Stitch often answers a broad brief with a proposal such as “shall I proceed with these five screens?”. A plain scope proposal is accepted automatically up to a configured number of replies; anything else is stored as a DesignQuestion, the stage moves to awaiting_user, and the user's next message is sent back to the provider that asked. Earlier, a question had been treated as an error and the fallback produced a design nobody had asked for; routing the question to the human fixed this, in line with the evidence that code generators guess rather than ask when a brief is ambiguous (Fang et al., 2026).

4.7 Build Stage

The build stage turns the requirements and design into a working application in the project's sandbox. One press of Build is meant to produce a verified, working application without the user repeatedly telling the agent what is broken. Figure 4.9 shows the activity diagram.

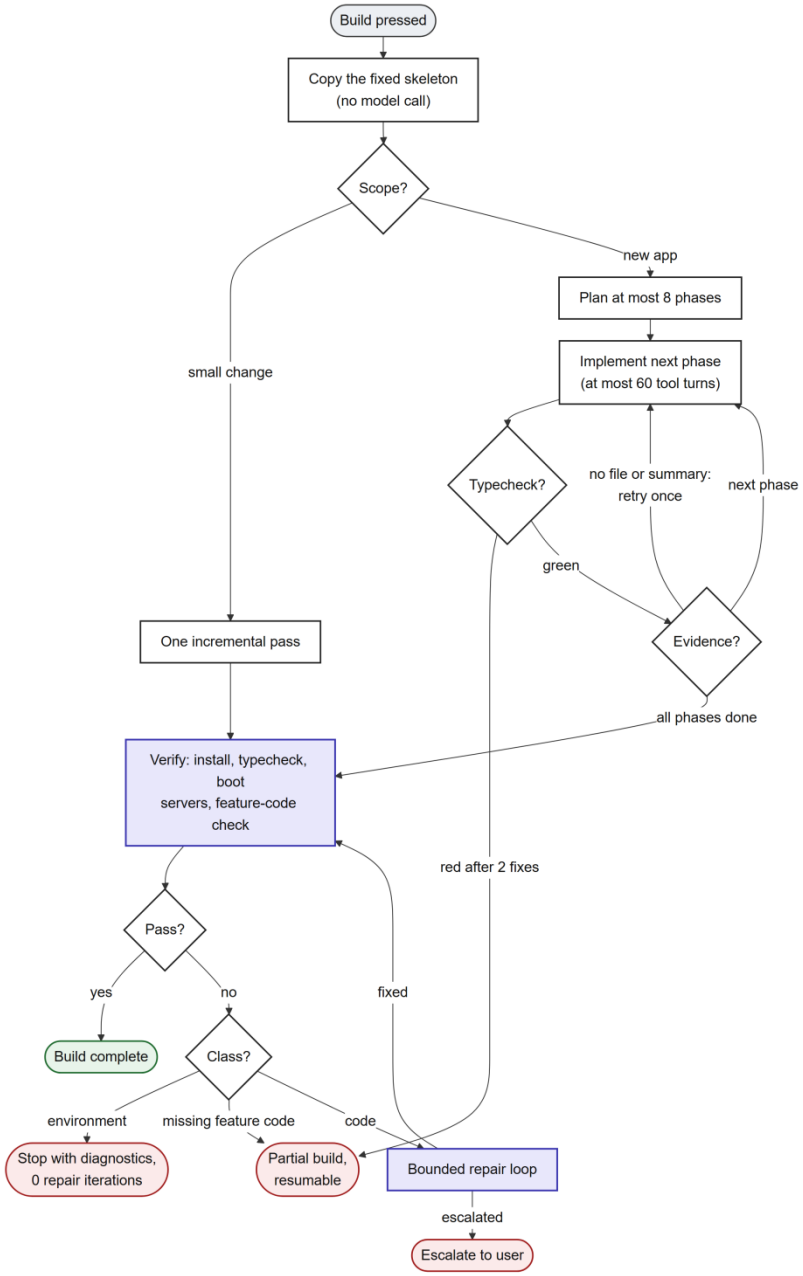


Figure 4.9: Activity diagram of the build stage

1. **Skeleton instantiation.** The fixed-stack skeleton (frontend, backend, test configurations and a test-database helper) is copied into the git-backed workspace. No model call is involved, so no tokens are spent on scaffolding.
2. **Scope classification.** A change request is first classified by the cheap MODEL_CLASSIFY role (with a heuristic fallback). A small change is handled in a single incremental pass instead of a full re-plan.

3. **Phased plan.** For a new application, a planner produces at most eight phases, saved as a plan in the workspace. A deterministic fallback plan is used when there are no requirements or no design, so build always runs.
4. **Phase-by-phase implementation.** Each phase is implemented by the code-generation agent through a tool loop of at most 60 turns, using a small context that summarises earlier phases. Each phase has its own typecheck gate with at most two fix attempts.
5. **Evidence-gated completion.** A phase counts as done only if it wrote at least one file, ended with a non-empty summary and passed the typecheck; a phase that wrote nothing gets one retry with explicit feedback. The tool loop is reasoning-aware: a turn cut off by the output-token limit is nudged and continued, at most twice, instead of being mistaken for a finished turn.
6. **Verification.** The build is verified by installing dependencies, typechecking, booting both development servers and checking their health, and then running a structural check that feature code really replaced the skeleton (feature files on both sides, a feature router mounted, and the index route rendering feature code).
7. **Failure classification and self-heal.** A failure is classified before anything else. Environment failures (for example an unreachable package registry, process-limit exhaustion or an unresolvable database host) stop the build with diagnostics after zero repair iterations, because no code patch can fix them. Missing feature code marks the build partial and resumable. Only code-class failures reach the shared repair loop, capped at three iterations for a build, and the whole build is bounded at 3,600 seconds.

Every step streams progress events, and the code appears in the Monaco editor as the agent writes it. The user can approve a build directly, mark it complete without paying for another run, or send a change request.

4.8 Requirements-Driven Test Generation and Execution

A single identifier, criterion_id, links a user's acceptance criterion to its tests, their results, the failure analysis and the repair context (Figure 4.10). The link is built in from the start rather than recovered afterwards.

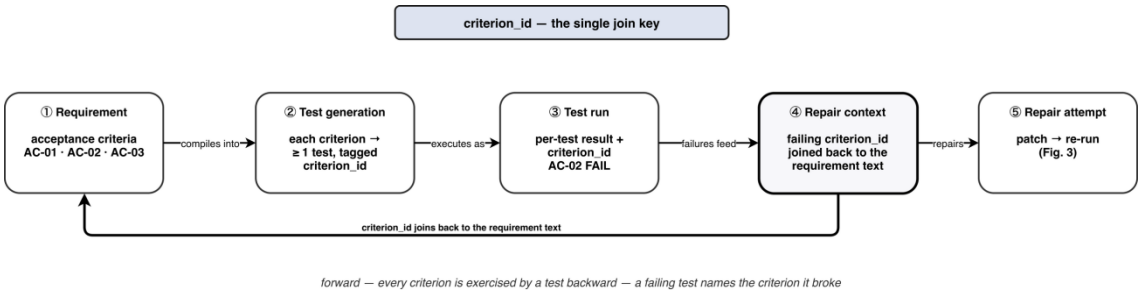


Figure 4.10: Requirement-to-repair traceability through criterion_id

- **Generation.** The test-generation agent maps each acceptance criterion to at least one executable test tagged with its criterion_id: Vitest with Testing Library for frontend units, Jest with supertest for backend units, and Playwright for end-to-end behaviour against the sandbox preview.
- **Offline test database.** Sandboxes have no internet access while tests run. The sandbox image therefore bakes in a checksum-verified mongod 8.0 binary, and the skeleton provides a useTestDb() helper and a Jest global setup, so every test reaches a real in-memory MongoDB without downloading anything. A missing test database is recognised deterministically and stops repair at zero attempts.
- **Execution and normalisation.** Suites run inside the sandbox. JUnit and JSON reporter output is normalised into one per-test record with status, assertion, stack, referenced files and criterion_id. A failing criterion is reported as unmet rather than hidden inside an aggregate pass count, and live validation produces the same shape, so the repair path does not depend on which framework or environment produced the failure.
- **Last-passing reference.** A git reference is moved forward on every green run. The diff from this reference to the current code answers the question “what changed since it last worked”.

4.9 The Bounded, Diff-Aware Self-Healing Repair Loop

The repair loop is the core contribution of the project. Its input is a test run with at least one failure, the current workspace and the requirements behind the failing tests. Its design follows four invariants: never feed the whole repository when a diff suffices; never loop without a bound; always leave an auditable trail; and always prefer escalation to a silent failure. Figure 4.11 shows the loop.

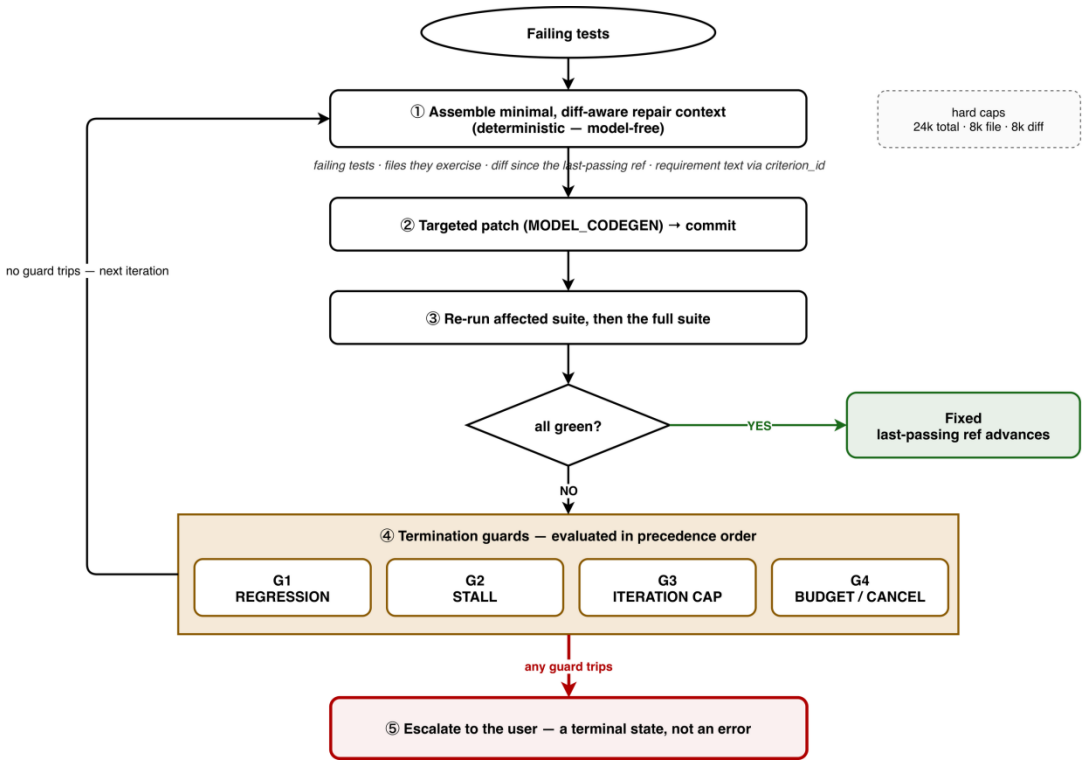


Figure 4.11: The bounded self-healing repair loop: minimal context assembly, targeted patching, re-execution, termination guards in precedence order, and escalation as a terminal state

4.9.1 Minimal, diff-aware repair context

Context assembly is deterministic and does not call a model, which makes it exhaustively testable and cheap. It admits only four kinds of content:

1. **The failing tests**: name, framework, assertion, stack, referenced files and criterion_id.
2. **The source files those tests reach.** The starting set is the failing test and the files named in its stack. It is widened along the failing test's import graph by a breadth-first walk bounded at depth 3 and 24 files, in which bare package specifiers never

resolve (so node_modules cannot enter the context) and test files are filtered out. Direct imports rank above folder neighbours. If the subject of a test is registered rather than imported, the agent may request up to three additional non-test files per attempt.

3. **The diff since the last passing run**, scoped to the implicated files.
4. **The requirement text** for each failing criterion, joined through criterion_id, so the agent repairs against the acceptance criterion and not only against the assertion.

Everything else, including the rest of the repository, passing tests and untouched modules, is excluded. The context is capped at 24,000 characters in total, 8,000 per file and 8,000 for the diff. Oversized parts are trimmed in priority order and every dropped part is recorded, so a trimmed context is never silently lossy. The import-graph walk was added after a live session in which repair was handed main.tsx while the bug lived in a page component one directory away; the model correctly refused to patch the wrong file. Following the test's own imports, as graph-based localisation research recommends (Ouyang et al., 2025; Z. Chen et al., 2025), fixed the problem while keeping the context minimal.

4.9.2 Loop controller

The controller runs one attempt per iteration: assemble the context, call the repair agent with that context and a strict instruction to fix the failing behaviour without breaking passing tests, commit the patch, and re-run the affected suite followed by the full suite. Algorithm 4.1 gives the complete procedure.

Algorithm 4.1: Bounded, diff-aware repair loop (RepairLoopController.run)

```
Input: project P, failing test run R; bounds N (iterations, default 5), S (stall, 2), G (regressions, 2), Z (empty attempts, 2)
Output: outcome in {fixed, escalated}, convergence metrics, list of attempts
F ← failing(R); if F is empty then return fixed
stall ← 0; reg ← 0; noop ← 0
for i ← 1 to N do
    if the run was cancelled then return escalate(cancelled)
    C ← assemble_context(P, R)        ▷ deterministic, no model call
    if C names a broken environment then return escalate(environment)   ▷ zero tokens spent
    A ← repair_agent(C)               ▷ a budget halt returns escalate(budget)
    if A had nothing safe to patch then return escalate(blocked)
    R′ ← test run after A; F′ ← failing(R′)
    shrank ← |F′| < |F|; regressed ← A broke a previously passing test
    if regressed then reg ← reg + 1
    if A wrote no file then noop ← noop + 1 else noop ← 0
    record attempt i with outcome regressed, fixed or no_progress; emit repair.iteration
    if regressed and not shrank and R′ is not green then revert to the pre-attempt commit
    else R ← R′; F ← F′
    if R′ is green then return fixed
    if reg ≥ G then return escalate(regressed)
    if noop ≥ Z then return escalate(no_patch)
    if shrank then stall ← 0 else stall ← stall + 1
    if stall ≥ S then return escalate(stalled)
end for
return escalate(cap_reached)
```

The guards are evaluated in a deliberate order. The regression guard comes first because "my patches keep breaking working code" is a more precise and more actionable diagnosis than "no progress"; a single regression is tolerated because a legitimate patch can briefly break one test. The empty-attempt guard comes next: the context is assembled deterministically from the failing run, so an attempt that wrote nothing would be asked the identical question again, and the agent's own explanation of what it needed is more useful to the user than a generic stall. The stall guard trips when the failing set has not shrunk for two consecutive iterations; a set that oscillates at the same size counts as no progress. The iteration cap of five sits just above the range of three to four iterations in which the literature finds most of the achievable gain (Kiecker et al., 2026; Arimbur, 2026). An attempt that broke something and fixed nothing is reverted, so the loop may spend iterations but never hands back an application worse than it found.

Every attempt is judged as regressed, fixed or no_progress, with regression dominating: an attempt that fixes one test and breaks another is still treated as the dangerous kind. All thresholds are resolved from the layered configuration (admin panel, then environment, then default), so they can be tuned from the admin dashboard and swept by the evaluation harness without redeploying.

4.9.3 Escalation as a terminal state

Any ending other than green is an escalation, and an escalation is a normal outcome rather than an error. The stage moves to awaiting_user and a repair.escalation event carries the reason, a short “here is where I am stuck” summary, the failing tests with their criterion ids, the diffs already tried, the convergence metrics and a resume contract (refine the build stage with guidance; the build regenerates and repair can run again). A summary is generated from a fixed template of the form: “I stopped after N attempt(s) because <reason>. K test(s) are still failing: <names>. I patched: <files>. Failing count per attempt: <trail>. Tell me what to change and I'll rebuild from your guidance.” For an environment failure the closing sentence instead asks the user to fix the environment and run the tests again.

4.9.4 Audit trail and convergence metrics

Each iteration is stored as a RepairAttempt with its target files, a reference to its diff, the resulting run and whether it was reverted. The whole loop is one costed Run, and a versioned repair report records the initial and final failing counts, the failing count after every iteration, regressions introduced, empty attempts, iterations, tokens, rupee cost and wall-clock time. These are exactly the quantities the evaluation harness consumes. Table 4.2 summarises the bounds that keep repair and build finite.

Table 4.2: Default bounds of the repair loop and the build (all admin-tunable)

Setting	Default	Effect
repair_max_iterations	5	Hard cap on repair iterations per loop
repair_stall_threshold	2	Consecutive iterations without the failing set shrinking
repair_max_regressions	2	Attempts that broke passing tests before escalation
repair_max_noop_attempts	2	Consecutive attempts that wrote nothing
repair_context_max_chars / file / diff	24,000 / 8,000 / 8,000	Character budgets for the repair context
repair_import_max_depth / max_files	3 / 24	Bounds on the import-graph walk
repair_request_file_max	3	Extra files the agent may request per attempt
build_max_phases	8	Maximum phases in a build plan
build_phase_max_tool_turns	60	Tool-loop bound per phase
build_phase_max_fix_attempts	2	Typecheck fixes per phase
build_integration_max_iterations	3	Repair cap for build verification
build_max_wall_clock_s	3,600	Outermost bound on a whole build
validate_max_cycles	2	Repair, redeploy and re-validate cycles for live failures
llm_max_truncated_turns	2	Continuations after a turn hits the output-token limit

Every bound in Table 4.2 is resolved through the layered configuration, so it can be changed from the admin dashboard (Figure 4.20) without redeploying, and the evaluation harness can sweep it. The repair-loop defaults follow the repair literature reviewed in Section 2.1.3, and the build bounds ensure that even a build which never converges ends within an hour.

4.10 Deployment and Live Validation

Deployment and live validation follow the flow in Figure 4.12.

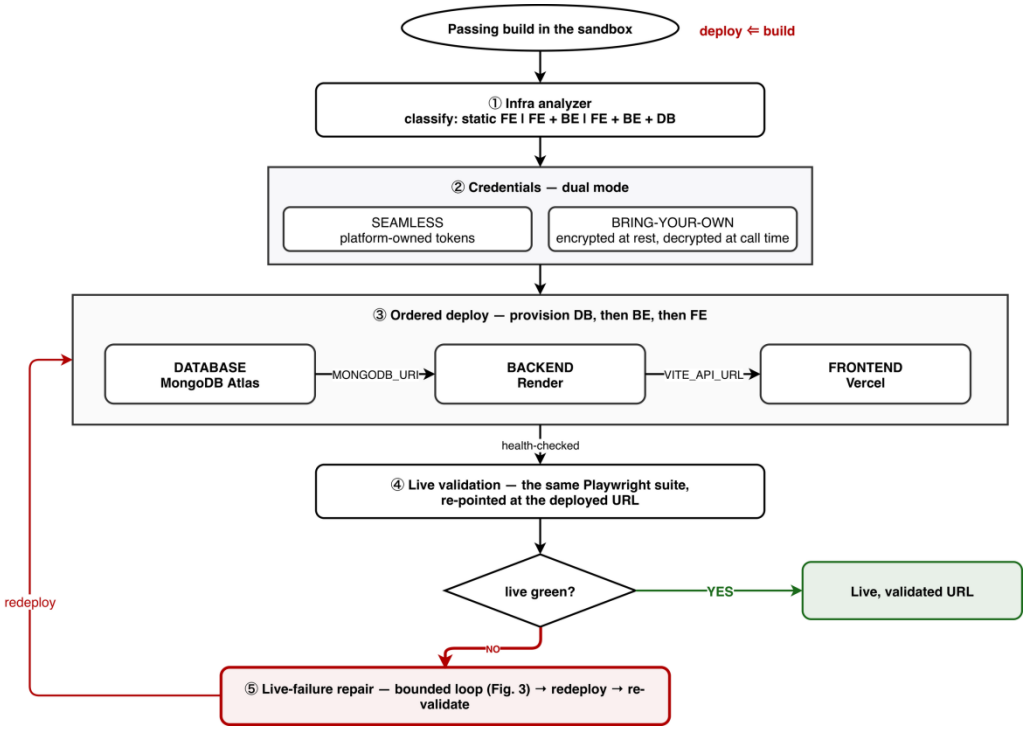


Figure 4.12: Infrastructure analysis, dual-mode credentials, ordered deployment and live validation with its repair path. The figure shows the original Render backend; since phase 58 the backend deploys to Vercel by default, with Render available as an option

4.10.1 Infrastructure analysis

An infrastructure analyzer reads the workspace and classifies the application as a static frontend, a frontend with a backend, or a frontend with a backend and a database. It detects the frontend build target, whether a server framework, start script and port are present, whether Mongoose or a connection string is used, and every environment variable referenced in code. The analysis is static and model-free, so it is cheap and reproducible. Where the evidence conflicts, the analyzer flags it with a lower confidence instead of guessing. One rule protects security: any variable prefixed with VITE_ is compiled into the browser bundle, so it is never treated as a secret, and a secret-looking VITE_ name is flagged as a leak.

4.10.2 Dual-mode, ordered deployment

ForgeFlow deploys either in seamless mode, on platform-owned accounts, or in bring-your-own mode with the user's tokens. All tokens are encrypted at rest with Fernet,

decrypted only in process at call time, never logged and never returned to the client. Deployment is ordered: provision the project's database; deploy the backend as a Vercel function with the database URI injected; deploy the frontend with the backend's public URL injected; and health-check every node. A dead backend can never ship behind a live frontend. Both tiers are delivered by the same inline file upload, so no git host is needed, and what ships is exactly what the repair loop validated.

Deployment is also truthful about what it did. Environment variables are mirrored into the provider project rather than living only on one immutable deployment; provider deployment identifiers are recorded so that the topology shows the provider's real build and runtime logs; a pre-flight check refuses a localhost database URI before anything is deployed; and a deployment can be torn down, which returns the deploy stage to empty and so restores the rule that validation requires a deployment. These behaviours were added after a live session exposed each gap, as reported in Section 5.4.

4.10.3 Post-deployment validation

After deployment, the same Playwright suite is pointed at the live URL. The suite can be narrowed to tests tagged smoke or critical. The sandbox is lent an egress network only for the duration of the run, and it is always detached in a finally block; a cold site is warmed before the suite starts. Live results have the same normalised shape as sandbox results, so a live failure enters the same repair loop: diagnose, repair, redeploy and re-validate, for at most two cycles. On success, the final link is recorded and delivered to the user.

4.11 Sandbox, Security and Threat Model

The sandbox is the most important trust boundary. Each project gets its own container created by the sandbox manager with the specification in Listing 4.3.

```
def build_run_kwargs(project_id, *, image, cpu_limit, mem_limit, pids_limit, read_only):
    return {
        "image": image,
        "name": container_name(project_id),
        "network_mode": sandbox_network_name(project_id),   # per-project internal network: no host, no internet
        "cap_drop": ["ALL"],
        "security_opt": ["no-new-privileges"],
        "read_only": read_only,                          # read-only root filesystem
        "pids_limit": pids_limit,                        # default 256
        "mem_limit": mem_limit,                          # default 1g
        "nano_cpus": int(cpu_limit * 1_000_000_000),        # default 1 CPU
        "tmpfs": {"/tmp": ""},
        "volumes": {volume_name(project_id): {"bind": WORKSPACE, "mode": "rw"},
                    home_volume_name(project_id): {"bind": SANDBOX_HOME, "mode": "rw"}},
    }
```

Listing 4.3: Isolation posture of a project sandbox (backend/app/sandbox/manager.py, abridged)

The container runs as a non-root user from an image containing Node.js 20, a pinned pnpm, Playwright with Chromium, git and a test-only mongod. Its default network is internal and per project. Only two relaxations exist, both deliberate and narrow: the preview network, which is also internal, and an egress network attached only for dependency installation and live validation and then detached. Idle sandboxes are reaped after 15 minutes. Table 4.3 summarises the written threat model.

Table 4.3: Main threats and mitigations from the threat model

Threat	Mitigation	Verified by
Generated code reaches the host or the internet	Sealed per-project internal network; preview and egress attached deliberately and temporarily	Sandbox isolation tests
Privilege escalation in the container	All capabilities dropped, no-new-privileges, non-root user	Sandbox isolation tests
Resource exhaustion (fork bomb, memory, CPU)	PID, memory and CPU limits; execution timeouts; idle reaping	Sandbox and execution tests
Docker-socket escape	The socket is never mounted into a sandbox (checked by a source-wide scan)	Security test suite
Path traversal out of the workspace	Single safe-path checkpoint plus symlink resolution at call time	Path-safety tests
Cross-project data access	Authorisation on every route; one database per project; no endpoint accepts a database name	Data-browser isolation tests
Secret disclosure	Fernet encryption at rest, decryption at call time, no read-back endpoint, log redaction	Vault no-leak tests
Denial of wallet	Per-project and global budget caps; bounded loops everywhere	Budget-cap tests

The threats in Table 4.3 share one assumption: the generated code, and the model that wrote it, may behave in any way. The mitigations therefore rely on the container, the network and the authorisation layer rather than on the model's behaviour, and each is backed by an automated test, so a change that weakened a mitigation would fail the test suite instead of passing unnoticed.

4.12 LLM Transport, Cost Discipline and Configuration

All model access goes through one Transport seam with two implementations: the Anthropic SDK and a pure OpenAI-compatible Chat Completions adapter, which is the primary interface because most providers implement it. Above the seam, no code depends on a vendor's request shape, model names or error format. Providers spell parameters differently; for example, one model rejected max_tokens and required max_completion_tokens. Rather than hard-coding such cases, the transport reads the provider's own 400 error, repairs the request (renaming or dropping a parameter, changing the system role, or falling back to non-streaming), retries, and remembers the working shape for that endpoint and model. The repair ladder is bounded, each repair must be new, and it never silently removes an output cap.

Cost discipline is enforced by role. MODEL_CODEGEN is used only for code, test and repair work; MODEL_ROUTING for summaries and routing; and MODEL_CLASSIFY (which defaults to the routing model) for classification such as build scope. Model identifiers are configuration, not identity, and are priced through a pricing table. Every call records tokens and rupee cost against its Run; per-project and global budget caps warn at 80% and then halt work with a clear message, which the repair loop turns into a budget escalation. On OpenRouter, where one model can be served by several upstream providers at different prices, requests ask for the cheapest upstream first while keeping fallbacks.

Configuration is layered with the precedence admin panel, then environment, then code default. Every setting (more than 100, in 15 sections) can be viewed and edited from the admin dashboard. Secrets entered there are write-only and encrypted; only the encryption key and the control-plane database settings remain environment-only, because they are needed before the configuration layer exists. A CI guard forbids reading environment variables outside the configuration layer, and every change is written to an audit log.

4.13 User Interface

The following screenshots were captured from the running application. The landing page (Figure 4.13) introduces the six stages. The dashboard (Figure 4.14) lists projects with their progress across the stages.

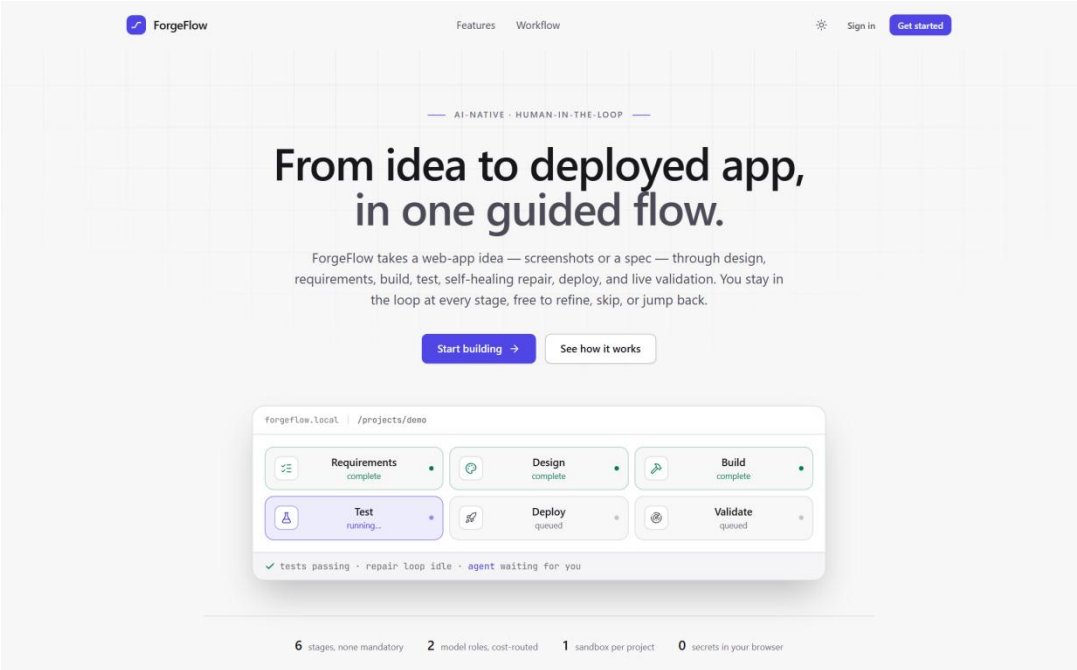


Figure 4.13: Landing page of the ForgeFlow PWA

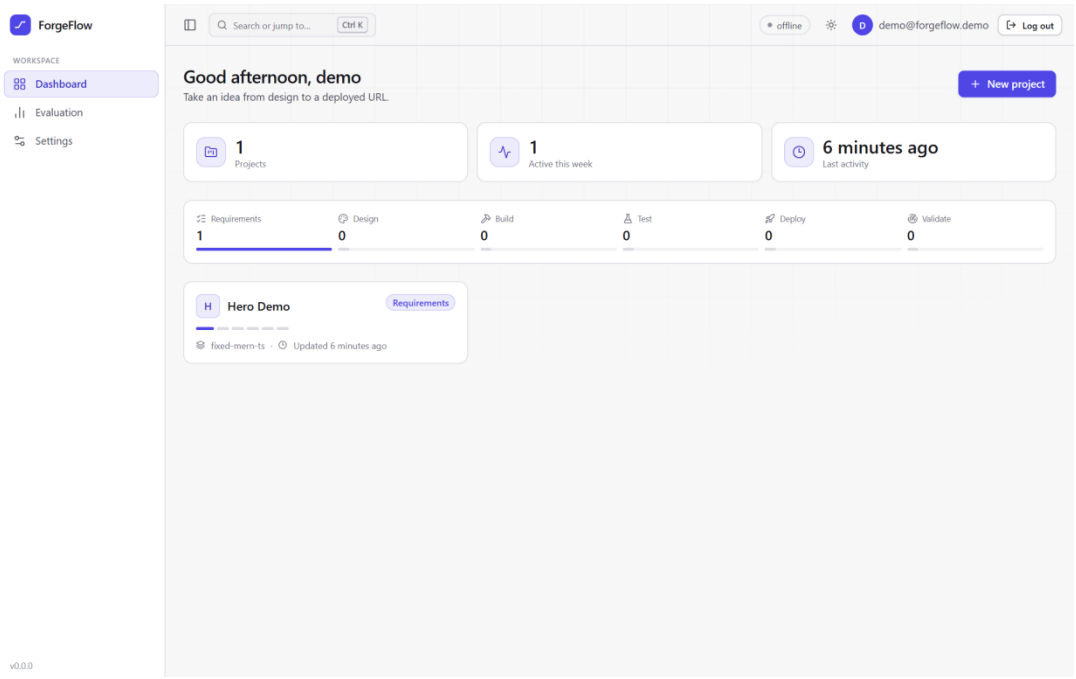


Figure 4.14: Projects dashboard with per-stage progress

The workspace (Figure 4.15) has the clickable stage navigator on the left with each stage's status and the hard-prerequisite locks on deploy and validate, the cost panel below it, the active stage panel in the centre and the stage's conversation on the right. Panels can be resized, collapsed or focused. Here the requirements of the hero demonstration are complete, with each acceptance criterion labelled by its kind.

Requirements are captured through the guided form in Figure 4.16, which adds one feature at a time with its inputs, expected behaviours and acceptance criteria. The optional AI draft turns a free-text description of the whole application into a proposed feature list, which is shown for editing and never saved automatically.

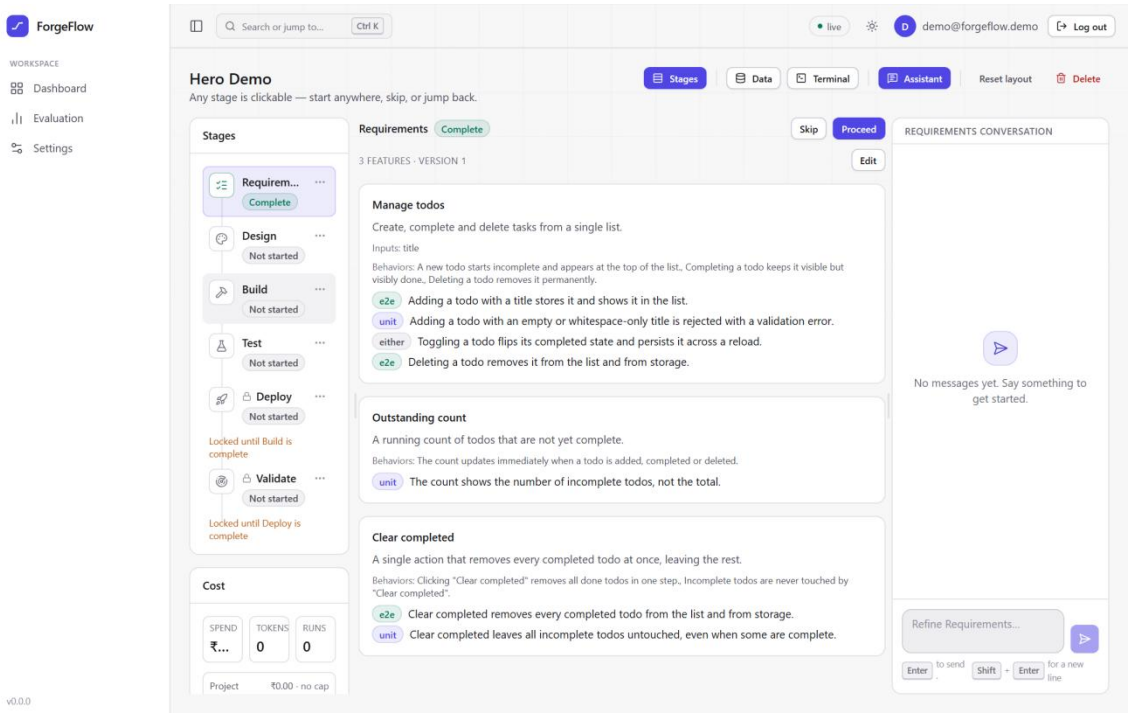


Figure 4.15: Project workspace showing the stage navigator and captured requirements

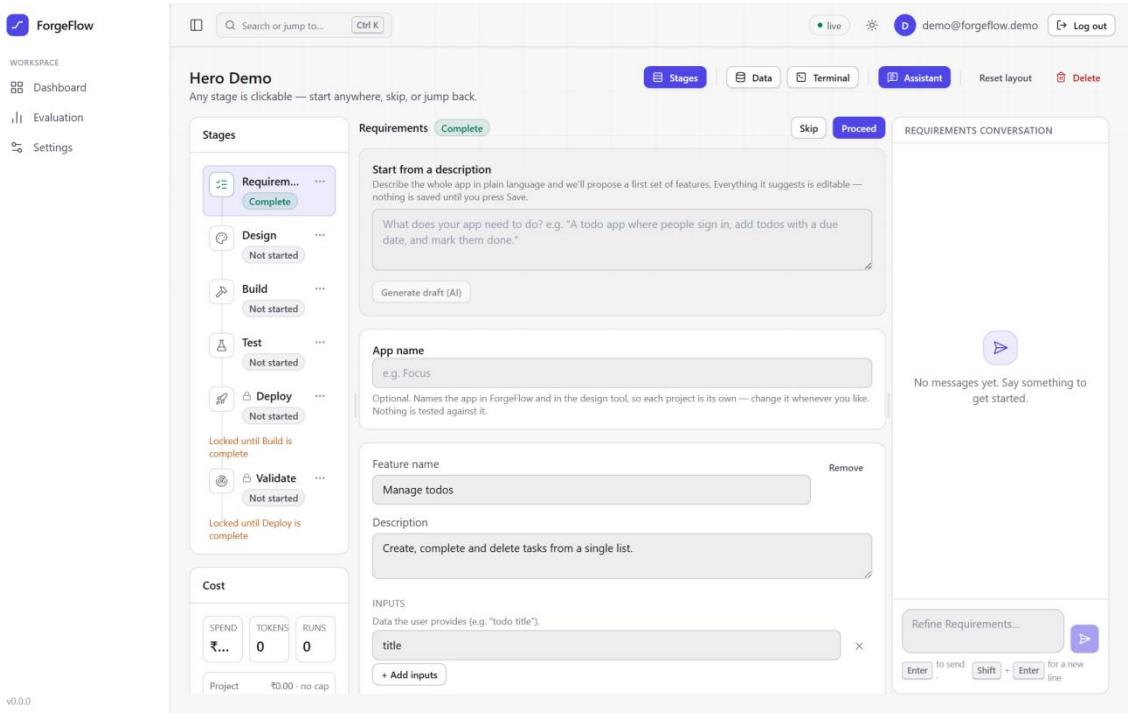


Figure 4.16: Guided requirements form with the optional AI draft from a description

The design stage (Figure 4.17) offers generation from text, upload of screenshots and import of an own design, with a version list and a refinement box; the badge next to the title shows the provider's health. The deploy stage (Figure 4.18) offers seamless and bring-

your-own modes, shows the frontend, backend and database topology, and explains that deploy is locked until a build is complete.

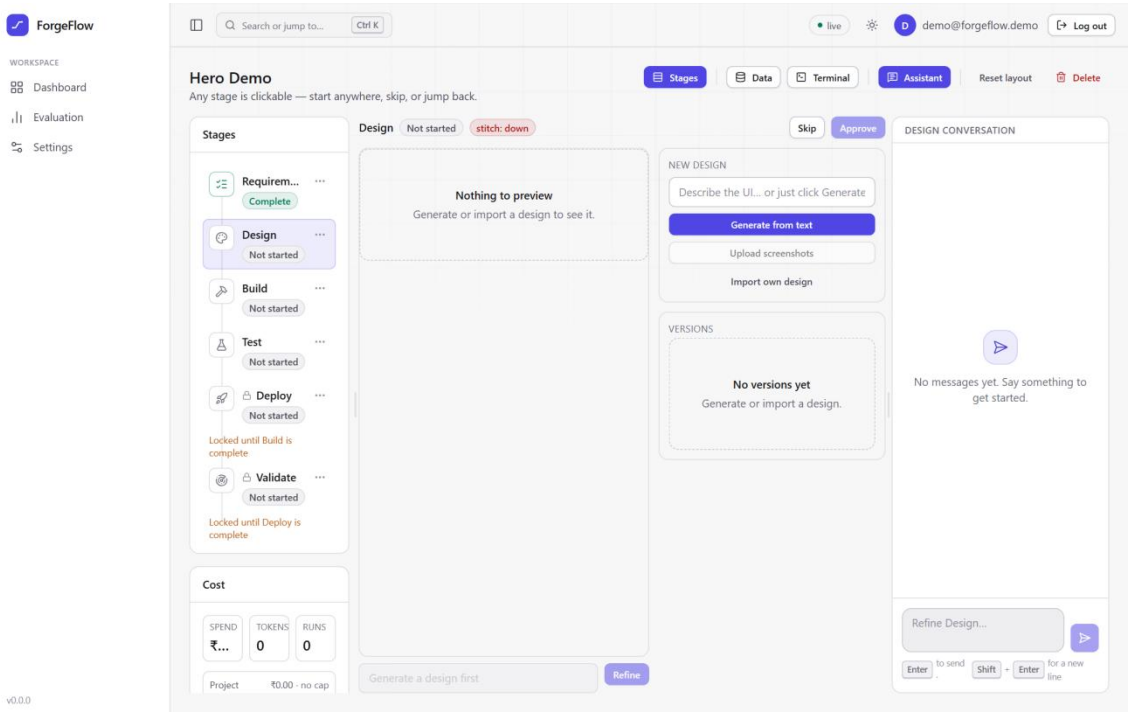


Figure 4.17: Design stage with text, screenshot and own-design intake

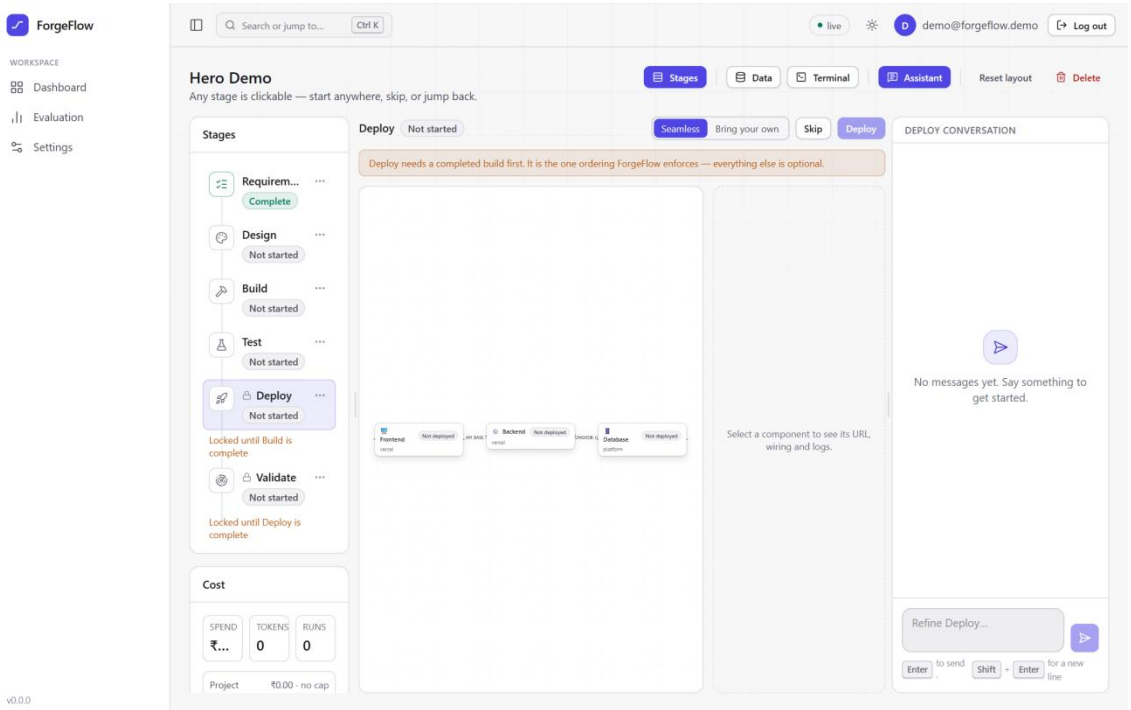


Figure 4.18: Deploy stage with dual-mode selection and the deployment topology

Users store their own provider tokens in Settings (Figure 4.19); tokens are encrypted and never shown again. Administrators tune the platform from the admin dashboard; Figure 4.20 shows the repair-loop section with the iteration cap, stall threshold, regression limit and context budgets, each showing its source (default, environment or admin) and allowed range.

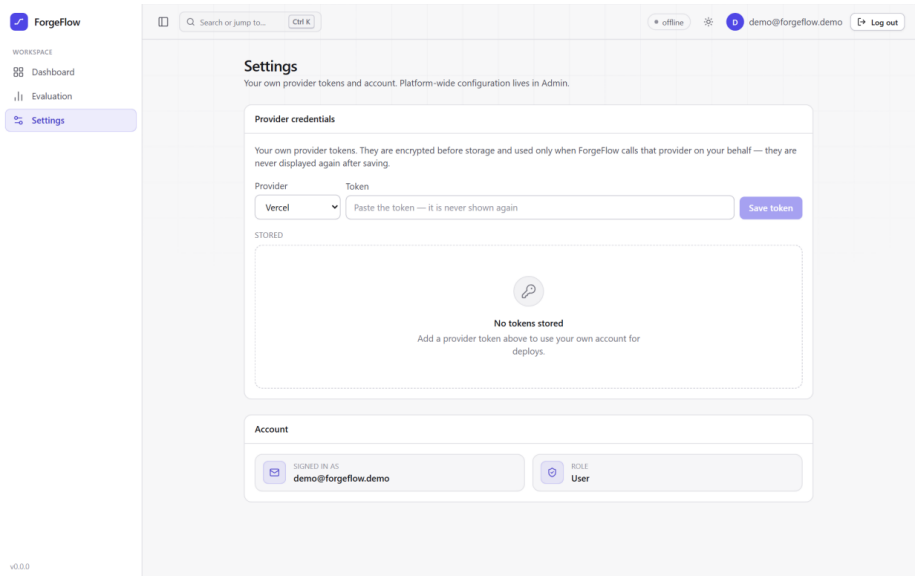


Figure 4.19: Provider credential vault in Settings

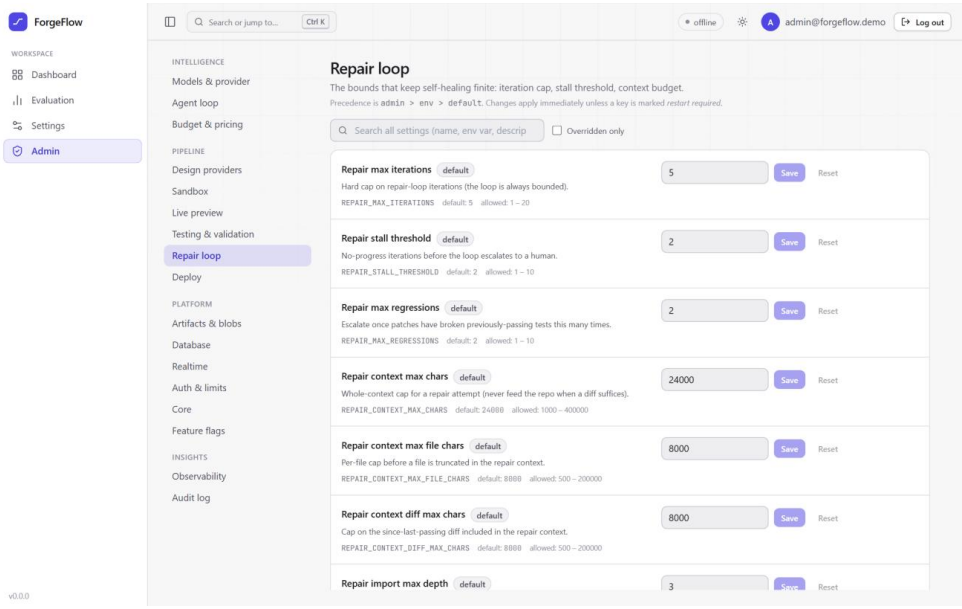


Figure 4.20: Admin dashboard: repair-loop bounds with their sources and allowed ranges

4.14 Platform Delivery: CI/CD, Containerisation, Orchestration and Monitoring

ForgeFlow holds generated applications to their tests in the sandbox and again on the live URL. The platform's own releases are held to the same rule. Its backend runs on an Oracle Cloud Ampere A1 virtual machine (3 OCPU, 16 GB, arm64, Ubuntu 24.04) and its frontend on Vercel (Figure 4.21).

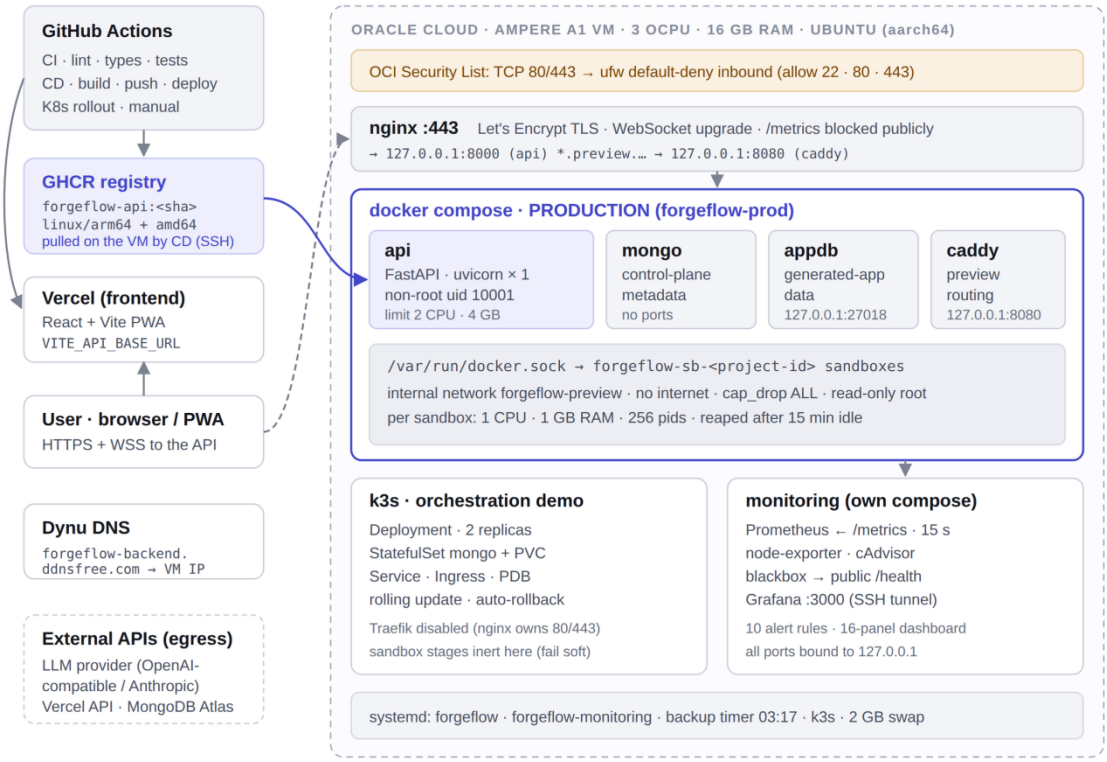


Figure 4.21: Production deployment of the platform: GitHub Actions, GHCR and Vercel, and the Oracle Cloud VM running the production Compose stack, the k3s demonstration and the monitoring stack

4.14.1 Continuous integration and delivery

Continuous integration runs six independent jobs on every push and pull request: the backend job (ruff, black, mypy and pytest with a MongoDB service container), a configuration-layering guard, the sandbox-image build with a Node and Playwright smoke test, a security job (gitleaks over the full history, a committed-secret guard, pip-audit and pnpm audit), the frontend job (ESLint, Prettier, tsc, Vitest, production build and a PWA

check), and an app-skeleton job that builds the generated-app template and runs its unit and Playwright tests.

Continuous delivery is triggered by a successful CI run rather than by the push, so CI passing is a real precondition and not a race (Figure 4.22). A gate job resolves the exact commit that CI tested; a build job produces a multi-architecture (arm64 and amd64) image tagged with the commit SHA and pushes it to GHCR; a deploy job records the currently running image as the rollback target, switches to the new image, polls the container's health check for up to 150 seconds, restores the previous image and fails the run if the new one is unhealthy, and finally checks the public HTTPS health endpoint; the frontend is deployed to Vercel last, so a new interface never points at an old API.

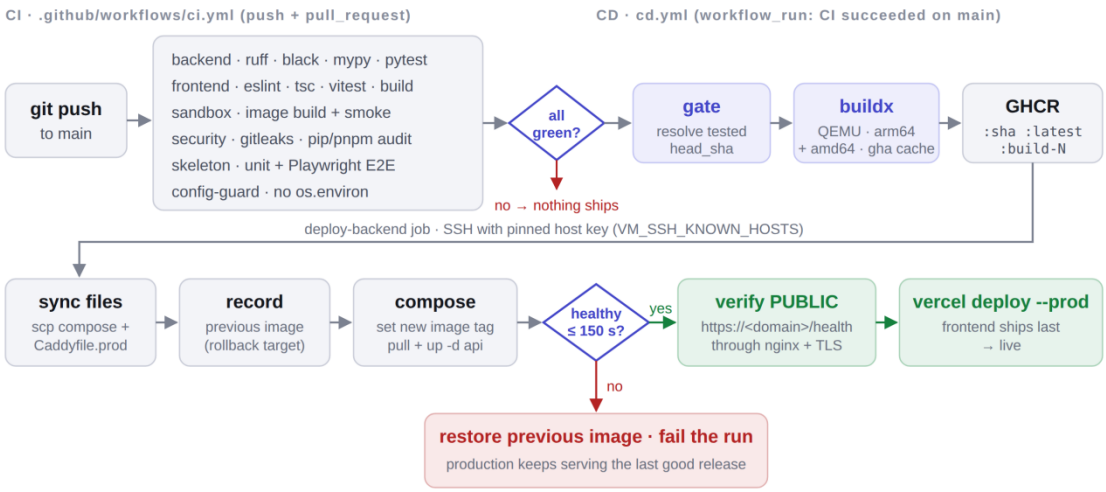


Figure 4.22: CI/CD pipeline: CI must be fully green before CD builds, pushes, deploys, verifies and ships the frontend; an unhealthy release is rolled back automatically

4.14.2 Configuration management, containers and orchestration

An Ansible playbook with seven roles (common, docker, forgeflow, nginx, k3s, monitoring and firewall) provisions the virtual machine idempotently; a second run reports no changes. Secrets such as the encryption key are generated once and read back on later runs, because regenerating the key would make every stored credential unreadable. The firewall role removes Oracle's pre-installed iptables reject rules by specification, which

otherwise silently override ufw. The production image is multi-stage, runs as a non-root user under tini with a health check, and uses a single Uvicorn worker because WebSocket subscribers and rate limiters live in process memory.

Production runs on Docker Compose (api, mongo, appdb and a Caddy preview proxy), while a single-node k3s cluster on the same machine demonstrates Kubernetes orchestration. The split is deliberate: the preview proxy reaches sandboxes by container name through Docker's embedded DNS, which a Kubernetes pod cannot resolve, so moving production to Kubernetes would require changing how the application discovers sandboxes. The k3s deployment runs the real production image with two replicas and a rolling-update strategy of one surge and zero unavailable pods (Figure 4.23).

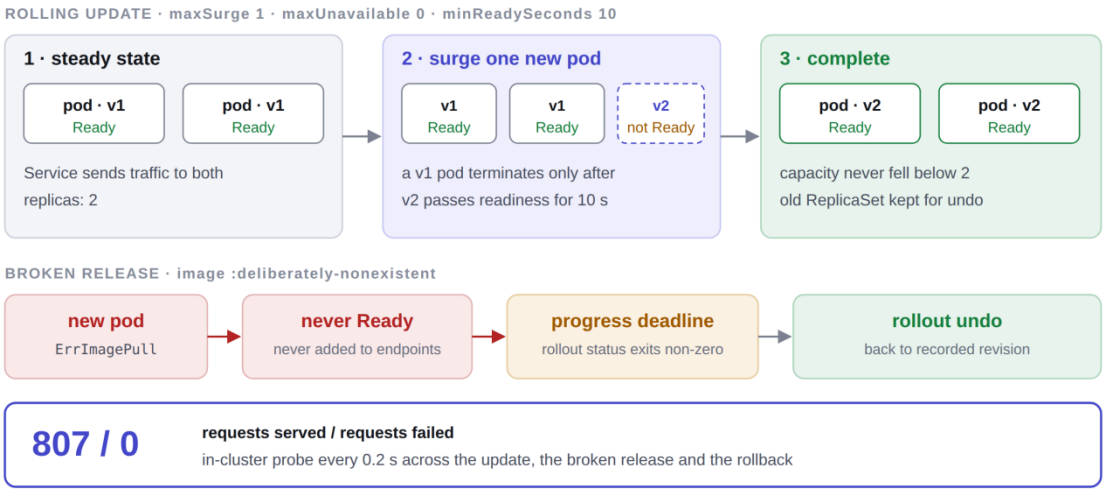


Figure 4.23: Rolling update with maxSurge 1 and maxUnavailable 0, and the broken-release path that ends in an automatic rollback

4.14.3 Monitoring

A separate monitoring stack (Figure 4.24) runs Prometheus with node-exporter, cAdvisor and a blackbox probe of the public health endpoint, a provisioned 16-panel Grafana dashboard covering uptime, latency and error rate, and ten alert rules. The application's /metrics endpoint labels requests by route template rather than raw path, so scanners or

project identifiers cannot create unbounded time series, and the endpoint is blocked from the public internet.

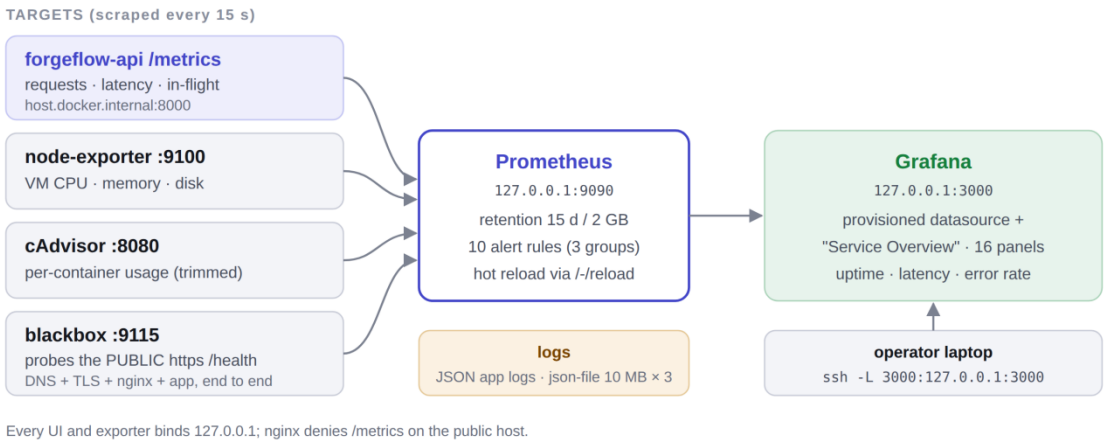


Figure 4.24: Monitoring data flow: targets scraped by Prometheus and visualised in Grafana

4.15 End-to-End Hero Path

Figure 4.25 shows the complete path that the demonstration follows, from screenshots or a specification to a validated live URL. The demonstration application is a todo list with a "Clear completed" bulk action. This action was chosen because first-pass generation often gets it subtly wrong, for example by clearing the whole list or only hiding completed rows, which makes the two "Clear completed" acceptance criteria the moment the bounded repair loop visibly earns its keep.

The sequence has ten numbered steps across four participants: the user, the ForgeFlow control plane, the project's sandbox and the external providers. The user supplies screenshots or a specification (1), and the control plane passes the captured requirements to the design provider to obtain a design (2). It then builds the application in the sandbox by copying the skeleton and generating code (3), while files and logs stream back to the IDE (4), and generates and runs the tests (5). Failing tests enter the bounded, diff-aware repair loop (6), which escalates to the user if a guard trips (7). Otherwise the application

is deployed to the hosting and database providers (8) and validated against its live URL (9), and the user receives the live, validated URL (10).

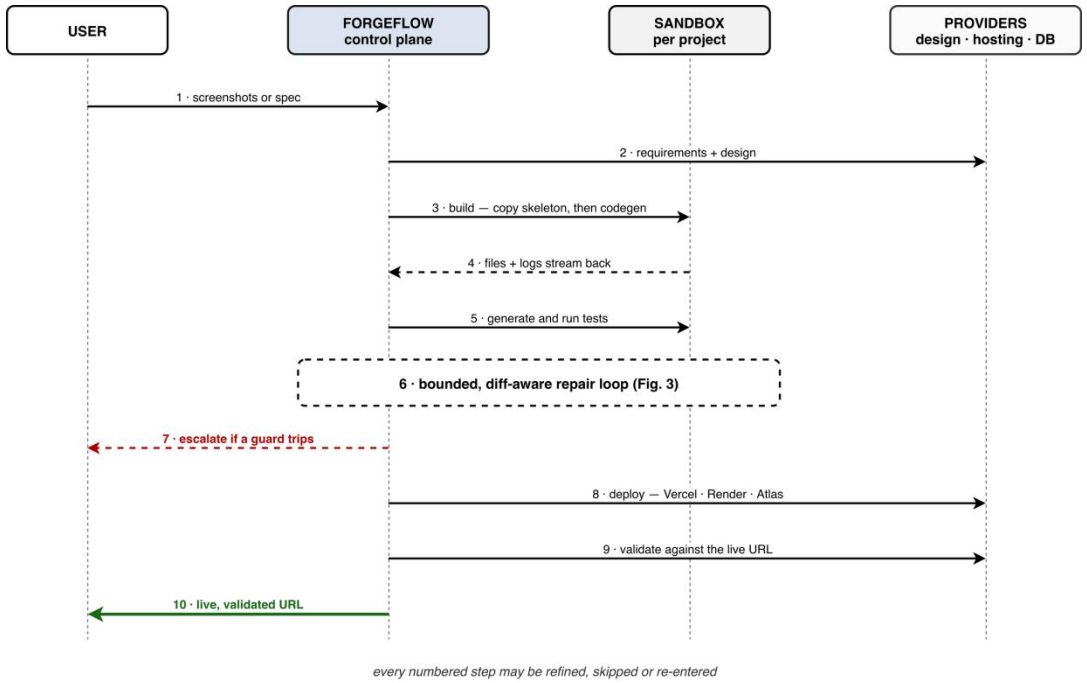


Figure 4.25: Sequence diagram of the hero path. Every numbered step can be refined, skipped or re-entered; the figure shows the original Render backend, which is now an option behind Vercel

Chapter 5
Results and Discussion

This chapter reports how ForgeFlow was tested and what the tests and the project produced. It covers the testing strategy, specific test cases and their results, the automated suite results, the defects found by running the full pipeline on real deployments, the verification of the platform's own delivery pipeline, performance and bounds, the evaluation protocol, the benchmark results, and the project outcomes. Only measured results are reported; where a measurement has not yet been made, the text says so.

5.1 Testing Strategy

Testing was part of every phase: a phase was marked done only when its acceptance criteria passed as automated tests, and every defect found in live use was fixed together with a regression test. External providers (LLM transports, Stitch, Figma, Vercel and Render) are mocked in automated tests, a disposable MongoDB database is used for persistence tests, and wherever the two LLM transports could behave differently, both are tested. Table 5.1 lists the levels of testing.

Table 5.1: Levels of testing used in the project

Level	What is tested	Tools
Unit (pure logic)	State machine, repair-context assembly, import-graph walk, result parsers, infrastructure analyzer, scope classifier, cost accounting	pytest
Integration (API and database)	Conductor legality, authentication, credential vault, data-browser isolation, configuration layering, deploy orchestration with mocked providers	pytest, httpx, test MongoDB
Provider contract	Anthropic and OpenAI-compatible transports, parameter-repair ladder, Stitch question handling, Vercel and Render adapters	pytest with mocked transports
Sandbox and system	Container isolation flags, lifecycle, execution, preview, offline test database (needs a Docker daemon)	pytest, Docker

Level	What is tested	Tools
Frontend component	Stage navigator, build streaming, IDE, deploy topology, data browser, admin settings	Vitest, Testing Library, jsdom
Generated-app template	Skeleton lint, typecheck, build, unit tests and Playwright E2E against the preview	Vitest, Jest, Playwright (CI job)
Security	Sandbox posture, no Docker socket, secret non-disclosure, cross-project access	pytest security suite, gitleaks, audits
End-to-end on real deployments	Full hero path from requirements to a live URL on Vercel and MongoDB Atlas, validated by Playwright	Manual sessions with live providers
Delivery pipeline	Workflows, manifests, playbooks, alert rules, rolling update and rollback, WebSocket proxying	actionlint, kubeconform, ansible-lint, promtool, k3s, nginx

The levels in Table 5.1 progress from fast, isolated checks to slower, more realistic ones. Unit, integration and provider-contract tests run on every push in CI with all external providers mocked. The sandbox, generated-app and delivery-pipeline levels need Docker, a browser or a cluster, and are covered by dedicated CI jobs or by executing the configuration itself (Section 5.5). The end-to-end level, run against live providers, is the slowest and the most realistic, and it is where the defects reported in Section 5.4 were found.

5.2 Test Cases and Results

Table 5.2 lists representative test cases from the backend suite, chosen to cover each requirement area. Each row names the actual test function in the repository. All of them passed in the full-suite run reported in Section 5.3.

Table 5.2: Representative test cases and results

ID	Module	Test case (function)	Expected result	Result
TC-01	State machine	test_skip_any_stage_sets_skipped	Any stage can be skipped	Pass
TC-02	Conductor	test_deploy_proceed_rejected_before_build_complete	Deploy refused until build is complete	Pass
TC-03	Conductor	test_skip_is_never_blocked_by_hard_prereqs	Skip is always legal	Pass
TC-04	Conductor	test_rejection_leaves_no_message_or_run_side_effects	Illegal intent leaves no trace	Pass

ID	Module	Test case (function)	Expected result	Result
TC-05	Auth	test_password_stored_as_bcrypt_hash	Passwords stored only as bcrypt hashes	Pass
TC-06	Realtime	test_replay_returns_events_after_last_seq	Reconnect resumes from the last sequence number	Pass
TC-07	Repair context	test_context_holds_only_the_implicated_files	Context excludes unrelated files	Pass
TC-08	Repair context	test_diff_runs_from_last_passing_to_head_scoped_to_implicated_files	Diff measured from last green run	Pass
TC-09	Repair context	test_requirement_snippets_join_on_criterion_id	Acceptance text joined by criterion_id	Pass
TC-10	Repair context	test_lowest_priority_file_is_dropped_and_recorded	Budget trims are recorded, never silent	Pass
TC-11	Import graph	test_the_reported_case_the_spec_names_the_component_it_renders	Walk reaches the component under test	Pass
TC-12	Import graph	test_a_node_modules_path_is_unreachable_because_bare_specifiers_never_resolve	Packages never enter the context	Pass
TC-13	Repair loop	test_never_exceeds_max_iterations	Loop stops at the iteration cap	Pass
TC-14	Repair loop	test_stalls_at_the_threshold_not_the_cap	Stall guard trips before the cap	Pass
TC-15	Repair loop	test_two_regressions_stop_the_loop	Regression guard escalates	Pass
TC-16	Repair loop	test_two_no_op_attempts_escalate_with_the_agents_own_explanation	Empty attempts escalate as no_patch	Pass
TC-17	Repair loop	test_a_missing_test_database_escalates_before_any_attempt	Environment failure costs zero attempts	Pass
TC-18	Repair loop	test_every_iteration_is_recorded_as_an_attempt	Full audit trail persisted	Pass
TC-19	Build	test_environment_failures_never_reach_the_repair_loop	Sandbox failures short-circuit	Pass
TC-20	Build	test_red_typecheck_is_repaired_then_reverified_to_complete	Self-heal then re-verify	Pass
TC-21	Infra analyzer	test_full_stack_plan_covers_all_three_tiers	FE, BE and DB tiers detected	Pass
TC-22	Infra analyzer	test_a_secret_shaped_frontend_var_is_flagged_as_a_leak	VITE_ secret flagged	Pass
TC-23	Vault	test_value_is_ciphertext_at_rest	Tokens stored encrypted	Pass
TC-24	Vault	test_there_is_no_endpoint_that_reads_a_secret_back	No secret read-back	Pass
TC-25	Data browser	test_two_projects_cannot_see_each_other_s_data	Per-project isolation	Pass
TC-26	Sandbox security	test_the_docker_socket_is_never_mounted	No Docker socket in sandboxes	Pass
TC-27	Live validation	test_egress_is_handed_back_even_when_the_suite_explodes	Egress always detached	Pass
TC-28	LLM transport	test_a_model_that_rejects_max_tokens_completes_after_one_repair	Parameter repaired, call succeeds	Pass
TC-29	Budget	test_project_cap_halts_with_error_and_event	Spending halts at the cap	Pass
TC-30	Evaluation	test_one_specs_requirements_never_leak_into_another	Benchmark specs isolated	Pass

The cases are weighted towards the repair context and the repair loop (TC-07 to TC-18) because the bounds on that loop are the project's central claim: the iteration, stall, regression, empty-attempt and environment guards of Algorithm 4.1 each have a test that drives the loop into that guard (TC-13 to TC-17). The remaining cases cover the stage model, authentication, realtime replay, the build, infrastructure analysis, credential secrecy, project isolation, live validation, the LLM transport and spending limits.

5.3 Automated Test-Suite Results

Both automated suites were run in full on 2 October 2026 on a Windows 11 development machine, with a local MongoDB server providing the backend's disposable test databases and the Docker daemon stopped. Table 5.3 gives the results next to the run reported at Review 3 on 19 September 2026.

Table 5.3: Automated test-suite results

Suite	Collected	Passed	Failed	Skipped	Wall time
Backend (pytest), 2 Oct 2026	1,815	1,745	0	70	653 s
Frontend (Vitest, 61 files), 2 Oct 2026	508	508	0	0	167 s
Total, 2 Oct 2026	**2,323**	**2,253**	**0**	**70**	–
Backend (pytest), 19 Sep 2026	1,713	1,642	2	69	674 s
Frontend (Vitest), 19 Sep 2026	506	506	0	0	331 s

All 2,253 executed tests passed and none failed. The 70 skipped backend tests are skipped by design on this host: 66 exercise POSIX-only behaviour (pseudo-terminals, process groups and pgrep) and run on the Linux CI runners, and 4 are opt-in tests that need a Docker daemon with the sandbox image or the real pnpm toolchain, which the CI sandbox-image and app-skeleton jobs cover. In the Review 3 run two tests had failed because the

Docker daemon was off; in the current run, also without Docker, no test failed. Collected counts are higher than the number of test definitions because parametrised cases are counted individually.

Figure 5.1 shows how the test definitions are distributed across the code. The agents (code generation, test generation, repair and their context builders), deployment, orchestration and sandbox modules carry the most tests, matching where the system's risk lies. Figure 5.2 shows how the suite grew as defects found in live use were fixed together with regression tests; test definitions are counted the same way at every point (each def test_ in Python and each it(or test(in TypeScript), so parametrised cases make the runner's collected count higher.

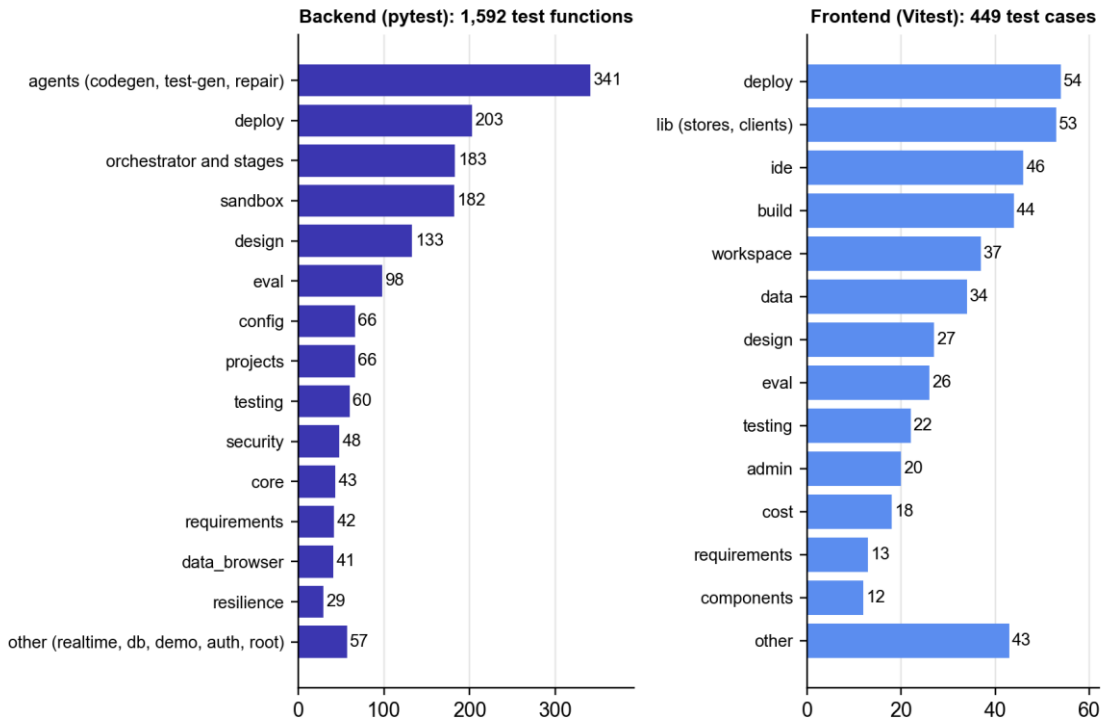


Figure 5.1: Distribution of automated test definitions by module (2 October 2026)

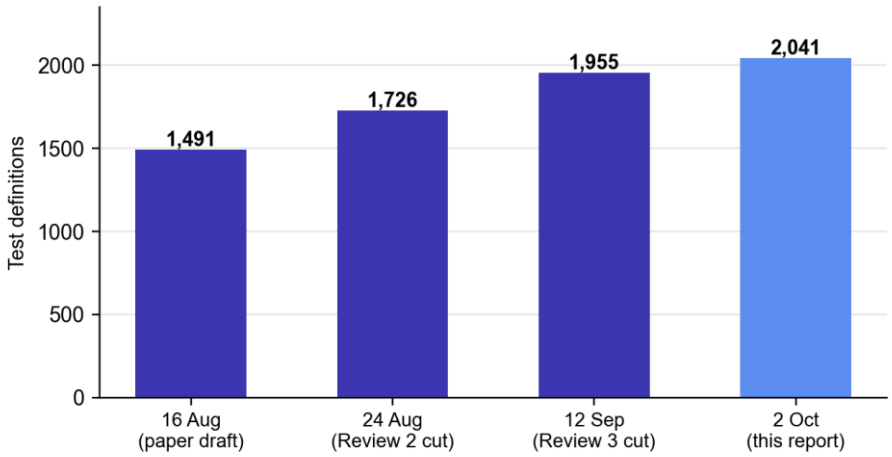


Figure 5.2: Growth of the automated test suite (test definitions)

Static quality gates were run alongside the tests: ruff and black for Python formatting and linting, strict TypeScript compilation, ESLint with zero warnings and Prettier for the frontend, and mypy for Python types. At Review 3 (19 September) mypy reported four type errors in two new test files; the CI gate caught them, and they were fixed the same day, before continuous deployment was enabled.

5.4 Debugging: Defects Found by End-to-End Validation

The most valuable debugging came from running the full pipeline on real deployments, with the frontend and backend on Vercel and data on MongoDB Atlas. Each defect found was traced to a root cause, fixed, and covered by regression tests; several led to whole new phases. Table 5.4 lists them.

Table 5.4: Defects found in live use, their root causes and fixes

Observed symptom	Root cause	Fix now in place	Phase
Backend could never deploy to Render	Render clones a git repository, but nothing pushed the sandbox workspace anywhere	Backend moved to Vercel using the same inline file upload as the frontend	58
Model call failed with “max_tokens is not supported”	Request shape assumed one vendor's parameter names	Transport reads the provider's 400, repairs the request and remembers the working shape	57
Environment variables shown as set but absent on Vercel	set_env was built and unit-tested but never wired	Variables mirrored to the provider project; readiness strip shows presence and last 4 characters	62

Observed symptom	Root cause	Fix now in place	Phase
“Logs” tab showed ForgeFlow's own step log	Provider deployment ids were never recorded	Ids recorded; real provider build and runtime logs shown	62
No way to delete a deployment	destroy was never wired	Teardown returns deploy to empty, restoring validate after deploy	62
Localhost database URI shipped to a function that timed out every query	No pre-flight reachability check	Database step refuses a localhost cluster URI before deploying	62
Repair wrote nothing, twice, then stalled	Directory-shaped context missed the component under test	Import-graph walk, per-test coverage, bounded file request, no_patch escalation	63
Report said 8 of 8 phases complete but the preview showed a placeholder	Output-token cap cut off reasoning; empty phases passed a trivial typecheck	Reasoning-aware loop, evidence-gated phases, structural feature-code check	64
Model concluded it could not download mongod with no network	Test database downloaded its binary on first use	mongod baked into the sandbox image; useTestDb() helper; offline failures detected	65
Build loop spent its budget and still failed	Skeleton demo tests asserted text the build gate forbids; tests are read-only to repair	Zero-iteration stop that names the stale test files	–
Preview failed with EAI_AGAIN after a Docker restart	Database and proxy did not restart with the sandbox	Unresolvable host detected, diagnosed and dependencies restarted	–
A design nobody asked for appeared after the provider asked a question	Question treated as an error, so fallback ran	Question parked; stage awaiting_user; answer resumes the same provider	–
A live-validation run appeared as the Test stage's latest result	Shared collection with an unscoped listing	Sandbox runs shown by default; explicit environment filter	–

Two patterns stand out. First, several defects were features that had been built and unit-tested but never connected to the rest of the system (set_env, destroy); only an end-to-end run against a real provider exposed them. Second, several defects were cases of the system reporting success it had not earned (phases marked done without writing code, environment variables shown as set). The fixes consistently replaced an optimistic signal with evidence, which is the same principle the project applies to generated applications.

5.5 Platform Delivery Verification

Every DevOps configuration was verified by executing it, not only by inspecting it, and the raw output is kept in the repository under docs/devops/evidence. Table 5.5 summarises the results.

Table 5.5: Verification of the delivery pipeline and infrastructure

Check	Tool	Result
Workflow syntax and embedded shell	actionlint with shellcheck	3 workflows, 0 findings
Kubernetes manifests	kubeconform -strict (Kubernetes 1.31)	9 of 9 valid
Ansible playbook	ansible-lint, production profile	0 failures, 0 warnings (30 files)
Prometheus configuration and rules	promtool check	Valid, 10 rules
nginx configuration	nginx -t	Valid, with and without the preview block
WebSocket upgrade through nginx	Live handshake	101 Switching Protocols (negative control: 200 OK)
Production image	docker build and run	Healthy, non-root uid 10001
/metrics content	Live scrape	Route templates only, no secrets
Rolling update on real k3s	In-cluster probe every 0.2 s	807 requests, 0 failures
Deliberately broken release	Real k3s	Detected and automatically rolled back; served 0 requests
Monitoring stack	Live Prometheus and Grafana	5 of 5 targets up, 16 panels, 10 alert rules
Backend test suite in CI	pytest	1,648 passed

The WebSocket test illustrates why execution matters. A standard nginx proxy block does not forward the upgrade handshake; with the three required directives the handshake returns 101 Switching Protocols, and without them it silently returns 200 OK. Every REST call still works in the broken case, so the only symptom would be that live build and test streaming never connects. Running the negative control is what proves the directives are needed.

5.6 Performance and Bounds

ForgeFlow's worst-case behaviour is bounded by construction, independently of how well the model performs. Table 5.6 lists the guaranteed bounds and the measurements made so far.

Table 5.6: Worst-case bounds and measured performance

Scope	Bound or measurement	Basis
Repair loop	At most 5 iterations; stops at 2 stalls, 2 regressions or 2 empty attempts	Gains concentrate in iterations 1 to 4
Repair context	At most 24,000 characters (8,000 per file, 8,000 of diff); every drop recorded	Context quality over quantity
Import walk	Depth at most 3, at most 24 files; at most 3 requested files per attempt	Right file, bounded reach
Build	At most 8 phases, 60 tool turns per phase, 2 typecheck fixes per phase, 3,600 s per build	No unbounded loop anywhere
Environment failures	0 repair iterations and 0 tokens; classified before the loop	Environment failures are the hard class
Live validation	At most 2 repair, redeploy and re-validate cycles	Bounded delivery loop
Cost	Per-project and global caps, warning at 80%	Cost of verification
Backend suite	1,815 tests collected and run in 653 s on the development host (about 0.36 s per test)	Measured
Frontend suite	508 tests across 61 files in 167 s	Measured
Rolling update	807 requests, 0 failures across update, broken release and rollback	Measured on k3s

The upper rows of Table 5.6 are guarantees: they follow from the code and its configuration and hold whichever model is used. The lower rows are measurements on one development host and one cluster, and they show that verification at this depth is affordable in practice: the full backend suite runs in about eleven minutes, the frontend suite in under three, and the rolling update served 807 requests without a single failure.

5.7 Evaluation Harness and Benchmark Protocol

A headless evaluation harness runs the whole pipeline without the user interface on a corpus of benchmark specifications and records, for each specification, the first-pass success rate, the pass rate after repair, the repair delta, the number of repair iterations, regressions introduced, tokens and rupee cost, per-stage and total wall-clock time, and the time from intake to a live URL. Each specification gets its own project and database, and costly legs such as deployment are opt-in. Ten specifications are seeded, from a simple

todo list to an issue tracker and an event scheduler, with 93 acceptance criteria in total (Table 5.7).

Table 5.7: Benchmark specifications

Specification	Difficulty	Features	Criteria	Focus
todo-list	Simple	2	5	CRUD on a single entity
notes-crud	Simple	2	7	CRUD with search
url-shortener	Simple	2	6	Redirects and uniqueness
hero-todo	Moderate	3	7	Bulk action (demonstration)
bookmark-manager	Moderate	3	9	Tags and many-to-many filtering
expense-tracker	Moderate	3	10	Aggregation, categories, dates
kanban-board	Moderate	3	10	Ordering, drag and drop, screenshot input
recipe-box	Moderate	3	9	Nested data, scaling, filtering
issue-tracker	Complex	4	15	Relations, workflow, comments, permissions
event-scheduler	Complex	4	15	Dates, conflict detection, capacity

The planned study has three arms run on the same specifications with repeated runs to report variance: (i) no repair, (ii) the proposed bounded, diff-aware repair, and (iii) bounded repair with the diff-aware context removed. Arm (i) against arm (ii) tests the value of repair; arm (ii) against arm (iii) tests hypothesis H2. Secondary sweeps over the iteration cap and stall threshold will locate the empirical point of diminishing returns and check whether the one-to-four-iteration range reported for function-level benchmarks (Kiecker et al., 2026; Arimbur, 2026) also holds for whole-application generation, which tests H1.

Status. The harness, the specifications and the metric pipeline are implemented and tested (130 evaluation tests). Section 5.8 reports the first scored run: four industry-standard benchmarks in full, and the first two of the ten benchmark applications, after which the run was stopped to stay within budget. The no-repair and context-ablated arms with repeated runs have not yet been run, so this report gives no variance and does not yet

separate the effect of diff-aware context. Three threats to validity are acknowledged. Generated-test pass rates measure agreement between generated code and generated tests, not necessarily the user's intent; the criterion_id chain reduces but does not remove this risk. Model non-determinism requires repeated runs. And results will be specific to one stack and to small and medium CRUD-style applications.

5.8 Benchmark Results

ForgeFlow was evaluated at two levels on 2 October 2026, using a single command (make eval) so that every number in this section can be reproduced from the repository. The first level uses four industry-standard benchmarks that are widely reported for language models that write code and call tools; they run through ForgeFlow's own model interface and its bounded repair policy. The second level runs the benchmark applications of Table 5.7 through the complete pipeline; that run was stopped after the first two applications to stay within the evaluation budget. All runs used the model z-ai/glm-5.3-flash, accessed through OpenRouter and pinned to the model developer's own endpoint, so that every call was served by the same deployment of the model.

5.8.1 Evaluation set-up

The pass@1 metric is the share of problems solved by one greedy sample (temperature 0), graded only by the benchmark's own tests; it is the figure reported on public leaderboards. The second figure applies ForgeFlow's repair policy to each failure: the failing test output is returned to the model for at most five attempts, and repair stops after two attempts that make no progress, which are the defaults of the product's repair loop (Table 4.2). Each attempt sees only the task, the current code and the failure, mirroring the loop's minimal context. As in the self-repair literature (Chen et al., 2024; Olausson et al., 2024), the

feedback comes from the benchmark's tests, so the second figure measures what repair adds and is not a pass@1 result.

Model-written code is executed only inside a container with no network access, all Linux capabilities dropped, a read-only root file system and limits on memory, CPU and processes. Before any model output is scored, the grader itself is checked: 426 of 426 reference solutions pass and 580 of 580 deliberately wrong solutions are rejected, so every pass rate below comes from a grader that accepts correct code and rejects incorrect code. Each dataset is fixed to an upstream commit and verified by a SHA-256 checksum before use. The industry benchmarks made 1,853 model calls costing ₹64.58. All costs in this section are computed from token counts at the provider's list price and ignore prompt-cache discounts, so they are upper bounds.

5.8.2 Code generation: HumanEval, MBPP and HumanEval-TS

HumanEval (Chen et al., 2021) contains 164 Python problems, and MBPP (Austin et al., 2021) contains 257 hand-verified problems in its sanitised test split. HumanEval-TS is the TypeScript translation of HumanEval from MultiPL-E (Cassano et al., 2023); it matches the language of the applications that ForgeFlow generates. Table 5.8 and Figure 5.3 give the results.

Table 5.8: Code-generation benchmark results, pass@1 and after bounded repair

Benchmark	Problems	pass@1	After repair	Gain (pp)	Repaired	Mean attempts	Cost (₹)
HumanEval (Python)	164	97.0%	99.4%	+2.4	4	1.2	14.74
MBPP sanitized (Python)	257	96.1%	98.8%	+2.7	7	1.3	19.82
HumanEval-TS (TypeScript)	159	97.5%	98.7%	+1.3	2	1.0	13.59

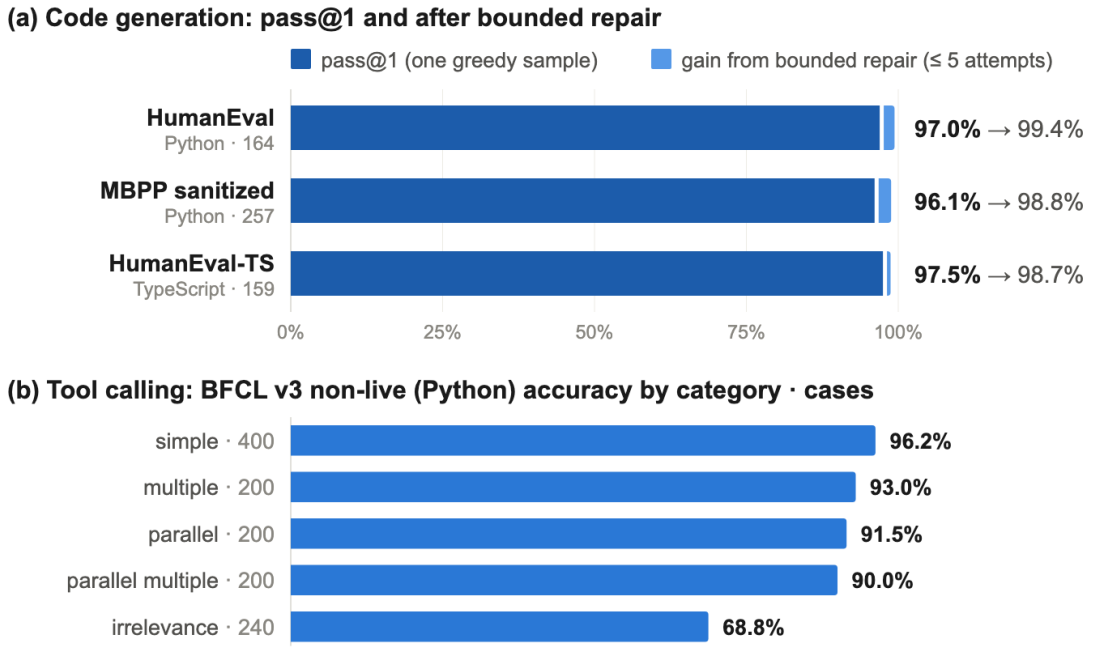


Figure 5.3: Industry benchmark results: code generation with and without bounded repair (top) and BFCL tool-calling accuracy by category (bottom)

The model solved 97.0% of HumanEval, 96.1% of MBPP and 97.5% of HumanEval-TS at the first attempt. Of the 19 problems that failed first time, bounded repair solved 13, with a mean of 1.2 attempts per repaired problem, raising the pass rates by between 1.3 and 2.7 percentage points. Of the remaining failures, six stopped early because two attempts made no progress and none reached the five-attempt cap, so no problem consumed more than the configured budget. No item was lost to a provider error. Because these function-level problems are short and self-contained, the absolute numbers say more about the model than about ForgeFlow. The contribution measured here is the repair policy, which turned 13 of 19 first-attempt failures (68%) into passes and stopped on its own on the rest.

5.8.3 Tool calling: Berkeley Function-Calling Leaderboard

The Berkeley Function-Calling Leaderboard (Patil et al., 2025) measures whether a model calls the right function with correctly typed arguments, which is what ForgeFlow's agents do in every build and repair turn. Its four non-live abstract syntax tree (AST) categories

cover a single function, a choice among several functions, several calls in parallel, and both together; the irrelevance category checks that the model calls nothing when no function fits. The functions are offered as tools through the same transport that the build agents use, the request follows the official harness, and the calls are graded by a port of the official AST checker (Table 5.9).

Table 5.9: BFCL v3 non-live (Python) accuracy, scored by the official AST checker

Category	Cases	Correct	Accuracy
simple	400	385	96.2%
multiple	200	186	93.0%
parallel	200	183	91.5%
parallel multiple	200	180	90.0%
Mean of the four AST categories	**1,000**	**–**	**92.7%**
Irrelevance (correctly calls nothing)	240	165	68.8%

Across the 1,000 AST cases the mean accuracy was 92.7%; the weakest category was parallel multiple (90.0%), and the model correctly declined to call a function in 68.8% of the irrelevance cases. This is the clearest weakness the benchmark shows: when no offered function fits, the model still calls one in about 31% of cases, which for an agent means acting where it should ask. ForgeFlow's approval gates and human-in-the-loop stages are the mitigation, and a tool-use prompt that rewards abstaining is a cheap follow-up. The mean covers only the Python non-live categories and is therefore not comparable with the leaderboard's overall score, which also includes Java, JavaScript, live and multi-turn categories.

5.8.4 End-to-end application generation

Each specification in Table 5.7 is run once through requirements, build, test generation, testing and the bounded repair loop, each in its own project, sandbox and database. The evaluation runs the product's own test-generation agent after the build, so that every

acceptance criterion has tests tagged with its identifier. A criterion counts as met only when at least one of its tests ran and none failed; a criterion without a test counts as unmet. An application counts as delivered only when all of its tests pass and every acceptance criterion is met. The run was stopped after two of the ten applications had completed, to keep the evaluation within its budget; Table 5.10 reports those applications, and the discussion below does not generalise from them.

Table 5.10: End-to-end results on the benchmark applications completed (one run each)

Specification	Difficulty	Criteria met: first → repaired	Repair attempts	Tokens	Cost (₹)	Time (min)	Outcome
notes-crud	Simple	5/7 → 5/7	0	1.4M	20.8	34	Escalated
bookmark-manager	Moderate	7/9 → 7/9	0	1.6M	24.9	44	Escalated

The generated tests covered all 16 acceptance criteria of the two applications, and 12 criteria (75%) were met at the first test run. Every unit test passed (78 of 78), while only 2 of 13 Playwright end-to-end tests did, so every unmet criterion was an end-to-end one. The bounded repair loop made no attempt on either application, and this is by design rather than a failure of the loop: an end-to-end test drives the browser and imports none of the application's source, so the minimal repair context (Section 4.9) contains no editable file and the loop escalates instead of guessing. Repair therefore helps where a failing test reaches the code it exercises, and mapping an end-to-end failure to the pages and routes it visits is the clearest next improvement. One caution applies to the end-to-end numbers themselves: in an isolated re-run of a third application's end-to-end suite, both of its tests passed in 2.3 seconds, so some of these failures may come from building four applications at once on one machine rather than from the generated code. Each application took between 34 and 44 minutes and about 1.5 million tokens.

5.8.5 Defects found by running the evaluation

Running the evaluation end to end exposed five defects that no unit test had caught. Each was fixed and is covered by a regression test:

- **Test generation was not wired in.** The test-generation agent was implemented and tested but never called by the pipeline, so test pass rates measured only the skeleton's placeholder tests, which pass whatever is built. The evaluation now runs it before the first test run and scores acceptance criteria rather than tests.
- **Detached builds were not awaited.** The build stage returns as soon as a build is accepted and finishes in the background, so the harness measured a half-built workspace and raced the build for its sandbox. It now waits for the build to finish.
- **Cheapest-first routing chose a slow, 4-bit host.** Sorting OpenRouter's upstream providers by price sent requests to a host serving a 4-bit quantised copy of the model at about three tokens per second, so a single planning call ran for over half an hour. A new setting pins requests to named upstreams without fallback, and every benchmark run uses it.
- **End-to-end browsers crashed inside the sandbox.** Playwright sizes its worker pool from the host's processor count rather than the sandbox's single-CPU limit, and each Chromium instance needs about a hundred threads, which count against the sandbox's 256-process limit. With two workers the sandbox peaked at 247 processes; with three, every browser failed at launch, so every end-to-end criterion failed for a reason the generated code never caused. The application skeleton now runs one browser at a time, which peaked at 142 processes.
- **Network timeouts inside a stream were not retried.** A read timeout raised while a response was streaming escaped the retry logic and failed the whole build. It is now retried like any other transient error, or reported as a provider error once output has been produced.

All results in this section are reproduced with make eval, which also writes a report of the latest run (eval/results/REPORT.md) and a JSON record of every item. The developer guide lists what must be in place first: Docker, MongoDB, the sandbox image, the model settings in the environment file, and the agents' loop bounds at their documented defaults.

The last condition matters: with the tool-turn cap left at 16 instead of 250, test generation stopped while still reading the code and wrote no tests, so every acceptance criterion scored as unmet.

5.9 Project Outcomes

Table 5.11 compares the outcomes planned at the start of the project with what had been achieved by 2 October 2026. The supporting evidence is collected in the appendices.

Table 5.11: Planned and achieved project outcomes

Planned outcome	Achieved	Evidence
Working end-to-end platform	All 65 phases shipped; hero path verified on real Vercel and Atlas deployments	Chapter 4; Section 5.4
Bounded, diff-aware repair loop	Implemented with five guards, revert of harmful attempts, escalation and full audit trail	Section 4.9; TC-13 to TC-18
Live deployment of the platform	Backend on Oracle Cloud with health-checked CD and rollback; frontend on Vercel	Section 4.14; Table 5.5
Automated quality gates	Six-job CI with 21 gates on every push and pull request	Section 4.14
Research publication	IEEE conference paper submitted; under review	Appendix D
Intellectual property	Invention disclosure revised after six-point IP review and submitted for filing	Appendix F
Competition participation	ForgeFlow submitted to the IBM Bob 2.0 Hackathon (lablab.ai, 25–27 September 2026)	Appendix E
Quantitative evaluation	Industry benchmarks complete: 97.0% pass@1 on HumanEval, 99.4% with bounded repair; BFCL 92.7%. 2 of 10 applications run; ablation arms pending	Sections 5.7–5.8

Seven of the eight planned outcomes in Table 5.11 have been achieved, although the paper is still under review and the invention disclosure has been submitted but not yet filed. The exception is the quantitative evaluation, which is complete for the industry benchmarks but covers only two of the ten benchmark applications and none of the ablation arms; it is therefore reported as partial, and completing it is the first item of future work (Section 6.3).

The research paper, titled “ForgeFlow: A Human-in-the-Loop AI Platform for End-to-End Web Application Delivery with a Bounded, Diff-Aware Self-Healing Repair Loop”, is authored by Faheemuddin Sayyed, Sanidhya Awasthi, Raghav Sonchhatra and Dr. Payal Kadam. The invention disclosure is titled “System and Method for Bounded Diff-Aware Self-Healing Software Application Delivery”; in response to the IP review its claims were narrowed to what the system demonstrably does, for example replacing “provable termination” with “deterministically bounded termination under configured guard conditions”.

5.10 Summary of Results and Discussion

The results support three conclusions. First, the system works end to end: all 65 phases were delivered against written acceptance criteria, the full path from requirements to a validated live URL was exercised on real deployments, and all 2,253 automated tests executed on the development host pass, with the remaining 70 POSIX-only or opt-in tests covered by the Linux CI runners. Second, the bounds that make the repair loop safe are enforced by construction and verified by tests: the loop stops at its cap, at a stall, after repeated regressions or empty attempts, and spends no tokens on environment failures. Third, the platform holds itself to the standard it applies to generated applications, with gated CI, health-checked deployment, automatic rollback and monitoring, all verified by execution.

With respect to the hypotheses, the scored run in Section 5.8 gives first evidence for H1: on the code benchmarks, bounded repair added between 1.3 and 2.7 percentage points to pass@1 within at most five attempts and stopped by itself when it made no progress. On the two applications completed, every remaining failure was an end-to-end test outside the repair context, so the loop escalated without an attempt. H1 at the level of whole

applications, and H2, still need the full corpus and the context-ablated arm, which remain the main open items. The live-use defects in Table 5.4 give qualitative support for H3: problems such as a database URI that worked in the sandbox but timed out every query on the deployed function, or environment variables that existed only on one immutable deployment, are invisible to sandbox tests and appear only when the deployed site is exercised. A quantitative test of H3 will compare sandbox and live pass rates in the benchmark study.

Chapter 6
Conclusion and Future Scope

This chapter concludes the report. It summarises what ForgeFlow achieved against the problem and the four main objectives set out in Chapter 1, states the limitations of the current system and of its evaluation, and outlines the work that would carry the project forward.

6.1 Conclusion

Generating a web application with AI is no longer the hard part; knowing whether it works is. The literature from 2025 and 2026 points to the same problem from five directions: AI-generated code carries systematic quality and security defects, iterative self-improvement can make code worse, generated infrastructure often fails to deploy, generated tests lose validity when code changes, and developers do not want to give up control. ForgeFlow is a complete response to this problem: a human-in-the-loop, non-linear platform that takes an idea from screenshots or a specification to a live, validated URL, with a bounded, diff-aware, escalating repair loop at its centre.

The project met its objectives. A non-linear orchestration model lets the user enter, refine, skip, unskip or revisit any stage, with only two enforced orderings. Agentic generation produces applications on a fixed stack from a copied skeleton, with phase-by-phase construction and evidence-gated verification. Structured requirements compile into unit and end-to-end tests linked to their acceptance criteria. The repair loop assembles a minimal context from the failing tests, the files they import and the diff since the last passing run, and it stops at the first of five guards, reverts harmful attempts, and hands the user an auditable summary when it cannot converge. Deployment analyses infrastructure needs, deploys in order, reports truthfully and is followed by validation on the live URL.

Untrusted code runs only in hardened per-project sandboxes, and every model call is costed and capped.

The prototype consists of about 50,000 lines of production code and more than 2,000 automated tests, built over 65 specified phases. The platform is itself delivered by a gated CI/CD pipeline to Oracle Cloud and Vercel, with automatic rollback and monitoring. The work produced an IEEE conference paper now under review, an invention disclosure submitted for filing and a hackathon submission. On industry benchmarks run through its own model interface it reached 97.0% pass@1 on HumanEval (99.4% with bounded repair) and 92.7% on the BFCL function-calling categories. The distinguishing commitment of ForgeFlow is a negative one: rather than promising that it can always fix an application, the system knows when to stop, and it returns the problem to the human together with an account of what it tried.

6.2 Limitations

- **Partial, single-run evaluation.** The industry benchmarks in Section 5.8 are complete, but only two of the ten benchmark applications were run, once each and with one model, so the application results report no variance; the context-ablated arm needed to test H2 has not been run, and the repair loop cannot yet act on end-to-end failures.
- **One generated-app stack.** Restricting output to React, Express and MongoDB makes verification tractable but limits applicability.
- **Design fidelity.** Design intake inherits the visual-fidelity limits of current multimodal models, which is why the design stage is refinable and skippable.
- **Oracle quality.** Generated tests check agreement with generated code; a test can satisfy the letter of an acceptance criterion but not its intent.
- **Single-node production.** The control plane runs as one worker on one virtual machine because WebSocket subscribers and rate limiters are held in process

memory, and sandbox-backed stages do not run on the Kubernetes demonstration cluster.

- **Scope.** Mobile, desktop and native targets, team collaboration, payments in generated applications and enterprise compliance are outside the scope of this version.

6.3 Future Scope

1. **Complete the ablation study.** Run all ten applications in the no-repair, bounded diff-aware and context-ablated arms, with repeated runs and more than one model, and report convergence, cost per success and latency with variance. Give the repair loop end-to-end context by mapping a failing browser test to the pages and routes it visits.
2. **Expand the corpus.** Grow the benchmark from 10 to 20–30 specifications of increasing difficulty and sweep the iteration cap and stall threshold to find the empirical point of diminishing returns for whole applications.
3. **Learned stopping.** Replace the fixed iteration cap with a stopping criterion learned from the trajectory of the failing set, while keeping a hard upper bound.
4. **Repository-level maintenance.** Test whether the bounded loop transfers to existing codebases, where the diff signal is richer but a regression has a larger blast radius.
5. **Horizontal scaling.** Move realtime subscribers and rate-limiter state to a shared store such as Redis, run multiple control-plane replicas, and add a Kubernetes-native sandbox runtime.
6. **More stacks and providers.** Add further generated-app stacks behind the same skeleton-and-verification pattern, and further deployment providers behind the existing DeployProvider interface.
7. **Alert routing and release automation.** Route Prometheus alerts to e-mail or chat, and build the sandbox image automatically in continuous deployment.

References

Alian, P., Nashid, N., Shahbandeh, M., Shabani, T., & Mesbah, A. (2025). Feature-driven end-to-end test generation. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 450–462). IEEE. https://doi.org/10.1109/ICSE55347.2025.00141

Alturayeif, N., Ahmad, I., & Hassine, J. (2026). TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability. *Requirements Engineering, 31*(1), Article 6. https://doi.org/10.1007/s00766-026-00460-1

Arimbur, J. J. (2026). *How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks* (arXiv:2604.10508). arXiv.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). *Program synthesis with large language models* (arXiv:2108.07732). arXiv. https://doi.org/10.48550/arXiv.2108.07732

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity* (arXiv:2507.09089). arXiv.

Bouzenia, I., Devanbu, P. T., & Pradel, M. (2025). RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 2188–2200). IEEE. https://doi.org/10.1109/ICSE55347.2025.00157

Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C. J., Feldman, M. Q., Guha, A., Greenberg, M., & Jangda, A. (2023). MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering, 49*(7), 3675–3691. https://doi.org/10.1109/TSE.2023.3267446

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., … Zaremba, W. (2021). *Evaluating large language models trained on code* (arXiv:2107.03374). arXiv. https://doi.org/10.48550/arXiv.2107.03374

Chen, X., Lin, M., Schärli, N., & Zhou, D. (2024). Teaching large language models to self-debug. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., & Wang, X. (2025). LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 8697–8727). Association for Computational Linguistics.

CloudBees. (2026). *The state of code abundance 2026*. CloudBees, Inc. https://www.cloudbees.com/lp/2026-state-of-code-abundance-report

DORA. (2025). *State of AI-assisted software development 2025*. Google Cloud. https://dora.dev/dora-report-2025/

Ehsani, R., Pathak, S., Rawal, S., Mujahid, A. A., Imran, M. M., & Chatterjee, P. (2026). Where do AI coding agents fail? An empirical study of failed agentic pull requests in GitHub. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. arXiv:2601.15195

Fang, Z., Jin, D., Dong, Y., Li, Y., Zhang, K., Jin, Z., & Li, G. (2026). *ClarifyCodeBench: Evaluating LLMs on clarifying ambiguous requirements for code generation* (arXiv:2607.00711). arXiv.

Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., Wang, L., Yao, J., Liu, T., Cai, Y., Bi, B., Guo, F., Guo, J., Liu, S., & Cheng, X. (2025). *A survey of vibe coding with large language models* (arXiv:2510.12399). arXiv.

Ghaleb, T. A. (2026). When AI agents touch CI/CD configurations: Frequency and success. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. arXiv:2601.17413

Ghammam, A., & Almukhtar, M. (2026). AI builds, we analyze: An empirical study of AI-generated build code quality. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)* (pp. 727–731). ACM. https://doi.org/10.1145/3793302.3793563

Gu, S., Nashid, N., & Mesbah, A. (2025). *LLM test generation via iterative hybrid program analysis* (arXiv:2503.13580). arXiv.

Haroon, S., Khan, M. T., & Gulzar, M. A. (2026). *Evaluating LLM-based test generation under software evolution* (arXiv:2603.23443). arXiv.

Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). *Agentic software engineering: Foundational pillars and a research roadmap* (arXiv:2509.06216). arXiv.

Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., & Cui, H. (2024). *AgentCoder: Multi-agent-based code generation with iterative testing and optimisation* (arXiv:2312.13010). arXiv.

Huang, K., Zhang, J., Bao, X., Wang, X., & Liu, Y. (2025). Comprehensive fine-tuning large language models of code for automated program repair. *IEEE Transactions on Software Engineering, 51*(4), 904–928. https://doi.org/10.1109/TSE.2025.3532759

Huang, R., Reyna, A., Lerner, S., Xia, H., & Hempel, B. (2025). *Professional software developers don't vibe, they control: AI agent use for coding in 2025* (arXiv:2512.14012). arXiv.

Jiang, Z., Lo, D., & Liu, Z. (2025). *Agentic software issue resolution with large language models: A survey* (arXiv:2512.22256). arXiv.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Kiecker, T., Reichmann, E., Kang, H., An, G., & Grunske, L. (2026). *Is three the magic number? An empirical evaluation of LLM-based repair loops* (arXiv:2607.05197). arXiv.

Li, H., Zhang, H., & Hassan, A. E. (2025). *The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering* (arXiv:2507.15003). arXiv.

Muna, R. K., Rafi, M. N., & Chen, T.-H. (2026). *CI-Repair-Bench: A repository-aware benchmark for automated patch validation via CI workflows* (arXiv:2604.27148). arXiv.

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., & Solar-Lezama, A. (2024). Is self-repair a silver bullet for code generation? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2025). RouteLLM: Learning to route LLMs from preference data. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Ouédraogo, W. C., Kaboré, A. K., Li, Y., Tian, H., Koyuncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2026). Prompt engineering in LLMs for automated unit test generation: A large-scale study. *Empirical Software Engineering, 31*(4), Article 103. https://doi.org/10.1007/s10664-026-10840-4

Ouyang, S., Yu, W., Ma, K., Xiao, Z., Zhang, Z., Jia, M., Han, J., Zhang, H., & Yu, D. (2025). RepoGraph: Enhancing AI software engineering with repository-level code graph. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Pasuksmit, J., Takerngsaksiri, W., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Wang, S., Jiang, F., Li, J., Cook, E., Chen, K., & Wu, M. (2025). Human-in-the-loop software development agents: Challenges and future directions. In *Proceedings of the IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)* (pp. 756–757). IEEE. https://doi.org/10.1109/MSR66628.2025.00112

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267). PMLR. https://proceedings.mlr.press/v267/patil25a.html

Rabin, R., Hostetler, J., McGregor, S., Weir, B., & Judd, J. (2025). *SandboxEval: Towards securing test environment for untrusted code* (arXiv:2504.00018). arXiv.

Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering, 50*(1), 85–105.

Sepidband, M., Pham, H. V., & Hemmati, H. (2026). *On the role of fault localization context for LLM-based program repair* (arXiv:2604.05481). arXiv.

Shang, Y., Zhang, Q., Fang, C., Gu, S., Zhou, J., & Chen, Z. (2025). A large-scale empirical study on fine-tuning large language models for unit testing. *Proceedings of the ACM on Software Engineering, 2*(ISSTA), 1678–1700. https://doi.org/10.1145/3728951

Shukla, S., Joshi, H., & Syed, R. (2025). Security degradation in iterative AI code generation: A systematic analysis of the paradox. In *Proceedings of the 2025 IEEE International Symposium on Technology and Society (ISTAS)* (pp. 1–8). IEEE. https://doi.org/10.1109/ISTAS65609.2025.11269659

Si, C., Zhang, Y., Li, R., Yang, Z., Liu, R., & Yang, D. (2025). Design2Code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of*

the Association for Computational Linguistics: Human Language Technologies (NAACL) (pp. 3956–3974). https://doi.org/10.18653/v1/2025.naacl-long.199

Stack Overflow. (2025). *2025 Developer Survey: AI*. https://survey.stackoverflow.co/2025/ai

Takerngsaksiri, W., Pasuksmit, J., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Jiang, F., Li, J., Cook, E., Chen, K., & Wu, M. (2025). Human-in-the-loop software development agents. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (pp. 342–352). IEEE. https://doi.org/10.1109/ICSE-SEIP66354.2025.00036

Tang, N., Chen, C., Xu, G., Shi, Y., Huang, Y., McMillan, C., Dong, T., & Li, T. J.-J. (2026). *How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions* (arXiv:2605.29442). arXiv.

Tao, W., Zhou, Y., Wang, Y., Zhang, W., Zhang, H., & Cheng, Y. (2024). MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)* (pp. 51963–51993).

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2025). Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering, 2*(FSE), 801–824. https://doi.org/10.1145/3715754

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Zhang, Q., Fang, C., Xie, Y., Zhang, Y., Yu, S., Sun, W., Yang, Y., & Chen, Z. (2026). A survey on large language models for software engineering. *Science China Information Sciences, 69*(4), Article 141102. https://doi.org/10.1007/s11432-025-4670-0

Zhang, T., Pan, S., Zhang, Z., Xing, Z., & Sun, X. (2026). Deployability-centric infrastructure-as-code generation: Fail, learn, refine, and succeed through LLM-empowered DevOps simulation. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. arXiv:2506.05623

Zhao, S., Wang, D., Zhang, K., Luo, J., Li, Z., & Li, L. (2026). Is vibe coding safe? Benchmarking vulnerability of agent-generated code in real-world tasks. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. arXiv:2512.03262

Appendix A: AIC Form

The completed AIC form for this project is attached on the following page, as required for every group.

Attach the completed and signed AIC form here (mandatory).

Appendix B: Similarity Report

The summary page of the Turnitin similarity report, reviewed by the project guide, is attached below. The similarity index must be less than 10%.

Attach the summary page of the Turnitin similarity report here (mandatory).

Appendix C: AI Plagiarism Report

The summary page of the Turnitin AI-writing report, reviewed by the project guide, is attached below.

Attach the summary page of the Turnitin AI plagiarism report here (mandatory).

Appendix D: Research Paper

A research paper describing ForgeFlow and its bounded repair loop has been submitted to an IEEE conference and is under review.

Table D.1: Details of the submitted research paper

Field	Details
Title	ForgeFlow: A Human-in-the-Loop AI Platform for End-to-End Web Application Delivery with a Bounded, Diff-Aware Self-Healing Repair Loop
Authors	Faheemuddin Sayyed, Sanidhya Awasthi, Raghav Sonchhatra and Dr. Payal Kadam (Department of CSE, Symbiosis Institute of Technology, Pune)
Type	IEEE conference paper
Status	Submitted; under review

Attach the first page of the paper here once it is accepted or published.

Appendix E: Project Competition: IBM Bob 2.0 Hackathon

The team took ForgeFlow to an external competition, the IBM Bob 2.0 Hackathon hosted on lablab.ai, and submitted the project.

Table E.1: IBM Bob 2.0 Hackathon participation details

Field	Details
Event	IBM Bob 2.0 Hackathon: “Your code meets your new AI dev partner.”
Organiser and platform	lablab.ai, with IBM (IBM Bob)
URL	https://lablab.ai/ai-hackathons/ibm-bob-2-hackathon
Format	Online hackathon, 48-hour build
Dates	25–27 September 2026 (kick-off Friday 25 September 2026, 8:30 PM IST)
Prize pool	US $12,000
Team members	Faheemuddin Sayyed (faheem219), Sanidhya Awasthi (sanidhya_awasthi), Raghav Sonchhatra (raghav18)
Submission	Forgeflow: “ForgeFlow turns a web-app idea into a verified live URL. It writes tests from your acceptance criteria, fixes failures in a bounded, diff-aware repair loop ...”
Status	Project submitted; event finished

Attach the hackathon participation certificate or a screenshot of the team and submission page here.

Appendix F: Other Project Outcomes

Invention disclosure. An invention disclosure titled “System and Method for Bounded Diff-Aware Self-Healing Software Application Delivery”, naming the three students and Dr. Payal Kadam, was revised in response to a six-point intellectual-property review and submitted for filing. The review led to narrower claims that foreground the technical effects of the invention: sandbox isolation, bounded context construction, diff-based repair, regression detection, deployment validation and credential isolation.

Attach the acknowledgement of the invention disclosure submission here.

Live deployment of the platform. The platform's API is served from https://forgeflow-backend.ddnsfree.com on an Oracle Cloud virtual machine, and its frontend from https://forgeflow-ecru.vercel.app on Vercel, both deployed by the CI/CD pipeline described in Chapter 4.

Certificate from industry. Not applicable: the project was carried out without an industry mentor or industry collaboration.